```
================================================================================
DEFMAN (DEFECT MANAGER) 2004 -> 2026 MODERNIZATION IMPLEMENTATION DOCUMENT
================================================================================
Architecture:
  - Database: MySQL 8.0+ (InnoDB, utf8mb4_unicode_ci, Foreign Keys, Indexes)
  - Backend:  PHP 8.3+ Strict-Typed PDO REST API, Argon2id Auth, RBAC Middleware
  - Frontend: React 19 (Vite + TypeScript) SPA
  - Graphics: PixiJS v8 (WebGPU / WebGL2) Visual Defect Triage & Cost Canvas

Legacy 2004 File Tree Mapping Summary:
  - install/installdb.php, install-part1.php, install-part2.php
      -> modern/database/schema.sql + DatabaseSeeder.php
  - config.php, dbconnect.php, db.php
      -> Database.php + User/Product/Defect/Analysis Repositories
  - defecttype.txt & faulttypes.html
      -> PHP 8.3 IeeeFaultType Backed Enum + TypeScript IeeeFaultType Enum
  - login.php, logout.php, randompassword.php, form_lostpassword.php,
    form_newuser.php, form_apply.php
      -> AuthController.php, UserController.php, RbacMiddleware.php
  - form_raportpub.php (+ helpreportpub.html) -> <PublicReportForm />
  - form_raport.php (+ helpreport.html)       -> <CustomerReportForm />
  - form_raportlist.php & form_list.php       -> <DefectReportGrid />
  - form_defrap.php                           -> <DefectDetailModal />
  - form_alist.php                            -> <AnalyzerQueue />
  - form_analyze.php & form_analyzed.php      -> <DefectAnalyzerWorkbench />
  - form_deffix.php & form_defreso.php        -> <DefectResolutionForm />
  - form_usermanagement.php & form_apply.php  -> <AdminUserApproval />
  - form_productdetails.php                   -> <ProductManager />
  - form_userdetails.php                      -> <UserProfileSettings />
  - Visual Triage & Cost Analytics            -> <DefectTriageCanvas /> (PixiJS v8)
================================================================================


================================================================================
SECTION 1.1 MySQL 8.0+ DDL Migration (modern/database/schema.sql)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\database\schema.sql
================================================================================

-- ============================================================================
-- DefMan (Defect Manager) -- Modernized MySQL 8.0+ DDL Schema
-- Refactored from: install/installdb.php, install/install-part1.php, install/install-part2.php
-- Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ============================================================================

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE = 'STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_E
    NGINE_SUBSTITUTION';

-- ----------------------------------------------------------------------------
-- 1. Table: `product` (Refactored from install/installdb.php lines 106-119)
-- ----------------------------------------------------------------------------
DROP TABLE IF EXISTS `analyse`;
DROP TABLE IF EXISTS `raport`;
DROP TABLE IF EXISTS `user_sessions`;
DROP TABLE IF EXISTS `users`;
DROP TABLE IF EXISTS `product`;

CREATE TABLE `product` (
  `id` SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(100) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_product_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- --------------------------------------------------------------------------
-- 2. Table: ‘users‘ (Refactored from install/installdb.php lines 55-82)
-- User Levels:
--   0 = Applying (Pending Admin Approval - form_apply.php)
--   1 = Public   (Unauthenticated / Public reporter - form_raportpub.php)
--   2 = Customer (Authenticated defect reporter - form_raport.php)
--   3 = Analyzer (Developer / QA triage & IEEE fault cost tracking - form_analyze.php)
--   4 = Admin    (Full system access, user approval & product CRUD - form_usermanagement.php)
-- --------------------------------------------------------------------------
CREATE TABLE ‘users‘ (
  ‘id‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘login‘ VARCHAR(64) NOT NULL COMMENT ’Legacy max 6 chars in 2004 UI; widened for modern auth’,
  ‘password‘ VARCHAR(255) NOT NULL COMMENT ’Argon2id password hash (PASSWORD_ARGON2ID)’,
  ‘email‘ VARCHAR(255) NOT NULL,
  ‘userlevel‘ TINYINT UNSIGNED NOT NULL DEFAULT 0 COMMENT ’0=Applying, 1=Public, 2=Customer, 3=A
    nalyzer, 4=Admin’,
  ‘company‘ VARCHAR(100) DEFAULT NULL,
  ‘phone‘ VARCHAR(50) DEFAULT NULL,
  ‘name‘ VARCHAR(100) DEFAULT NULL,
  ‘activate‘ TINYINT(1) NOT NULL DEFAULT 0 COMMENT ’0=Pending/Inactive, 1=Active’,
  ‘logged‘ TINYINT(1) NOT NULL DEFAULT 0 COMMENT ’0=Offline, 1=Online’,
  ‘reset_token_hash‘ CHAR(64) DEFAULT NULL COMMENT ’SHA-256 hash of random_bytes(32) recovery to
    ken’,
  ‘reset_token_expires_at‘ DATETIME DEFAULT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ‘updated_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uq_users_login‘ (‘login‘),
  UNIQUE KEY ‘uq_users_email‘ (‘email‘),
  KEY ‘idx_users_level_activate‘ (‘userlevel‘, ‘activate‘)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------
-- 3. Table: ‘user_sessions‘ (Cryptographically secure bearer/session tokens)
-- --------------------------------------------------------------------------
CREATE TABLE ‘user_sessions‘ (
  ‘id‘ BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘user_id‘ SMALLINT UNSIGNED NOT NULL,
  ‘token_hash‘ CHAR(64) NOT NULL COMMENT ’SHA-256 hash of 256-bit random_bytes token’,
  ‘ip_address‘ VARCHAR(45) DEFAULT NULL,
  ‘user_agent‘ VARCHAR(255) DEFAULT NULL,
  ‘expires_at‘ DATETIME NOT NULL,
  ‘created_at‘ TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (‘id‘),
  UNIQUE KEY ‘uq_user_sessions_token_hash‘ (‘token_hash‘),
  KEY ‘idx_user_sessions_user_id‘ (‘user_id‘),
  KEY ‘idx_user_sessions_expires_at‘ (‘expires_at‘),
  CONSTRAINT ‘fk_user_sessions_user‘
    FOREIGN KEY (‘user_id‘) REFERENCES ‘users‘ (‘id‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------
-- 4. Table: ‘raport‘ (Refactored from install/installdb.php lines 84-104)
-- Preserves 2004 table & column names while enforcing foreign keys & indexes.
-- Note: ‘CustomerID‘ is nullable with ON DELETE SET NULL so deleted users
-- still render as "[DELETED USER]" per form_raportlist.php lines 1002-1012.
-- --------------------------------------------------------------------------
CREATE TABLE ‘raport‘ (
  ‘ID‘ SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
  ‘Product‘ VARCHAR(100) NOT NULL,
  ‘product_id‘ SMALLINT UNSIGNED DEFAULT NULL,
  ‘Environment‘ TEXT DEFAULT NULL,
  ‘CustomerID‘ SMALLINT UNSIGNED DEFAULT NULL,
  ‘Who‘ VARCHAR(64) NOT NULL,
  ‘Contactemail‘ VARCHAR(255) NOT NULL,
```

```
  `Whenreported` VARCHAR(32) NOT NULL COMMENT 'Format: YYYY-MM-DD HH:mm',
  `Title` VARCHAR(255) NOT NULL,
  `Description` TEXT NOT NULL,
  `Discoveredby` VARCHAR(100) DEFAULT NULL,
  `Persontype` ENUM('Customer', 'Developer', 'Tester') NOT NULL DEFAULT 'Customer',
  `Occured` DATE NOT NULL,
  `Location` VARCHAR(255) DEFAULT NULL,
  `Severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `Priority` ENUM('m_urgent', 'average', 'l_urgent') NOT NULL DEFAULT 'average',
  `Submitted` DATE NOT NULL,
  `Addinfo` TEXT DEFAULT NULL,
  PRIMARY KEY (`ID`),
  KEY `idx_raport_product` (`Product`),
  KEY `idx_raport_product_id` (`product_id`),
  KEY `idx_raport_customer` (`CustomerID`),
  KEY `idx_raport_severity_submitted` (`Severity`, `Submitted` DESC),
  KEY `idx_raport_priority_submitted` (`Priority`, `Submitted` DESC),
  KEY `idx_raport_submitted` (`Submitted` DESC),
  CONSTRAINT `fk_raport_product`
    FOREIGN KEY (`product_id`) REFERENCES `product` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT `fk_raport_customer`
    FOREIGN KEY (`CustomerID`) REFERENCES `users` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. Table: `analyse` (Refactored from install/installdb.php lines 21-53)
-- 1:1 extension of `raport` tracking IEEE (1992) fault classification,
-- origin phase, fix effort (hours), fix cost (EUR), and total defect cost (EUR).
-- ------------------------------------------------------------------------
CREATE TABLE `analyse` (
  `id` SMALLINT UNSIGNED NOT NULL COMMENT '1:1 Primary & Foreign Key to raport.ID',
  `devdesc` TEXT DEFAULT NULL COMMENT 'Defect description by developer',
  `classification` VARCHAR(100) DEFAULT NULL COMMENT 'IEEE 1992 Fault Classification (IeeeFaultT
    ype or custom Other)',
  `dateoriginated` DATE DEFAULT NULL,
  `phase` VARCHAR(255) DEFAULT NULL COMMENT 'Comma-delimited phases: Requirements,Design,Impleme
    ntation,Testing,Deployment,Maintenance',
  `actoriginated` TEXT DEFAULT NULL COMMENT 'Activity which originated the defect',
  `deliverable` VARCHAR(255) DEFAULT NULL COMMENT 'Deliverable originated',
  `whattodo` TEXT DEFAULT NULL,
  `submitted` DATE DEFAULT NULL,
  `assigned` DATE DEFAULT NULL,
  `uniqueness` ENUM('New defect', 'Already submitted', 'Critical') NOT NULL DEFAULT 'New defect'
    ,
  `priority` ENUM('most urgent', 'average', 'less urgent') NOT NULL DEFAULT 'average',
  `severity` ENUM('minor', 'major', 'critical') NOT NULL DEFAULT 'minor',
  `workaround` TEXT DEFAULT NULL,
  `proposedfix` TEXT DEFAULT NULL,
  `status` ENUM('submitted', 'assigned', 'postponed', 'opened', 'fixed', 'resolved', 'closed') N
    OT NULL DEFAULT 'submitted',
  `fixdesc` TEXT DEFAULT NULL COMMENT 'Description of fix (form_deffix.php)',
  `fixstarted` DATE DEFAULT NULL COMMENT 'Date work started on fix',
  `fixcomplete` DATE DEFAULT NULL COMMENT 'Date work completed on fix',
  `fiximpact` TEXT DEFAULT NULL COMMENT 'Deliverables impacted by fix',
  `fixeffort` DECIMAL(8,2) UNSIGNED DEFAULT NULL COMMENT 'Effort expended on fix in hours',
  `fixcost` DECIMAL(10,2) UNSIGNED DEFAULT NULL COMMENT 'Monetary cost of fix in EUR',
  `resodesc` TEXT DEFAULT NULL COMMENT 'Description of resolution (form_defreso.php)',
  `resocomplete` DATE DEFAULT NULL COMMENT 'Date of resolution',
  `resoclosed` DATE DEFAULT NULL COMMENT 'Date closed',
  `costofdefect` DECIMAL(10,2) UNSIGNED DEFAULT NULL COMMENT 'Total cost of defect in EUR',
  `actfound` TEXT DEFAULT NULL COMMENT 'Activity that should have found defect',
  PRIMARY KEY (`id`),
  KEY `idx_analyse_status_submitted` (`status`, `submitted` DESC),
  KEY `idx_analyse_severity_submitted` (`severity`, `submitted` DESC),
```

```
  KEY ‘idx_analyse_priority_submitted‘ (‘priority‘, ‘submitted‘ DESC),
  KEY ‘idx_analyse_classification‘ (‘classification‘),
  CONSTRAINT ‘fk_analyse_raport‘
    FOREIGN KEY (‘id‘) REFERENCES ‘raport‘ (‘ID‘)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;
```

```
================================================================================
SECTION 1.2 PHP 8.3 Backed Enum: IeeeFaultType (modern/backend/src/Domain/Enum/IeeeFaultType.php
    )
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Domain\Enum\IeeeFaultType.php
================================================================================
```

```php
<?php
declare(strict_types=1);

namespace DefMan\Domain\Enum;

/**
 * IEEE (1992) Standard Classification for Software Anomalies (IEEE Std 1044-1993 / 1992 draft)
 * Refactored from: defecttype.txt & faulttypes.html
 */
enum IeeeFaultType: string
{
    case LOGIC                  = 'Logic';
    case COMPUTATIONAL          = 'Computational';
    case INTERFACE              = 'Interface';
    case DATA_HANDLING          = 'Data handling';
    case DATA                   = 'Data';
    case DOCUMENTATION          = 'Documentation';
    case DOCUMENT_QUALITY       = 'Document quality';
    case ENHANCEMENT            = 'Enhancement';
    case CAUSED_BY_PREVIOUS_FIX = 'Caused by a prev. Fix';
    case OTHER                  = 'Other';

    /**
     * Hex color palette used by PixiJS v8 WebGPU DefectTriageCanvas & UI badges
     */
    public function hexColor(): int
    {
        return match ($this) {
            self::LOGIC                  => 0xEF4444, // Red-500
            self::COMPUTATIONAL          => 0xF97316, // Orange-500
            self::INTERFACE              => 0xEC4899, // Pink-500
            self::DATA_HANDLING          => 0x8B5CF6, // Violet-500
            self::DATA                   => 0x6366F1, // Indigo-500
            self::DOCUMENTATION          => 0x06B6D4, // Cyan-500
            self::DOCUMENT_QUALITY       => 0x14B8A6, // Teal-500
            self::ENHANCEMENT            => 0x10B981, // Emerald-500
            self::CAUSED_BY_PREVIOUS_FIX => 0xF59E0B, // Amber-500
            self::OTHER                  => 0x64748B, // Slate-500
        };
    }

    /**
     * Detailed sub-classifications from faulttypes.html (IEEE, 1992)
     *
     * @return list<string>
     */
    public function subcategories(): array
    {
        return match ($this) {
```

```
            self::LOGIC => [
                'Forgotten cases or steps',
                'Duplicate logic',
                'Extreme conditions neglected',
                'Unnecessary function',
                'Misinterpretation',
                'Missing condition test',
                'Checking wrong variable',
                'Iterating loop incorrectly',
            ],
            self::COMPUTATIONAL => [
                'Equation insufficient or incorrect: Missing computation',
                'Equation insufficient or incorrect: Operand in equation incorrect',
                'Equation insufficient or incorrect: Operator in equation incorrect',
                'Equation insufficient or incorrect: Parentheses used incorrectly',
                'Precision loss: Rounding or truncation fault',
                'Precision loss: Mixed modes',
                'Sign convention fault',
            ],
            self::INTERFACE => [
                'Interrupts handled incorrectly',
                'I/O timing incorrect: Timing fault causes data loss',
                'Subroutine/module mismatch: Wrong subroutine called',
                'Subroutine/module mismatch: Incorrectly-located subroutine call',
                'Subroutine/module mismatch: Non-existent subroutine called',
                'Subroutine/module mismatch: Inconsistent subroutine arguments',
            ],
            self::DATA_HANDLING => [
                'Initialized data incorrectly',
                'Accessed or stored data incorrectly: Flag or index set incorrectly',
                'Accessed or stored data incorrectly: Packed/unpacked data incorrectly',
                'Accessed or stored data incorrectly: Referenced wrong data variable',
                'Accessed or stored data incorrectly: Data referenced out of bounds',
                'Scaling or units of data incorrect',
                'Dimensioned data incorrectly: Variable type incorrect',
                'Dimensioned data incorrectly: Subscripted variable incorrectly',
                'Scope of data incorrect',
            ],
            self::DATA => [
                'Sensor data incorrect or missing',
                'Operator data incorrect or missing',
                'Embedded data in tables incorrect or missing',
                'External data incorrect or missing',
                'Output data incorrect or missing',
                'Input data incorrect or missing',
            ],
            self::DOCUMENTATION => [
                'Ambiguous statement',
                'Incomplete item',
                'Incorrect item',
                'Missing item',
                'Conflicting items',
                'Redundant items',
                'Confusing item',
                'Illogical item',
                'Non-verifiable item',
            ],
            self::DOCUMENT_QUALITY => [
                'Applicable standards not met',
                'Not traceable',
                'Not current',
                'Inconsistencies',
                'Incomplete',
                'No identification',
            ],
            self::ENHANCEMENT => [
```

```
                'Change in program requirements: Add new capability',
                'Change in program requirements: Remove unnecessary capability',
                'Change in program requirements: Update current capability',
                'Improve comments',
                'Improve code efficiency',
                'Implement editorial changes',
                'Improve usability',
                'Software fix of a hardware problem',
                'Other enhancement',
            ],
            self::CAUSED_BY_PREVIOUS_FIX => [
                'Failure caused by a previous fix',
            ],
            self::OTHER => [
                'Other problem',
            ],
        };
    }

    /**
     * Normalizes legacy case variations (e.g. "Caused by a prev. fix" vs "Caused by a prev. Fix
    ")
     */
    public static function fromLegacyString(string $value): self
    {
        $trimmed = trim($value);
        foreach (self::cases() as $case) {
            if (strcasecmp($case->value, $trimmed) === 0) {
                return $case;
            }
        }
        return self::OTHER;
    }
}
```

```
================================================================================
SECTION 1.3 PHP 8.3 Domain Enums (modern/backend/src/Domain/Enum/DomainEnums.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Domain\Enum\DomainEnums.php
================================================================================
```

```
<?php
declare(strict_types=1);

namespace DefMan\Domain\Enum;

/**
 * RBAC User Levels from menubar.php, form_usermanagement.php, form_apply.php
 */
enum UserLevel: int
{
    case APPLYING = 0; // Unapproved applicant (form_apply.php)
    case PUBLIC   = 1; // Public defect reporter (form_raportpub.php)
    case CUSTOMER = 2; // Authenticated customer (form_raport.php)
    case ANALYZER = 3; // Developer / QA triage & cost tracking (form_analyze.php)
    case ADMIN    = 4; // Full administrator (form_usermanagement.php, form_productdetails.php)

    public function label(): string
    {
        return match ($this) {
            self::APPLYING => 'Applying',
            self::PUBLIC   => 'Public',
            self::CUSTOMER => 'Customer',
            self::ANALYZER => 'Analyzer',
            self::ADMIN    => 'Admin',
```

```
        };
    }
}

/**
 * Defect Severity from form_raport.php & form_analyze.php
 */
enum DefectSeverity: string
{
    case MINOR    = 'minor';
    case MAJOR    = 'major';
    case CRITICAL = 'critical';
}

/**
 * Defect Priority from form_raport.php (m_urgent, average, l_urgent)
 * and form_analyze.php (most urgent, average, less urgent)
 */
enum DefectPriority: string
{
    case MOST_URGENT = 'm_urgent';
    case AVERAGE     = 'average';
    case LESS_URGENT = 'l_urgent';

    public function analysisLabel(): string
    {
        return match ($this) {
            self::MOST_URGENT => 'most urgent',
            self::AVERAGE     => 'average',
            self::LESS_URGENT => 'less urgent',
        };
    }

    public static function fromInput(string $input): self
    {
        return match (strtolower(trim($input))) {
            'm_urgent', 'most urgent' => self::MOST_URGENT,
            'l_urgent', 'less urgent' => self::LESS_URGENT,
            default                   => self::AVERAGE,
        };
    }
}

/**
 * Defect Status from form_analyze.php & PixiJS v8 Triage Lanes
 */
enum DefectStatus: string
{
    case SUBMITTED = 'submitted';
    case ASSIGNED  = 'assigned';
    case POSTPONED = 'postponed';
    case OPENED    = 'opened';
    case FIXED     = 'fixed';
    case RESOLVED  = 'resolved';
    case CLOSED    = 'closed';
}

/**
 * Person Type who discovered the defect (form_raport.php)
 */
enum PersonType: string
{
    case CUSTOMER  = 'Customer';
    case DEVELOPER = 'Developer';
    case TESTER    = 'Tester';
}
```

```
/**
 * Defect Uniqueness (form_analyze.php)
 */
enum Uniqueness: string
{
    case NEW_DEFECT        = 'New defect';
    case ALREADY_SUBMITTED = 'Already submitted';
    case CRITICAL          = 'Critical';
}

/**
 * Lifecycle Phase in which the defect originated (form_analyze.php)
 */
enum OriginPhase: string
{
    case REQUIREMENTS   = 'Requirements';
    case DESIGN         = 'Design';
    case IMPLEMENTATION = 'Implementation';
    case TESTING        = 'Testing';
    case DEPLOYMENT     = 'Deployment';
    case MAINTENANCE    = 'Maintenance';
}
```

==========

SECTION 1.4 PDO Database Seeder (modern/backend/src/Database/DatabaseSeeder.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Database\DatabaseSeeder.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Database;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\OriginPhase;
use DefMan\Domain\Enum\PersonType;
use DefMan\Domain\Enum\Uniqueness;
use DefMan\Domain\Enum\UserLevel;
use PDO;

/**
 * PDO Database Seeder
 * Refactored from: install/installdb.php, install/install-part2.php, helpanalyze.html, helprepo
    rt.html
 *
 * Test/Seed Account Rule:
 * - Admin / System placeholders: zuckerberg@fb.com, billg@microsoft.com
 * - Customer / Analyzer accounts: Sabrina (sabrina@hotmail.com), Julia (julia@hotmail.com)
 */
final readonly class DatabaseSeeder
{
    public function __construct(private PDO $pdo) {}

    public function run(): void
    {
        $this->pdo->beginTransaction();

        try {
            // 1. Seed Products (including historical MMF 1.5 from helpanalyze.html)
            $products = [
```

```
        'MMF 1.5',
        'DefMan 2004 Core',
        'Katiska Auth Gateway 2.0',
        'Telemetry Engine v4',
    ];

    $productStmt = $this->pdo->prepare(
        'INSERT INTO `product` (`name`) VALUES (:name)
         ON DUPLICATE KEY UPDATE `name` = VALUES(`name`)'
    );
    foreach ($products as $productName) {
        $productStmt->execute([':name' => $productName]);
    }

    // Map product names to IDs
    $productMap = [];
    $rows = $this->pdo->query('SELECT `id`, `name` FROM `product`')->fetchAll();
    foreach ($rows as $row) {
        $productMap[$row['name']] = (int) $row['id'];
    }

    // 2. Seed Users with Argon2id hashes
    $users = [
        [
            'login'     => 'zuck',
            'password'  => password_hash('Admin#2026!Zuck', PASSWORD_ARGON2ID),
            'email'     => 'zuckerberg@fb.com',
            'userlevel' => UserLevel::ADMIN->value,
            'company'   => 'Meta Platforms',
            'phone'     => '+1-650-543-4800',
            'name'      => 'Mark Zuckerberg',
            'activate'  => 1,
            'logged'    => 1,
        ],
        [
            'login'     => 'billg',
            'password'  => password_hash('System#2026!Bill', PASSWORD_ARGON2ID),
            'email'     => 'billg@microsoft.com',
            'userlevel' => UserLevel::ADMIN->value,
            'company'   => 'Microsoft Corp',
            'phone'     => '+1-425-882-8080',
            'name'      => 'Bill Gates',
            'activate'  => 1,
            'logged'    => 0,
        ],
        [
            'login'     => 'julia',
            'password'  => password_hash('Analyze#2026!Julia', PASSWORD_ARGON2ID),
            'email'     => 'julia@hotmail.com',
            'userlevel' => UserLevel::ANALYZER->value,
            'company'   => 'DefMan QA Labs',
            'phone'     => '+358-40-555-0103',
            'name'      => 'Julia',
            'activate'  => 1,
            'logged'    => 1,
        ],
        [
            'login'     => 'sabrin',
            'password'  => password_hash('Cust#2026!Sabrina', PASSWORD_ARGON2ID),
            'email'     => 'sabrina@hotmail.com',
            'userlevel' => UserLevel::CUSTOMER->value,
            'company'   => 'Nordic Enterprise Oy',
            'phone'     => '+358-40-555-0102',
            'name'      => 'Sabrina',
            'activate'  => 1,
            'logged'    => 0,
```

```
            ],
        ];

        $userStmt = $this->pdo->prepare(
            'INSERT INTO `users` (`login`, `password`, `email`, `userlevel`, `company`, `pho
ne`, `name`, `activate`, `logged`)
             VALUES (:login, :password, :email, :userlevel, :company, :phone, :name, :activa
te, :logged)
             ON DUPLICATE KEY UPDATE
               `userlevel` = VALUES(`userlevel`),
               `company`   = VALUES(`company`),
               `name`      = VALUES(`name`),
               `activate`  = VALUES(`activate`)'
        );

        foreach ($users as $u) {
            $userStmt->execute([
                ':login'     => $u['login'],
                ':password'  => $u['password'],
                ':email'     => $u['email'],
                ':userlevel' => $u['userlevel'],
                ':company'   => $u['company'],
                ':phone'     => $u['phone'],
                ':name'      => $u['name'],
                ':activate'  => $u['activate'],
                ':logged'    => $u['logged'],
            ]);
        }

        $sabrinaId = (int) $this->pdo->query(
            "SELECT `id` FROM `users` WHERE `email` = 'sabrina@hotmail.com' LIMIT 1"
        )->fetchColumn();

        // 3. Seed Sample Defects & Analyses (preserving helpanalyze.html & helpreport.html
canon)
        $sampleDefects = [
            [
                'Product'      => 'MMF 1.5',
                'product_id'   => $productMap['MMF 1.5'] ?? null,
                'Environment'  => 'AMD Athlon XP 2000, WinXP',
                'CustomerID'   => $sabrinaId,
                'Who'          => 'sabrin',
                'Contactemail' => 'sabrina@hotmail.com',
                'Whenreported' => '2004-04-14 14:20',
                'Title'        => 'PopUp Menu Crash',
                'Description'  => 'When adding entries into the popup menu in a certain way,
 the application will close without any error messages.',
                'Discoveredby' => 'Sabrina',
                'Persontype'   => PersonType::CUSTOMER->value,
                'Occured'      => '2004-04-14',
                'Location'     => 'settings form',
                'Severity'     => DefectSeverity::CRITICAL->value,
                'Priority'     => DefectPriority::MOST_URGENT->value,
                'Submitted'    => '2004-04-14',
                'Addinfo'      => 'please fix this soon',
                'analysis'     => [
                    'devdesc'        => 'It seems to be happening when you drag a parent ite
m and drop it to its children.',
                    'classification' => IeeeFaultType::LOGIC->value,
                    'dateoriginated' => '2004-04-16',
                    'phase'          => implode(',', [OriginPhase::IMPLEMENTATION->value, Or
iginPhase::TESTING->value]),
                    'actoriginated'  => 'Testing',
                    'deliverable'    => 'the pop-up menu settings form',
                    'whattodo'       => 'Prevent cyclic parent-to-child drag-and-drop in pop
up tree.',
```

```
                    'submitted'      => '2004-04-14',
                    'assigned'       => '2004-04-17',
                    'uniqueness'     => Uniqueness::CRITICAL->value,
                    'priority'       => DefectPriority::MOST_URGENT->analysisLabel(),
                    'severity'       => DefectSeverity::CRITICAL->value,
                    'workaround'     => 'Do not drag and drop.',
                    'proposedfix'    => "Let's fix the pop-up menu module.",
                    'status'         => DefectStatus::RESOLVED->value,
                    'fixdesc'        => 'Fixed the pop-up menu function call.',
                    'fixstarted'     => '2004-04-19',
                    'fixcomplete'    => '2004-04-20',
                    'fiximpact'      => 'MMF 1.5 settings.cpp popup.cpp',
                    'fixeffort'      => 2.00,
                    'fixcost'        => 60.00,
                    'resodesc'       => 'The pop-up menu is fixed.',
                    'resocomplete'   => '2004-04-22',
                    'resoclosed'     => '2004-04-23',
                    'costofdefect'   => 240.00,
                    'actfound'       => 'sequential testing',
                ],
            ],
            [
                'Product'      => 'DefMan 2004 Core',
                'product_id'   => $productMap['DefMan 2004 Core'] ?? null,
                'Environment'  => 'Linux 2.4, Apache 1.3, PHP 4.3, MySQL 4.0',
                'CustomerID'   => $sabrinaId,
                'Who'          => 'sabrin',
                'Contactemail' => 'sabrina@hotmail.com',
                'Whenreported' => '2004-04-18 09:15',
                'Title'        => 'Precision Loss in EUR Effort Multiplier',
                'Description'  => 'Fractional effort hours get truncated when stored in smal
lint(6) fixeffort and fixcost columns.',
                'Discoveredby' => 'Julia',
                'Persontype'   => PersonType::DEVELOPER->value,
                'Occured'      => '2004-04-18',
                'Location'     => 'form_deffix.php',
                'Severity'     => DefectSeverity::MAJOR->value,
                'Priority'     => DefectPriority::MOST_URGENT->value,
                'Submitted'    => '2004-04-18',
                'Addinfo'      => 'Affects IEEE cost reporting accuracy.',
                'analysis'     => [
                    'devdesc'        => 'Integer truncation in legacy MyISAM schema causes r
ounding fault.',
                    'classification' => IeeeFaultType::COMPUTATIONAL->value,
                    'dateoriginated' => '2004-04-10',
                    'phase'          => OriginPhase::DESIGN->value,
                    'actoriginated'  => 'Database schema definition',
                    'deliverable'    => 'install/installdb.php',
                    'whattodo'       => 'Migrate fixeffort and fixcost to DECIMAL(8,2) / DEC
IMAL(10,2).',
                    'submitted'      => '2004-04-18',
                    'assigned'       => '2004-04-19',
                    'uniqueness'     => Uniqueness::NEW_DEFECT->value,
                    'priority'       => DefectPriority::MOST_URGENT->analysisLabel(),
                    'severity'       => DefectSeverity::MAJOR->value,
                    'workaround'     => 'Round effort to nearest whole hour.',
                    'proposedfix'    => 'Alter table columns to DECIMAL in InnoDB.',
                    'status'         => DefectStatus::OPENED->value,
                    'fixdesc'        => 'Migrated schema to MySQL 8 InnoDB DECIMAL columns.'
,
                    'fixstarted'     => '2004-04-20',
                    'fixcomplete'    => null,
                    'fiximpact'      => 'schema.sql, form_deffix.php',
                    'fixeffort'      => 6.50,
                    'fixcost'        => 195.00,
                    'resodesc'       => null,
```

```
                    'resocomplete'   => null,
                    'resoclosed'     => null,
                    'costofdefect'   => 420.00,
                    'actfound'       => 'schema review',
                ],
            ],
            [
                'Product'      => 'Katiska Auth Gateway 2.0',
                'product_id'   => $productMap['Katiska Auth Gateway 2.0'] ?? null,
                'Environment'  => 'Netscape 7.1 / Solaris X-Session',
                'CustomerID'   => $sabrinaId,
                'Who'          => 'sabrin',
                'Contactemail' => 'sabrina@hotmail.com',
                'Whenreported' => '2004-04-21 11:40',
                'Title'        => 'Subroutine Mismatch in Session Logout Redirect',
                'Description'  => 'Session state flag logged=1 remains active if user closes
 browser tab without hitting logout.php.',
                'Discoveredby' => 'Sabrina',
                'Persontype'   => PersonType::TESTER->value,
                'Occured'      => '2004-04-21',
                'Location'     => 'login.php / logout.php',
                'Severity'     => DefectSeverity::MINOR->value,
                'Priority'     => DefectPriority::AVERAGE->value,
                'Submitted'    => '2004-04-21',
                'Addinfo'      => 'User management shows stale Online badge.',
                'analysis'     => [
                    'devdesc'        => 'Missing session expiration heartbeat in authenticat
ion interface.',
                    'classification' => IeeeFaultType::INTERFACE->value,
                    'dateoriginated' => '2004-04-12',
                    'phase'          => implode(',', [OriginPhase::DESIGN->value, OriginPhas
e::IMPLEMENTATION->value]),
                    'actoriginated'  => 'Session handling implementation',
                    'deliverable'    => 'login.php, logout.php',
                    'whattodo'       => 'Introduce expiring bearer sessions in user_sessions
 table.',
                    'submitted'      => '2004-04-21',
                    'assigned'       => '2004-04-22',
                    'uniqueness'     => Uniqueness::NEW_DEFECT->value,
                    'priority'       => DefectPriority::AVERAGE->analysisLabel(),
                    'severity'       => DefectSeverity::MINOR->value,
                    'workaround'     => 'Manually log out via menubar.',
                    'proposedfix'    => 'Use token expiration and session TTL.',
                    'status'         => DefectStatus::FIXED->value,
                    'fixdesc'        => 'Implemented token TTL and automatic online presence
 check.',
                    'fixstarted'     => '2004-04-23',
                    'fixcomplete'    => '2004-04-24',
                    'fiximpact'      => 'AuthController.php, user_sessions',
                    'fixeffort'      => 4.00,
                    'fixcost'        => 120.00,
                    'resodesc'       => 'Awaiting final QA sign-off.',
                    'resocomplete'   => null,
                    'resoclosed'     => null,
                    'costofdefect'   => 180.00,
                    'actfound'       => 'integration testing',
                ],
            ],
        ];

        $raportStmt = $this->pdo->prepare(
            'INSERT INTO `raport`
             (`Product`, `product_id`, `Environment`, `CustomerID`, `Who`, `Contactemail`, `
Whenreported`,
              `Title`, `Description`, `Discoveredby`, `Persontype`, `Occured`, `Location`,
              `Severity`, `Priority`, `Submitted`, `Addinfo`)
```

```
                VALUES
                (:Product, :product_id, :Environment, :CustomerID, :Who, :Contactemail, :Whenre
    ported,
                 :Title, :Description, :Discoveredby, :Persontype, :Occured, :Location,
                 :Severity, :Priority, :Submitted, :Addinfo)'
            );

            $analyseStmt = $this->pdo->prepare(
                'INSERT INTO 'analyse'
                ('id', 'devdesc', 'classification', 'dateoriginated', 'phase', 'actoriginated',
                 'deliverable', 'whattodo', 'submitted', 'assigned', 'uniqueness', 'priority',
                 'severity', 'workaround', 'proposedfix', 'status', 'fixdesc', 'fixstarted',
                 'fixcomplete', 'fiximpact', 'fixeffort', 'fixcost', 'resodesc', 'resocomplete'
    ,
                 'resoclosed', 'costofdefect', 'actfound')
                VALUES
                (:id, :devdesc, :classification, :dateoriginated, :phase, :actoriginated,
                 :deliverable, :whattodo, :submitted, :assigned, :uniqueness, :priority,
                 :severity, :workaround, :proposedfix, :status, :fixdesc, :fixstarted,
                 :fixcomplete, :fiximpact, :fixeffort, :fixcost, :resodesc, :resocomplete,
                 :resoclosed, :costofdefect, :actfound)'
            );

            foreach ($sampleDefects as $defect) {
                $analysis = $defect['analysis'];
                unset($defect['analysis']);

                $raportStmt->execute([
                    ':Product'      => $defect['Product'],
                    ':product_id'   => $defect['product_id'],
                    ':Environment'  => $defect['Environment'],
                    ':CustomerID'   => $defect['CustomerID'],
                    ':Who'          => $defect['Who'],
                    ':Contactemail' => $defect['Contactemail'],
                    ':Whenreported' => $defect['Whenreported'],
                    ':Title'        => $defect['Title'],
                    ':Description'  => $defect['Description'],
                    ':Discoveredby' => $defect['Discoveredby'],
                    ':Persontype'   => $defect['Persontype'],
                    ':Occured'      => $defect['Occured'],
                    ':Location'     => $defect['Location'],
                    ':Severity'     => $defect['Severity'],
                    ':Priority'     => $defect['Priority'],
                    ':Submitted'    => $defect['Submitted'],
                    ':Addinfo'      => $defect['Addinfo'],
                ]);

                $defectId = (int) $this->pdo->lastInsertId();

                $analyseStmt->execute([
                    ':id'             => $defectId,
                    ':devdesc'        => $analysis['devdesc'],
                    ':classification' => $analysis['classification'],
                    ':dateoriginated' => $analysis['dateoriginated'],
                    ':phase'          => $analysis['phase'],
                    ':actoriginated'  => $analysis['actoriginated'],
                    ':deliverable'    => $analysis['deliverable'],
                    ':whattodo'       => $analysis['whattodo'],
                    ':submitted'      => $analysis['submitted'],
                    ':assigned'       => $analysis['assigned'],
                    ':uniqueness'     => $analysis['uniqueness'],
                    ':priority'       => $analysis['priority'],
                    ':severity'       => $analysis['severity'],
                    ':workaround'     => $analysis['workaround'],
                    ':proposedfix'    => $analysis['proposedfix'],
                    ':status'         => $analysis['status'],
```

```
                    ':fixdesc'        => $analysis['fixdesc'],
                    ':fixstarted'     => $analysis['fixstarted'],
                    ':fixcomplete'    => $analysis['fixcomplete'],
                    ':fiximpact'      => $analysis['fiximpact'],
                    ':fixeffort'      => $analysis['fixeffort'],
                    ':fixcost'        => $analysis['fixcost'],
                    ':resodesc'       => $analysis['resodesc'],
                    ':resocomplete'   => $analysis['resocomplete'],
                    ':resoclosed'     => $analysis['resoclosed'],
                    ':costofdefect'   => $analysis['costofdefect'],
                    ':actfound'       => $analysis['actfound'],
                ]);
            }

            $this->pdo->commit();
        } catch (\Throwable $e) {
            if ($this->pdo->inTransaction()) {
                $this->pdo->rollBack();
            }
            throw $e;
        }
    }
}
```

==========

SECTION 2.1 PDO Connection Singleton (modern/backend/src/Infrastructure/Database/Database.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Infrastructure\Database\Database.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Infrastructure\Database;

use PDO;
use PDOException;
use RuntimeException;

/**
 * Strict-typed PHP 8.3 PDO Connection Singleton
 * Refactored from: config.php & dbconnect.php
 */
final class Database
{
    private static ?PDO $instance = null;

    private function __construct() {}

    private function __clone() {}

    public function __wakeup(): never
    {
        throw new RuntimeException('Cannot unserialize singleton Database instance.');
    }

    public static function getConnection(): PDO
    {
        if (self::$instance === null) {
            $host    = $_ENV['DB_HOST'] ?? getenv('DB_HOST') ?: 'localhost';
            $port    = (int) ($_ENV['DB_PORT'] ?? getenv('DB_PORT') ?: 3306);
            $dbName  = $_ENV['DB_NAME'] ?? getenv('DB_NAME') ?: 'opr0413';
            $user    = $_ENV['DB_USER'] ?? getenv('DB_USER') ?: 'opr0413';
            $pass    = $_ENV['DB_PASS'] ?? getenv('DB_PASS') ?: 'd3fman13';
```

```
            $charset = 'utf8mb4';

            $dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=%s', $host, $port, $dbName,
    $charset);

            try {
                self::$instance = new PDO($dsn, $user, $pass, [
                    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES   => false,
                    PDO::ATTR_STRINGIFY_FETCHES  => false,
                ]);
            } catch (PDOException $e) {
                throw new RuntimeException('Database connection failed: ' . $e->getMessage(), (i
    nt) $e->getCode(), $e);
            }
        }

        return self::$instance;
    }
}
```

```
================================================================================
SECTION 2.2 UserRepository (modern/backend/src/Repository/UserRepository.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\UserRepository.php
================================================================================
```

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\UserLevel;
use PDO;

/**
 * User & Session Repository
 * Refactored from: db.php, login.php, logout.php, form_apply.php, form_newuser.php,
 *                  form_lostpassword.php, form_userdetails.php, form_usermanagement.php
 */
final readonly class UserRepository
{
    public function __construct(private PDO $pdo) {}

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT `id`, `login`, `password`, `email`, `userlevel`, `company`, `phone`, `name`,
     `activate`, `logged`
             FROM `users` WHERE `id` = :id LIMIT 1'
        );
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function findActiveByLogin(string $login): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT `id`, `login`, `password`, `email`, `userlevel`, `company`, `phone`, `name`,
     `activate`, `logged`
             FROM `users` WHERE `login` = :login AND `activate` = 1 LIMIT 1'
        );
        $stmt->execute([':login' => $login]);
```

```
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function findActiveByEmail(string $email): ?array
    {
        $stmt = $this->pdo->prepare(
            'SELECT `id`, `login`, `password`, `email`, `userlevel`, `company`, `phone`, `name`,
     `activate`, `logged`
             FROM `users` WHERE `email` = :email AND `activate` = 1 AND `userlevel` > 1 LIMIT 1'
        );
        $stmt->execute([':email' => $email]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function existsByEmail(string $email, ?int $excludeUserId = null): bool
    {
        $sql = 'SELECT COUNT(*) FROM `users` WHERE `email` = :email';
        $params = [':email' => $email];
        if ($excludeUserId !== null) {
            $sql .= ' AND `id` != :exclude_id';
            $params[':exclude_id'] = $excludeUserId;
        }
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function existsByLogin(string $login): bool
    {
        $stmt = $this->pdo->prepare('SELECT COUNT(*) FROM `users` WHERE `login` = :login');
        $stmt->execute([':login' => $login]);
        return (int) $stmt->fetchColumn() > 0;
    }

    /**
     * Lists approved users (userlevel > 0) and pending applicants (userlevel = 0)
     * Refactored from: form_usermanagement.php
     */
    public function listAllByStatus(): array
    {
        $activeStmt = $this->pdo->query(
            'SELECT `id`, `login`, `email`, `userlevel`, `company`, `phone`, `name`, `activate`,
     `logged`
             FROM `users` WHERE `userlevel` > 0 ORDER BY `userlevel` ASC, `login` ASC'
        );
        $applyingStmt = $this->pdo->query(
            'SELECT `id`, `login`, `email`, `userlevel`, `company`, `phone`, `name`, `activate`,
     `logged`
             FROM `users` WHERE `userlevel` = 0 ORDER BY `id` ASC'
        );

        return [
            'users'      => $activeStmt->fetchAll(),
            'applicants' => $applyingStmt->fetchAll(),
        ];
    }

    /**
     * Creates a new user or applicant
     * Refactored from: form_apply.php (userlevel=0, activate=0) & form_newuser.php (activate=1)
     */
    public function create(
        string $login,
        string $plainPassword,
```

```
        string $email,
        UserLevel $userLevel,
        ?string $company = null,
        ?string $phone = null,
        ?string $name = null,
        bool $activate = false
    ): int {
        $stmt = $this->pdo->prepare(
            'INSERT INTO `users` (`login`, `password`, `email`, `userlevel`, `company`, `phone`,
     `name`, `activate`, `logged`)
             VALUES (:login, :password, :email, :userlevel, :company, :phone, :name, :activate,
    0)'
        );
        $stmt->execute([
            ':login'     => $login,
            ':password'  => password_hash($plainPassword, PASSWORD_ARGON2ID),
            ':email'     => $email,
            ':userlevel' => $userLevel->value,
            ':company'   => $company,
            ':phone'     => $phone,
            ':name'      => $name,
            ':activate'  => $activate ? 1 : 0,
        ]);

        return (int) $this->pdo->lastInsertId();
    }

    /**
     * Admin approval of pending user application (db.php?function=accept)
     */
    public function approveApplicant(int $id, UserLevel $assignedLevel = UserLevel::CUSTOMER): b
    ool
    {
        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `userlevel` = :level, `activate` = 1 WHERE `id` = :id'
        );
        $stmt->execute([
            ':level' => $assignedLevel->value,
            ':id'    => $id,
        ]);
        return $stmt->rowCount() > 0;
    }

    public function updateRole(int $id, UserLevel $level): bool
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `userlevel` = :level WHERE `id` = :id');
        $stmt->execute([':level' => $level->value, ':id' => $id]);
        return $stmt->rowCount() > 0;
    }

    /**
     * Deletes a non-admin user or declines an applicant (db.php?function=delete)
     */
    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `users` WHERE `id` = :id AND `userlevel` != :ad
    min');
        $stmt->execute([
            ':id'    => $id,
            ':admin' => UserLevel::ADMIN->value,
        ]);
        return $stmt->rowCount() > 0;
    }

    public function updatePassword(int $userId, string $newPlainPassword): void
    {
```

```
        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `password` = :password, `reset_token_hash` = NULL, `reset_token_
    expires_at` = NULL
             WHERE `id` = :id'
        );
        $stmt->execute([
            ':password' => password_hash($newPlainPassword, PASSWORD_ARGON2ID),
            ':id'       => $userId,
        ]);
    }

    public function updateEmail(int $userId, string $email): void
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `email` = :email WHERE `id` = :id');
        $stmt->execute([':email' => $email, ':id' => $userId]);
    }

    public function updatePersonalDetails(int $userId, string $name, string $company, string $ph
    one): void
    {
        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `name` = :name, `company` = :company, `phone` = :phone WHERE `id
    ` = :id'
        );
        $stmt->execute([
            ':name'    => $name,
            ':company' => $company,
            ':phone'   => $phone,
            ':id'      => $userId,
        ]);
    }

    public function setLoggedStatus(int $userId, bool $logged): void
    {
        $stmt = $this->pdo->prepare('UPDATE `users` SET `logged` = :logged WHERE `id` = :id');
        $stmt->execute([':logged' => $logged ? 1 : 0, ':id' => $userId]);
    }

    public function createSessionToken(int $userId, ?string $ipAddress, ?string $userAgent): str
    ing
    {
        $rawToken  = bin2hex(random_bytes(32));
        $tokenHash = hash('sha256', $rawToken);
        $expiresAt = (new \DateTimeImmutable('+12 hours'))->format('Y-m-d H:i:s');

        $stmt = $this->pdo->prepare(
            'INSERT INTO `user_sessions` (`user_id`, `token_hash`, `ip_address`, `user_agent`, `
    expires_at`)
             VALUES (:user_id, :token_hash, :ip_address, :user_agent, :expires_at)'
        );
        $stmt->execute([
            ':user_id'    => $userId,
            ':token_hash' => $tokenHash,
            ':ip_address' => $ipAddress,
            ':user_agent' => $userAgent ? substr($userAgent, 0, 255) : null,
            ':expires_at' => $expiresAt,
        ]);

        $this->setLoggedStatus($userId, true);
        return $rawToken;
    }

    public function findUserBySessionToken(string $rawToken): ?array
    {
        $tokenHash = hash('sha256', $rawToken);
        $stmt = $this->pdo->prepare(
```

```
            'SELECT u.`id`, u.`login`, u.`email`, u.`userlevel`, u.`company`, u.`phone`, u.`name
    `, u.`activate`, u.`logged`
             FROM `user_sessions` s
             INNER JOIN `users` u ON u.`id` = s.`user_id`
             WHERE s.`token_hash` = :token_hash
               AND s.`expires_at` > NOW()
               AND u.`activate` = 1
             LIMIT 1'
        );
        $stmt->execute([':token_hash' => $tokenHash]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function revokeSessionToken(string $rawToken, int $userId): void
    {
        $tokenHash = hash('sha256', $rawToken);
        $stmt = $this->pdo->prepare('DELETE FROM `user_sessions` WHERE `token_hash` = :token_has
    h');
        $stmt->execute([':token_hash' => $tokenHash]);
        $this->setLoggedStatus($userId, false);
    }

    public function createPasswordResetToken(int $userId): string
    {
        $rawToken  = bin2hex(random_bytes(32));
        $tokenHash = hash('sha256', $rawToken);
        $expiresAt = (new \DateTimeImmutable('+1 hour'))->format('Y-m-d H:i:s');

        $stmt = $this->pdo->prepare(
            'UPDATE `users` SET `reset_token_hash` = :hash, `reset_token_expires_at` = :expires
    WHERE `id` = :id'
        );
        $stmt->execute([
            ':hash'    => $tokenHash,
            ':expires' => $expiresAt,
            ':id'      => $userId,
        ]);

        return $rawToken;
    }
}
```

==========

SECTION 2.3 ProductRepository (modern/backend/src/Repository/ProductRepository.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\ProductRepository.p
    hp

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use PDO;

/**
 * Product Repository
 * Refactored from: form_productdetails.php
 */
final readonly class ProductRepository
{
    public function __construct(private PDO $pdo) {}
```

```
    public function findAll(): array
    {
        $stmt = $this->pdo->query(
            'SELECT p.`id`, p.`name`, COUNT(r.`ID`) AS `report_count`
             FROM `product` p
             LEFT JOIN `raport` r ON r.`Product` = p.`name` OR r.`product_id` = p.`id`
             GROUP BY p.`id`, p.`name`
             ORDER BY p.`name` ASC'
        );
        return $stmt->fetchAll();
    }

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare('SELECT `id`, `name` FROM `product` WHERE `id` = :id LIMIT 1
    ');
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function findByName(string $name): ?array
    {
        $stmt = $this->pdo->prepare('SELECT `id`, `name` FROM `product` WHERE `name` = :name LIM
    IT 1');
        $stmt->execute([':name' => $name]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function create(string $name): int
    {
        $stmt = $this->pdo->prepare('INSERT INTO `product` (`name`) VALUES (:name)');
        $stmt->execute([':name' => trim(strip_tags($name))]);
        return (int) $this->pdo->lastInsertId();
    }

    public function delete(int $id): ?string
    {
        $product = $this->findById($id);
        if ($product === null) {
            return null;
        }
        $stmt = $this->pdo->prepare('DELETE FROM `product` WHERE `id` = :id');
        $stmt->execute([':id' => $id]);
        return (string) $product['name'];
    }
}
```

==========

SECTION 2.4 DefectRepository (modern/backend/src/Repository/DefectRepository.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\DefectRepository.ph
p

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\PersonType;
use PDO;
```

```
/**
 * Defect Report Repository (‘raport‘ + joined ‘analyse‘ telemetry)
 * Refactored from: form_raport.php, form_raportpub.php, form_raportlist.php,
 *                  form_list.php, form_defrap.php, db.php (deleteraport)
 */
final readonly class DefectRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Multi-column sorting & filtering by severity, priority, status, product, and customer.
     * Also computes ‘days_since_submitted‘ and deleted-user badge per form_raportlist.php.
     */
    public function findFiltered(
        ?string $severity = null,
        ?string $priority = null,
        ?string $status = null,
        ?string $product = null,
        ?int $customerId = null,
        string $sortBy = ’date’
    ): array {
        $conditions = [];
        $params     = [];

        if ($severity !== null && $severity !== ’’) {
            $conditions[]        = ’r.‘Severity‘ = :severity’;
            $params[’:severity’] = $severity;
        }

        if ($priority !== null && $priority !== ’’) {
            $normalizedPriority  = DefectPriority::fromInput($priority)->value;
            $conditions[]        = ’r.‘Priority‘ = :priority’;
            $params[’:priority’] = $normalizedPriority;
        }

        if ($status !== null && $status !== ’’) {
            if ($status === ’unassigned’) {
                $conditions[] = ’a.‘status‘ IS NULL’;
            } else {
                $conditions[]      = ’a.‘status‘ = :status’;
                $params[’:status’] = $status;
            }
        }

        if ($product !== null && $product !== ’’) {
            $conditions[]       = ’r.‘Product‘ = :product’;
            $params[’:product’] = $product;
        }

        if ($customerId !== null) {
            $conditions[]           = ’r.‘CustomerID‘ = :customer_id’;
            $params[’:customer_id’] = $customerId;
        }

        $whereSql = count($conditions) > 0 ? ’WHERE ’ . implode(’ AND ’, $conditions) : ’’;

        $orderBySql = match ($sortBy) {
            ’product’  => ’r.‘Product‘ ASC, r.‘ID‘ DESC’,
            ’contact’  => ’r.‘Who‘ DESC, r.‘ID‘ DESC’,
            ’severity’ => "FIELD(r.‘Severity‘, ’critical’, ’major’, ’minor’) ASC, r.‘Submitted‘
    DESC",
            ’priority’, ’all’ => "FIELD(r.‘Priority‘, ’m_urgent’, ’average’, ’l_urgent’) ASC, r.
    ‘Submitted‘ DESC",
            default    => ’r.‘Submitted‘ DESC, r.‘ID‘ DESC’,
        };
```

```
        $sql = "
            SELECT
                r.‘ID‘,
                r.‘Product‘,
                r.‘product_id‘,
                r.‘Environment‘,
                r.‘CustomerID‘,
                r.‘Who‘,
                r.‘Contactemail‘,
                r.‘Whenreported‘,
                r.‘Title‘,
                r.‘Description‘,
                r.‘Discoveredby‘,
                r.‘Persontype‘,
                r.‘Occured‘,
                r.‘Location‘,
                r.‘Severity‘,
                r.‘Priority‘,
                CASE r.‘Priority‘
                    WHEN ’m_urgent’ THEN ’most urgent’
                    WHEN ’l_urgent’ THEN ’less urgent’
                    ELSE ’average’
                END AS ‘PriorityLabel‘,
                r.‘Submitted‘,
                DATEDIFF(CURDATE(), r.‘Submitted‘) AS ‘days_since_submitted‘,
                r.‘Addinfo‘,
                CASE WHEN r.‘CustomerID‘ IS NOT NULL AND u.‘id‘ IS NULL THEN 1 ELSE 0 END AS ‘is
    _deleted_user‘,
                a.‘classification‘,
                COALESCE(a.‘status‘, ’submitted’) AS ‘analysis_status‘,
                a.‘severity‘ AS ‘developer_severity‘,
                a.‘priority‘ AS ‘developer_priority‘,
                a.‘fixeffort‘,
                a.‘fixcost‘,
                a.‘costofdefect‘,
                a.‘assigned‘,
                a.‘fixcomplete‘,
                a.‘resocomplete‘,
                a.‘resoclosed‘
            FROM ‘raport‘ r
            LEFT JOIN ‘users‘ u ON u.‘id‘ = r.‘CustomerID‘
            LEFT JOIN ‘analyse‘ a ON a.‘id‘ = r.‘ID‘
            {$whereSql}
            ORDER BY {$orderBySql}
        ";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }

    public function findById(int $id): ?array
    {
        $stmt = $this->pdo->prepare(
            "SELECT
                r.*,
                CASE r.‘Priority‘
                    WHEN ’m_urgent’ THEN ’most urgent’
                    WHEN ’l_urgent’ THEN ’less urgent’
                    ELSE ’average’
                END AS ‘PriorityLabel‘,
                DATEDIFF(CURDATE(), r.‘Submitted‘) AS ‘days_since_submitted‘
             FROM ‘raport‘ r
             WHERE r.‘ID‘ = :id
             LIMIT 1"
```

```
        );
        $stmt->execute([':id' => $id]);
        $report = $stmt->fetch();
        if (!$report) {
            return null;
        }

        $analysisStmt = $this->pdo->prepare('SELECT * FROM `analyse` WHERE `id` = :id LIMIT 1');
        $analysisStmt->execute([':id' => $id]);
        $report['analysis'] = $analysisStmt->fetch() ?: null;

        return $report;
    }

    /**
     * Creates a new defect report (form_raport.php & form_raportpub.php)
     */
    public function create(array $data): int
    {
        $severity   = DefectSeverity::from($data['severity'] ?? 'minor');
        $priority   = DefectPriority::fromInput($data['priority'] ?? 'average');
        $personType = PersonType::from($data['persontype'] ?? 'Customer');

        $productStmt = $this->pdo->prepare('SELECT `id` FROM `product` WHERE `name` = :name LIMI
    T 1');
        $productStmt->execute([':name' => $data['product']]);
        $productId = $productStmt->fetchColumn() ?: null;

        $stmt = $this->pdo->prepare(
            'INSERT INTO `raport`
             (`Product`, `product_id`, `Environment`, `CustomerID`, `Who`, `Contactemail`, `When
    reported`,
              `Title`, `Description`, `Discoveredby`, `Persontype`, `Occured`, `Location`,
              `Severity`, `Priority`, `Submitted`, `Addinfo`)
             VALUES
             (:Product, :product_id, :Environment, :CustomerID, :Who, :Contactemail, :Whenreport
    ed,
              :Title, :Description, :Discoveredby, :Persontype, :Occured, :Location,
              :Severity, :Priority, :Submitted, :Addinfo)'
        );

        $nowDate = date('Y-m-d');
        $nowTime = date('Y-m-d G:i');

        $stmt->execute([
            ':Product'      => trim(strip_tags((string) $data['product'])),
            ':product_id'   => $productId ? (int) $productId : null,
            ':Environment'  => trim(strip_tags((string) ($data['environment'] ?? ''))),
            ':CustomerID'   => isset($data['customer_id']) ? (int) $data['customer_id'] : null,
            ':Who'          => substr(trim(strip_tags((string) $data['who'])), 0, 64),
            ':Contactemail' => trim(strip_tags((string) $data['contact_email'])),
            ':Whenreported' => $nowTime,
            ':Title'        => trim(strip_tags((string) $data['title'])),
            ':Description'  => trim(strip_tags((string) $data['description'])),
            ':Discoveredby' => trim(strip_tags((string) ($data['discovered_by'] ?? ''))),
            ':Persontype'   => $personType->value,
            ':Occured'      => $data['occured'],
            ':Location'     => trim(strip_tags((string) ($data['location'] ?? ''))),
            ':Severity'     => $severity->value,
            ':Priority'     => $priority->value,
            ':Submitted'    => $nowDate,
            ':Addinfo'      => trim(strip_tags((string) ($data['addinfo'] ?? ''))),
        ]);

        return (int) $this->pdo->lastInsertId();
    }
```

```
    /**
     * Updates an existing defect report (form_defrap.php)
     */
    public function update(int $id, array $data): bool
    {
        $severity   = DefectSeverity::from($data['severity']);
        $priority   = DefectPriority::fromInput($data['priority']);
        $personType = PersonType::from($data['persontype']);

        $stmt = $this->pdo->prepare(
            'UPDATE `raport` SET
                `Product`      = :Product,
                `Environment`  = :Environment,
                `Title`        = :Title,
                `Description`  = :Description,
                `Discoveredby` = :Discoveredby,
                `Persontype`   = :Persontype,
                `Occured`      = :Occured,
                `Location`     = :Location,
                `Severity`     = :Severity,
                `Priority`     = :Priority,
                `Submitted`    = :Submitted,
                `Addinfo`      = :Addinfo
             WHERE `ID` = :id'
        );

        $stmt->execute([
            ':Product'      => trim(strip_tags((string) $data['product'])),
            ':Environment'  => trim(strip_tags((string) ($data['environment'] ?? ''))),
            ':Title'        => trim(strip_tags((string) $data['title'])),
            ':Description'  => trim(strip_tags((string) $data['description'])),
            ':Discoveredby' => trim(strip_tags((string) ($data['discovered_by'] ?? ''))),
            ':Persontype'   => $personType->value,
            ':Occured'      => $data['occured'],
            ':Location'     => trim(strip_tags((string) ($data['location'] ?? ''))),
            ':Severity'     => $severity->value,
            ':Priority'     => $priority->value,
            ':Submitted'    => $data['submitted'],
            ':Addinfo'      => trim(strip_tags((string) ($data['addinfo'] ?? ''))),
            ':id'           => $id,
        ]);

        return $stmt->rowCount() >= 0;
    }

    /**
     * Deletes a defect report and cascades to `analyse` (db.php?function=deleteraport)
     */
    public function delete(int $id): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `raport` WHERE `ID` = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->rowCount() > 0;
    }
}
```

==========

SECTION 2.5 AnalysisRepository (modern/backend/src/Repository/AnalysisRepository.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Repository\AnalysisRepository.php

==========

```
<?php
```

```
declare(strict_types=1);

namespace DefMan\Repository;

use DefMan\Domain\Enum\DefectPriority;
use DefMan\Domain\Enum\DefectSeverity;
use DefMan\Domain\Enum\DefectStatus;
use DefMan\Domain\Enum\IeeeFaultType;
use DefMan\Domain\Enum\Uniqueness;
use PDO;

/**
 * Defect Analysis, Fix & Resolution Repository (‘analyse‘ table)
 * Refactored from: form_alist.php, form_analyze.php, form_analyzed.php,
 *                  form_deffix.php, form_defreso.php, db.php (deleteanalyze)
 */
final readonly class AnalysisRepository
{
    public function __construct(private PDO $pdo) {}

    /**
     * Returns the full Analyzer Queue with lead-time metrics:
     * - submitted_to_assigned_days
     * - submitted_to_fixed_days
     * - submitted_to_resolved_days
     * - assigned_to_fixed_days
     * - assigned_to_resolved_days
     * Refactored from: form_alist.php & form_analyzed.php
     */
    public function getAnalyzerQueue(?string $sort = ’all’): array
    {
        $whereSql = ’’;
        $params   = [];

        if (in_array($sort, [’critical’, ’major’, ’minor’], true)) {
            $whereSql        = ’WHERE COALESCE(a.‘severity‘, r.‘Severity‘) = :sort’;
            $params[’:sort’] = $sort;
        } elseif (in_array($sort, [’most urgent’, ’average’, ’less urgent’], true)) {
            $whereSql        = ’WHERE a.‘priority‘ = :sort’;
            $params[’:sort’] = $sort;
        } elseif (in_array($sort, [’submitted’, ’assigned’, ’postponed’, ’opened’, ’fixed’, ’res
    olved’, ’closed’], true)) {
            $whereSql        = ’WHERE COALESCE(a.‘status‘, \’submitted\’) = :sort’;
            $params[’:sort’] = $sort;
        }

        $orderBySql = match ($sort) {
            ’date’ => ’r.‘Submitted‘ DESC, r.‘ID‘ DESC’,
            default => "FIELD(COALESCE(a.‘priority‘, ’average’), ’most urgent’, ’average’, ’less
     urgent’) ASC, r.‘Submitted‘ DESC",
        };

        $sql = "
            SELECT
                r.‘ID‘ AS ‘id‘,
                r.‘Product‘ AS ‘product‘,
                r.‘Title‘ AS ‘title‘,
                r.‘Description‘ AS ‘report_description‘,
                r.‘Environment‘ AS ‘environment‘,
                r.‘Occured‘ AS ‘occured‘,
                r.‘Contactemail‘ AS ‘contact_email‘,
                r.‘Who‘ AS ‘who‘,
                r.‘Severity‘ AS ‘report_severity‘,
                r.‘Priority‘ AS ‘report_priority‘,
                r.‘Submitted‘ AS ‘report_submitted‘,
                a.‘devdesc‘,
```

```
            COALESCE(a.‘classification‘, ’Other’) AS ‘classification‘,
            a.‘dateoriginated‘,
            a.‘phase‘,
            a.‘actoriginated‘,
            a.‘deliverable‘,
            a.‘whattodo‘,
            COALESCE(a.‘submitted‘, r.‘Submitted‘) AS ‘submitted‘,
            a.‘assigned‘,
            COALESCE(a.‘uniqueness‘, ’New defect’) AS ‘uniqueness‘,
            COALESCE(a.‘severity‘, r.‘Severity‘) AS ‘severity‘,
            COALESCE(
                a.‘priority‘,
                CASE r.‘Priority‘
                    WHEN ’m_urgent’ THEN ’most urgent’
                    WHEN ’l_urgent’ THEN ’less urgent’
                    ELSE ’average’
                END
            ) AS ‘priority‘,
            a.‘workaround‘,
            a.‘proposedfix‘,
            COALESCE(a.‘status‘, ’submitted’) AS ‘status‘,
            a.‘fixdesc‘,
            a.‘fixstarted‘,
            a.‘fixcomplete‘,
            a.‘fiximpact‘,
            COALESCE(a.‘fixeffort‘, 0.00) AS ‘fixeffort‘,
            COALESCE(a.‘fixcost‘, 0.00) AS ‘fixcost‘,
            a.‘resodesc‘,
            a.‘resocomplete‘,
            a.‘resoclosed‘,
            COALESCE(a.‘costofdefect‘, 0.00) AS ‘costofdefect‘,
            a.‘actfound‘,
            DATEDIFF(CURDATE(), COALESCE(a.‘submitted‘, r.‘Submitted‘)) AS ‘days_open‘,
            CASE WHEN a.‘assigned‘ IS NOT NULL
                 THEN DATEDIFF(a.‘assigned‘, COALESCE(a.‘submitted‘, r.‘Submitted‘))
                 ELSE NULL END AS ‘submitted_to_assigned_days‘,
            CASE WHEN a.‘fixcomplete‘ IS NOT NULL
                 THEN DATEDIFF(a.‘fixcomplete‘, COALESCE(a.‘submitted‘, r.‘Submitted‘))
                 ELSE NULL END AS ‘submitted_to_fixed_days‘,
            CASE WHEN a.‘resocomplete‘ IS NOT NULL
                 THEN DATEDIFF(a.‘resocomplete‘, COALESCE(a.‘submitted‘, r.‘Submitted‘))
                 ELSE NULL END AS ‘submitted_to_resolved_days‘,
            CASE WHEN a.‘fixcomplete‘ IS NOT NULL AND a.‘assigned‘ IS NOT NULL
                 THEN DATEDIFF(a.‘fixcomplete‘, a.‘assigned‘)
                 ELSE NULL END AS ‘assigned_to_fixed_days‘,
            CASE WHEN a.‘resocomplete‘ IS NOT NULL AND a.‘assigned‘ IS NOT NULL
                 THEN DATEDIFF(a.‘resocomplete‘, a.‘assigned‘)
                 ELSE NULL END AS ‘assigned_to_resolved_days‘
        FROM ‘raport‘ r
        LEFT JOIN ‘analyse‘ a ON a.‘id‘ = r.‘ID‘
        {$whereSql}
        ORDER BY {$orderBySql}
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute($params);
    $rows = $stmt->fetchAll();

    foreach ($rows as &$row) {
        $faultEnum = IeeeFaultType::fromLegacyString((string) $row[’classification’]);
        $row[’classification_color’] = sprintf(’#%06X’, $faultEnum->hexColor());
        $row[’phases’] = !empty($row[’phase’]) ? explode(’,’, (string) $row[’phase’]) : [];
    }

    return $rows;
}
```

```
    /**
     * Upserts initial developer analysis & IEEE (1992) classification (form_analyze.php)
     */
    public function upsertAnalysis(int $defectId, array $data): void
    {
        $phases = is_array($data['phases'] ?? null)
            ? implode(',', array_map('strip_tags', $data['phases']))
            : strip_tags((string) ($data['phase'] ?? ''));

        $classification = !empty($data['classification_other'])
            ? trim(strip_tags((string) $data['classification_other']))
            : IeeeFaultType::fromLegacyString((string) ($data['classification'] ?? 'Logic'))->va
    lue;

        $uniqueness = Uniqueness::from($data['uniqueness'] ?? 'New defect')->value;
        $severity   = DefectSeverity::from($data['severity'] ?? 'minor')->value;
        $priority   = DefectPriority::fromInput($data['priority'] ?? 'average')->analysisLabel()
    ;
        $status     = DefectStatus::from($data['status'] ?? 'opened')->value;

        $stmt = $this->pdo->prepare(
            'INSERT INTO `analyse`
             (`id`, `devdesc`, `classification`, `dateoriginated`, `phase`, `actoriginated`,
              `deliverable`, `whattodo`, `submitted`, `assigned`, `uniqueness`, `priority`,
              `severity`, `workaround`, `proposedfix`, `status`, `actfound`)
             VALUES
             (:id, :devdesc, :classification, :dateoriginated, :phase, :actoriginated,
              :deliverable, :whattodo, :submitted, :assigned, :uniqueness, :priority,
              :severity, :workaround, :proposedfix, :status, :actfound)
             ON DUPLICATE KEY UPDATE
              `devdesc`        = VALUES(`devdesc`),
              `classification` = VALUES(`classification`),
              `dateoriginated` = VALUES(`dateoriginated`),
              `phase`          = VALUES(`phase`),
              `actoriginated`  = VALUES(`actoriginated`),
              `deliverable`    = VALUES(`deliverable`),
              `whattodo`       = VALUES(`whattodo`),
              `submitted`      = VALUES(`submitted`),
              `assigned`       = VALUES(`assigned`),
              `uniqueness`     = VALUES(`uniqueness`),
              `priority`       = VALUES(`priority`),
              `severity`       = VALUES(`severity`),
              `workaround`     = VALUES(`workaround`),
              `proposedfix`    = VALUES(`proposedfix`),
              `status`         = VALUES(`status`),
              `actfound`       = VALUES(`actfound`)'
        );

        $stmt->execute([
            ':id'             => $defectId,
            ':devdesc'        => trim(strip_tags((string) $data['devdesc'])),
            ':classification' => $classification,
            ':dateoriginated' => $data['dateoriginated'],
            ':phase'          => $phases,
            ':actoriginated'  => trim(strip_tags((string) $data['actoriginated'])),
            ':deliverable'    => trim(strip_tags((string) $data['deliverable'])),
            ':whattodo'       => trim(strip_tags((string) ($data['whattodo'] ?? ''))),
            ':submitted'      => $data['submitted'] ?? date('Y-m-d'),
            ':assigned'       => $data['assigned'],
            ':uniqueness'     => $uniqueness,
            ':priority'       => $priority,
            ':severity'       => $severity,
            ':workaround'     => trim(strip_tags((string) ($data['workaround'] ?? ''))),
            ':proposedfix'    => trim(strip_tags((string) $data['proposedfix'])),
            ':status'         => $status,
```

```
            ':actfound'       => trim(strip_tags((string) $data['actfound'])),
        ]);
    }

    /**
     * Upserts defect fix effort, monetary cost & resolution details
     * Refactored from: form_deffix.php & form_defreso.php
     */
    public function upsertResolutionAndCost(int $defectId, array $data): void
    {
        $status = isset($data['status'])
            ? DefectStatus::from((string) $data['status'])->value
            : DefectStatus::RESOLVED->value;

        $stmt = $this->pdo->prepare(
            'INSERT INTO `analyse`
             (`id`, `status`, `fixdesc`, `fixstarted`, `fixcomplete`, `fiximpact`,
              `fixeffort`, `fixcost`, `resodesc`, `resocomplete`, `resoclosed`, `costofdefect`)
             VALUES
             (:id, :status, :fixdesc, :fixstarted, :fixcomplete, :fiximpact,
              :fixeffort, :fixcost, :resodesc, :resocomplete, :resoclosed, :costofdefect)
             ON DUPLICATE KEY UPDATE
              `status`       = VALUES(`status`),
              `fixdesc`      = COALESCE(VALUES(`fixdesc`), `fixdesc`),
              `fixstarted`   = COALESCE(VALUES(`fixstarted`), `fixstarted`),
              `fixcomplete`  = COALESCE(VALUES(`fixcomplete`), `fixcomplete`),
              `fiximpact`    = COALESCE(VALUES(`fiximpact`), `fiximpact`),
              `fixeffort`    = COALESCE(VALUES(`fixeffort`), `fixeffort`),
              `fixcost`      = COALESCE(VALUES(`fixcost`), `fixcost`),
              `resodesc`     = COALESCE(VALUES(`resodesc`), `resodesc`),
              `resocomplete` = COALESCE(VALUES(`resocomplete`), `resocomplete`),
              `resoclosed`   = COALESCE(VALUES(`resoclosed`), `resoclosed`),
              `costofdefect` = COALESCE(VALUES(`costofdefect`), `costofdefect`)'
        );

        $stmt->execute([
            ':id'           => $defectId,
            ':status'       => $status,
            ':fixdesc'      => isset($data['fixdesc']) ? trim(strip_tags((string) $data['fixdesc
    ']) : null,
            ':fixstarted'   => !empty($data['fixstarted']) ? $data['fixstarted'] : null,
            ':fixcomplete'  => !empty($data['fixcomplete']) ? $data['fixcomplete'] : null,
            ':fiximpact'    => isset($data['fiximpact']) ? trim(strip_tags((string) $data['fixim
    pact'])) : null,
            ':fixeffort'    => isset($data['fixeffort']) ? (float) $data['fixeffort'] : null,
            ':fixcost'      => isset($data['fixcost']) ? (float) $data['fixcost'] : null,
            ':resodesc'     => isset($data['resodesc']) ? trim(strip_tags((string) $data['resode
    sc'])) : null,
            ':resocomplete' => !empty($data['resocomplete']) ? $data['resocomplete'] : null,
            ':resoclosed'   => !empty($data['resoclosed']) ? $data['resoclosed'] : null,
            ':costofdefect' => isset($data['costofdefect']) ? (float) $data['costofdefect'] : nu
    ll,
        ]);
    }

    /**
     * Deletes analysis entry only (db.php?function=deleteanalyze)
     */
    public function delete(int $defectId): bool
    {
        $stmt = $this->pdo->prepare('DELETE FROM `analyse` WHERE `id` = :id');
        $stmt->execute([':id' => $defectId]);
        return $stmt->rowCount() > 0;
    }
}
```

```
================================================================================
SECTION 2.6 RBAC Middleware (modern/backend/src/Http/Middleware/RbacMiddleware.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Middleware\RbacMiddleware
    .php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Http\Middleware;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Repository\UserRepository;

/**
 * RBAC Authentication & Authorization Middleware
 * Enforces the 4 DefMan user levels:
 *   - Level 1 (Public):   Unauthenticated / public defect submission
 *   - Level 2 (Customer): Authenticated user tracking their own submitted defects
 *   - Level 3 (Analyzer): Developer/QA triage, IEEE fault classification, fix & cost tracking
 *   - Level 4 (Admin):    User registration approval queue, role assignment, product management
 */
final readonly class RbacMiddleware
{
    public function __construct(private UserRepository $userRepository) {}

    public function extractBearerToken(): ?string
    {
        $header = $_SERVER['HTTP_AUTHORIZATION'] ?? $_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '
    ';
        if (preg_match('/^Bearer\s+([a-f0-9]{64})$/i', $header, $matches)) {
            return $matches[1];
        }
        return $_COOKIE['defman_token'] ?? null;
    }

    public function authenticateOptional(): ?array
    {
        $token = $this->extractBearerToken();
        if ($token === null) {
            return null;
        }
        return $this->userRepository->findUserBySessionToken($token);
    }

    public function requireLevel(UserLevel $minimumLevel): array
    {
        if ($minimumLevel === UserLevel::PUBLIC) {
            return $this->authenticateOptional() ?? [
                'id'        => null,
                'login'     => 'public',
                'userlevel' => UserLevel::PUBLIC->value,
            ];
        }

        $user = $this->authenticateOptional();
        if ($user === null) {
            http_response_code(401);
            header('Content-Type: application/json');
            echo json_encode(['error' => 'Authentication required.']);
            exit;
        }

        $userLevel = (int) $user['userlevel'];
```

```
        if ($userLevel < $minimumLevel->value) {
            http_response_code(403);
            header('Content-Type: application/json');
            echo json_encode([
                'error'          => 'Insufficient permissions.',
                'required_level' => $minimumLevel->label(),
                'current_level'  => UserLevel::tryFrom($userLevel)?->label() ?? 'Unknown',
            ]);
            exit;
        }

        return $user;
    }
}
```

==========

SECTION 2.7 AuthController (modern/backend/src/Http/Controller/AuthController.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\AuthController.php

==========

```
<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\UserRepository;

/**
 * Authentication & Password Recovery Controller
 * Refactored from: login.php, logout.php, randompassword.php, form_lostpassword.php, form_apply.php
 */
final readonly class AuthController
{
    public function __construct(
        private UserRepository $userRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * POST /api/auth/login (Refactored from login.php)
     */
    public function login(array $body): array
    {
        $username = trim((string) ($body['username'] ?? ''));
        $password = (string) ($body['password'] ?? '');

        if ($username === '' || $password === '') {
            http_response_code(422);
            return ['error' => 'Username and password are required.'];
        }

        $user = $this->userRepository->findActiveByLogin($username);
        if ($user === null || !password_verify($password, (string) $user['password'])) {
            http_response_code(401);
            return ['error' => 'Authentication failed'];
        }

        if (password_needs_rehash((string) $user['password'], PASSWORD_ARGON2ID)) {
            $this->userRepository->updatePassword((int) $user['id'], $password);
        }
```

```
        $token = $this->userRepository->createSessionToken(
            (int) $user['id'],
            $_SERVER['REMOTE_ADDR'] ?? null,
            $_SERVER['HTTP_USER_AGENT'] ?? null
        );

        return [
            'token' => $token,
            'user'  => [
                'id'        => (int) $user['id'],
                'login'     => $user['login'],
                'email'     => $user['email'],
                'userlevel' => (int) $user['userlevel'],
                'role'      => UserLevel::from((int) $user['userlevel'])->label(),
                'name'      => $user['name'],
                'company'   => $user['company'],
                'phone'     => $user['phone'],
            ],
        ];
    }

    /**
     * POST /api/auth/logout (Refactored from logout.php)
     */
    public function logout(): array
    {
        $user  = $this->rbac->authenticateOptional();
        $token = $this->rbac->extractBearerToken();

        if ($user !== null && $token !== null) {
            $this->userRepository->revokeSessionToken($token, (int) $user['id']);
        }

        return ['message' => 'Logged out'];
    }

    /**
     * POST /api/auth/apply (Refactored from form_apply.php)
     */
    public function apply(array $body): array
    {
        $username       = trim((string) ($body['username'] ?? ''));
        $password       = (string) ($body['password'] ?? '');
        $verifyPassword = (string) ($body['verify_password'] ?? '');
        $email          = trim((string) ($body['email'] ?? ''));
        $fullName       = trim(strip_tags((string) ($body['name'] ?? '')));
        $company        = trim(strip_tags((string) ($body['company'] ?? '')));
        $phone          = trim(strip_tags((string) ($body['phone'] ?? '')));

        if ($email === '') {
            http_response_code(422);
            return ['error' => 'Empty E-mail!'];
        }
        if ($username === '') {
            http_response_code(422);
            return ['error' => 'Empty Username!'];
        }
        if ($password === '' || $verifyPassword === '') {
            http_response_code(422);
            return ['error' => 'Empty Password!'];
        }
        if ($password !== $verifyPassword) {
            http_response_code(422);
            return ['error' => 'Passwords do not match!'];
        }
```

```
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            http_response_code(422);
            return ['error' => 'E-mail is not valid'];
        }
        if ($this->userRepository->existsByEmail($email)) {
            http_response_code(409);
            return ['error' => 'Someone has already selected the e-mail address you typed.'];
        }
        if ($this->userRepository->existsByLogin($username)) {
            http_response_code(409);
            return ['error' => 'Someone has already selected the username you typed.'];
        }

        $userId = $this->userRepository->create(
            login: $username,
            plainPassword: $password,
            email: $email,
            userLevel: UserLevel::APPLYING,
            company: $company ?: null,
            phone: $phone ?: null,
            name: $fullName ?: null,
            activate: false
        );

        http_response_code(201);
        return [
            'id'      => $userId,
            'message' => 'Your application has been sent.',
        ];
    }

    /**
     * POST /api/auth/lost-password (Refactored from form_lostpassword.php)
     * Uses cryptographically secure random_bytes() reset token instead of emailing plaintext pa
    sswords.
     */
    public function lostPassword(array $body): array
    {
        $email = trim((string) ($body['email'] ?? ''));
        if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            http_response_code(422);
            return ['error' => 'E-mail not found'];
        }

        $user = $this->userRepository->findActiveByEmail($email);
        if ($user === null) {
            http_response_code(404);
            return ['error' => 'E-mail not found'];
        }

        $resetToken = $this->userRepository->createPasswordResetToken((int) $user['id']);

        return [
            'message'     => 'Information sent to given address.',
            'reset_token' => $resetToken, // Exposed in dev/response for testable workflow
        ];
    }

    /**
     * Cryptographically secure replacement for randompassword.php
     */
    public static function generateSecurePassword(int $length = 12): string
    {
        $alphabet = 'ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz23456789!@#$%';
        $maxIndex = strlen($alphabet) - 1;
        $password = '';
```

```
        for ($i = 0; $i < $length; $i++) {
            $password .= $alphabet[random_int(0, $maxIndex)];
        }
        return $password;
    }
}
```

```
================================================================================
SECTION 2.8 UserController (modern/backend/src/Http/Controller/UserController.php)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\UserController
    .php
================================================================================
```

```
<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\UserRepository;

/**
 * User & Admin Account Management Controller
 * Refactored from: form_usermanagement.php, form_newuser.php, form_userdetails.php, db.php
 */
final readonly class UserController
{
    public function __construct(
        private UserRepository $userRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * GET /api/admin/users (Refactored from form_usermanagement.php)
     */
    public function listUsers(): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);
        $lists = $this->userRepository->listAllByStatus();
        $lists['generated_password_suggestion'] = AuthController::generateSecurePassword(10);
        return $lists;
    }

    /**
     * POST /api/admin/users (Refactored from form_newuser.php)
     */
    public function createUser(array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        $username = trim((string) ($body['username'] ?? ''));
        $email    = trim((string) ($body['email'] ?? ''));
        $password = (string) ($body['password'] ?? AuthController::generateSecurePassword(10));
        $levelInt = (int) ($body['userlevel'] ?? UserLevel::CUSTOMER->value);
        $level    = UserLevel::tryFrom($levelInt) ?? UserLevel::CUSTOMER;

        if ($email === '') {
            http_response_code(422);
            return ['error' => 'Empty E-mail!'];
        }
        if ($username === '' || preg_match('/[^A-Za-z0-9]/', $username)) {
            http_response_code(422);
            return ['error' => 'Illegal characters in username or too short username'];
```

```
        }
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            http_response_code(422);
            return ['error' => 'Bad e-mail address.'];
        }
        if ($this->userRepository->existsByEmail($email)) {
            http_response_code(409);
            return ['error' => 'There is user with that e-mail already.'];
        }
        if ($this->userRepository->existsByLogin($username)) {
            http_response_code(409);
            return ['error' => 'There is user with that username already.'];
        }

        $id = $this->userRepository->create(
            login: $username,
            plainPassword: $password,
            email: $email,
            userLevel: $level,
            activate: true
        );

        http_response_code(201);
        return [
            'id'      => $id,
            'message' => 'User added. Mail sent.',
        ];
    }

    /**
     * PATCH /api/admin/users/{id} (Refactored from db.php?function=accept & role assignment)
     */
    public function updateUserStatus(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        $action   = (string) ($body['action'] ?? 'approve');
        $levelInt = (int) ($body['userlevel'] ?? UserLevel::CUSTOMER->value);
        $level    = UserLevel::tryFrom($levelInt) ?? UserLevel::CUSTOMER;

        if ($action === 'approve') {
            $this->userRepository->approveApplicant($id, $level);
            return ['message' => "User #{$id} approved with role {$level->label()}."];
        }

        if ($action === 'role') {
            $this->userRepository->updateRole($id, $level);
            return ['message' => "User #{$id} role updated to {$level->label()}."];
        }

        http_response_code(400);
        return ['error' => 'Unsupported action.'];
    }

    /**
     * DELETE /api/admin/users/{id} (Refactored from db.php?function=delete)
     */
    public function deleteUser(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);
        $deleted = $this->userRepository->delete($id);

        if (!$deleted) {
            http_response_code(400);
            return ['error' => 'Cannot delete Admin user or non-existent user.'];
        }
```

```
        return ['message' => "User #{$id} deleted."];
    }

    /**
     * GET /api/users/me (Refactored from form_userdetails.php)
     */
    public function getMe(): array
    {
        $user = $this->rbac->requireLevel(UserLevel::CUSTOMER);
        return [
            'id'        => (int) $user['id'],
            'login'     => ucfirst((string) $user['login']),
            'email'     => $user['email'],
            'name'      => $user['name'] ?? '',
            'company'   => $user['company'] ?? '',
            'phone'     => $user['phone'] ?? '',
            'userlevel' => (int) $user['userlevel'],
        ];
    }

    /**
     * PUT /api/users/me (Refactored from form_userdetails.php: updatepassword, updateemail, upd
    atepersonal)
     */
    public function updateMe(array $body): array
    {
        $user   = $this->rbac->requireLevel(UserLevel::CUSTOMER);
        $userId = (int) $user['id'];
        $section = (string) ($body['section'] ?? 'personal');

        if ($section === 'password') {
            $newPass    = (string) ($body['change_password'] ?? '');
            $verifyPass = (string) ($body['verify_password'] ?? '');

            if ($newPass === '') {
                http_response_code(422);
                return ['error' => 'Empty New Password'];
            }
            if ($verifyPass === '') {
                http_response_code(422);
                return ['error' => 'Empty Verify Password'];
            }
            if ($newPass !== $verifyPass) {
                http_response_code(422);
                return ['error' => 'Passwords do not match'];
            }

            $this->userRepository->updatePassword($userId, $newPass);
            return ['message' => 'Settings changed'];
        }

        if ($section === 'email') {
            $email = trim(strip_tags((string) ($body['email'] ?? '')));
            if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
                http_response_code(422);
                return ['error' => 'E-mail not valid!'];
            }
            if ($this->userRepository->existsByEmail($email, $userId)) {
                http_response_code(409);
                return ['error' => 'E-mail already in use!'];
            }

            $this->userRepository->updateEmail($userId, $email);
            return ['message' => 'E-mail changed!'];
        }
```

```
        $name    = trim(strip_tags((string) ($body['name'] ?? '')));
        $company = trim(strip_tags((string) ($body['company'] ?? '')));
        $phone   = trim(strip_tags((string) ($body['phone'] ?? '')));

        $this->userRepository->updatePersonalDetails($userId, $name, $company, $phone);
        return ['message' => 'Personal information updated!'];
    }
}
```

```
================================================================================
```

SECTION 2.9 DefectController (modern/backend/src/Http/Controller/DefectController.php)

```
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\DefectControll
    er.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\AnalysisRepository;
use DefMan\Repository\DefectRepository;

/**
 * Defect & IEEE (1992) Fault Analysis Controller
 * Refactored from: form_raportpub.php, form_raport.php, form_raportlist.php,
 *                  form_list.php, form_defrap.php, form_alist.php,
 *                  form_analyze.php, form_analyzed.php, form_deffix.php, form_defreso.php
 */
final readonly class DefectController
{
    public function __construct(
        private DefectRepository $defectRepository,
        private AnalysisRepository $analysisRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * POST /api/defects/public (Refactored from form_raportpub.php -- Level 1 Public)
     */
    public function createPublicReport(array $body): array
    {
        $this->rbac->requireLevel(UserLevel::PUBLIC);

        $errors = $this->validateReportFields($body, isPublic: true);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $id = $this->defectRepository->create([
            'product'       => $body['product'],
            'environment'   => $body['environment'] ?? '',
            'customer_id'   => null,
            'who'           => $body['who'],
            'contact_email' => $body['contact_email'],
            'title'         => $body['title'],
            'description'   => $body['description'],
            'discovered_by' => $body['discovered_by'] ?? '',
            'persontype'    => $body['persontype'],
            'occured'       => $body['occured'],
```

```
            'location'      => $body['location'] ?? '',
            'severity'      => $body['severity'],
            'priority'      => $body['priority'],
            'addinfo'       => $body['addinfo'] ?? '',
        ]);

        http_response_code(201);
        return [
            'id'      => $id,
            'message' => 'Report has been stored to the database',
        ];
    }

    /**
     * POST /api/defects (Refactored from form_raport.php -- Level 2+ Customer)
     */
    public function createCustomerReport(array $body): array
    {
        $user = $this->rbac->requireLevel(UserLevel::CUSTOMER);

        $errors = $this->validateReportFields($body, isPublic: false);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $id = $this->defectRepository->create([
            'product'       => $body['product'],
            'environment'   => $body['environment'] ?? '',
            'customer_id'   => (int) $user['id'],
            'who'           => (string) $user['login'],
            'contact_email' => (string) $user['email'],
            'title'         => $body['title'],
            'description'   => $body['description'],
            'discovered_by' => $body['discovered_by'] ?? ($user['name'] ?? $user['login']),
            'persontype'    => $body['persontype'],
            'occured'       => $body['occured'],
            'location'      => $body['location'] ?? '',
            'severity'      => $body['severity'],
            'priority'      => $body['priority'],
            'addinfo'       => $body['addinfo'] ?? '',
        ]);

        http_response_code(201);
        return [
            'id'      => $id,
            'message' => 'Report has been stored into the database',
        ];
    }

    /**
     * GET /api/defects (Refactored from form_raportlist.php & form_list.php)
     */
    public function listDefects(array $query): array
    {
        $user = $this->rbac->requireLevel(UserLevel::CUSTOMER);

        // Customers (Level 2) default to viewing their own defects unless Level 3+ Analyzer/Adm
    in
        $customerId = isset($query['customer_id']) && $query['customer_id'] !== ''
            ? (int) $query['customer_id']
            : ((int) $user['userlevel'] === UserLevel::CUSTOMER->value ? (int) $user['id'] : nul
    l);

        $defects = $this->defectRepository->findFiltered(
            severity: $query['severity'] ?? null,
```

```
            priority: $query['priority'] ?? null,
            status: $query['status'] ?? null,
            product: $query['product'] ?? null,
            customerId: $customerId,
            sortBy: (string) ($query['sort'] ?? 'date')
        );

        return [
            'count'   => count($defects),
            'defects' => $defects,
        ];
    }

    /**
     * GET /api/defects/{id} (Refactored from form_defrap.php)
     */
    public function getDefect(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::CUSTOMER);
        $defect = $this->defectRepository->findById($id);

        if ($defect === null) {
            http_response_code(404);
            return ['error' => "Defect report #{$id} not found."];
        }

        return $defect;
    }

    /**
     * PUT /api/defects/{id} (Refactored from form_defrap.php update)
     */
    public function updateDefectReport(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $errors = $this->validateReportFields($body, isPublic: false);
        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $this->defectRepository->update($id, [
            'product'       => $body['product'],
            'environment'   => $body['environment'] ?? '',
            'title'         => $body['title'],
            'description'   => $body['description'],
            'discovered_by' => $body['discovered_by'] ?? '',
            'persontype'    => $body['persontype'],
            'occured'       => $body['occured'],
            'location'      => $body['location'] ?? '',
            'severity'      => $body['severity'],
            'priority'      => $body['priority'],
            'submitted'     => $body['submitted'] ?? date('Y-m-d'),
            'addinfo'       => $body['addinfo'] ?? '',
        ]);

        return ['message' => "You updated the defect raport #{$id} in the database."];
    }

    /**
     * DELETE /api/defects/{id} (Refactored from db.php?function=deleteraport)
     */
    public function deleteDefect(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);
```

```
        $this->defectRepository->delete($id);
        return ['message' => "Defect #{$id} and its analysis deleted."];
    }

    /**
     * GET /api/analyzer/queue (Refactored from form_alist.php & form_analyzed.php)
     */
    public function getAnalyzerQueue(array $query): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);
        $items = $this->analysisRepository->getAnalyzerQueue($query['sort'] ?? 'all');

        return [
            'count' => count($items),
            'queue' => $items,
        ];
    }

    /**
     * PUT /api/defects/{id}/analyze (Refactored from form_analyze.php)
     */
    public function analyzeDefect(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $errors = [];
        if (empty(trim((string) ($body['devdesc'] ?? '')))) {
            $errors[] = 'Empty Defect description!';
        }
        if (empty($body['classification']) && empty($body['classification_other'])) {
            $errors[] = 'Empty Classification!';
        }
        if (empty($body['dateoriginated']) || !$this->isValidDate((string) $body['dateoriginated
    ']))) {
            $errors[] = 'Invalid Date defect originated!';
        }
        if (empty($body['phases']) && empty($body['phase'])) {
            $errors[] = 'Empty Phase originated!';
        }
        if (empty(trim((string) ($body['actoriginated'] ?? '')))) {
            $errors[] = 'Empty Activity which originated!';
        }
        if (empty(trim((string) ($body['deliverable'] ?? '')))) {
            $errors[] = 'Empty Deliverable originated!';
        }
        if (empty(trim((string) ($body['actfound'] ?? '')))) {
            $errors[] = 'Empty Activity that should have found!';
        }
        if (empty($body['assigned']) || !$this->isValidDate((string) $body['assigned'])) {
            $errors[] = 'Invalid Date assigned!';
        }
        if (empty(trim((string) ($body['proposedfix'] ?? '')))) {
            $errors[] = 'Empty Proposed fix!';
        }

        if (count($errors) > 0) {
            http_response_code(422);
            return ['errors' => $errors];
        }

        $this->analysisRepository->upsertAnalysis($id, $body);
        return ['message' => "You updated analyze report #{$id} in the database."];
    }

    /**
     * PUT /api/defects/{id}/resolve (Refactored from form_deffix.php & form_defreso.php)
```

```
     */
    public function resolveDefect(int $id, array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ANALYZER);

        $this->analysisRepository->upsertResolutionAndCost($id, $body);
        return [
            'id'      => $id,
            'status'  => $body['status'] ?? 'resolved',
            'message' => "You updated the analyze report #{$id} in the database.",
        ];
    }

    /**
     * Validates report fields per form_raport.php & form_raportpub.php
     *
     * @return list<string>
     */
    private function validateReportFields(array $body, bool $isPublic): array
    {
        $errors = [];

        if ($isPublic) {
            $who   = trim((string) ($body['who'] ?? ''));
            $email = trim((string) ($body['contact_email'] ?? ''));
            if ($who === '') {
                $errors[] = 'Empty Who reported the problem!';
            } elseif (strlen($who) > 64) {
                $errors[] = 'Who reported the problem is too long!';
            }
            if ($email === '') {
                $errors[] = 'Empty Contact email!';
            } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
                $errors[] = 'Bad Contact email address!';
            }
        }

        if (empty(trim((string) ($body['title'] ?? '')))) {
            $errors[] = 'Empty Title!';
        }
        if (empty(trim((string) ($body['description'] ?? '')))) {
            $errors[] = 'Empty Description!';
        }
        if (empty($body['occured']) || !$this->isValidDate((string) $body['occured'])) {
            $errors[] = 'Invalid Date Problem occurred!';
        }
        if (empty($body['product'])) {
            $errors[] = 'Empty Product!';
        }
        if (empty($body['severity'])) {
            $errors[] = 'Empty Severity!';
        }
        if (empty($body['priority'])) {
            $errors[] = 'Empty Priority!';
        }
        if (empty($body['persontype'])) {
            $errors[] = 'Empty Type of person!';
        }

        return $errors;
    }

    /**
     * Preserves 2004 date bounds validation (1980 < year < 2100) from form_raport.php
     */
    private function isValidDate(string $dateStr): bool
```

```
    {
        $parts = explode('-', $dateStr);
        if (count($parts) !== 3) {
            return false;
        }
        [$year, $month, $day] = array_map('intval', $parts);
        return $year > 1980 && $year < 2100 && checkdate($month, $day, $year);
    }
}
```

```
================================================================================
```

SECTION 2.10 ProductController (modern/backend/src/Http/Controller/ProductController.php)

```
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\src\Http\Controller\ProductControl
    ler.php
================================================================================

<?php
declare(strict_types=1);

namespace DefMan\Http\Controller;

use DefMan\Domain\Enum\UserLevel;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Repository\ProductRepository;

/**
 * Product Management Controller
 * Refactored from: form_productdetails.php
 */
final readonly class ProductController
{
    public function __construct(
        private ProductRepository $productRepository,
        private RbacMiddleware $rbac
    ) {}

    /**
     * GET /api/products (Available to all levels so Public/Customer forms can populate dropdown
    s)
     */
    public function listProducts(): array
    {
        return ['products' => $this->productRepository->findAll()];
    }

    /**
     * POST /api/admin/products (Refactored from form_productdetails.php?function=newproduct)
     */
    public function createProduct(array $body): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        $name = trim(strip_tags((string) ($body['name'] ?? '')));
        if ($name === '') {
            http_response_code(422);
            return ['error' => 'Empty Product name!'];
        }

        if ($this->productRepository->findByName($name) !== null) {
            http_response_code(409);
            return ['error' => "Product '{$name}' already exists."];
        }

        $id = $this->productRepository->create($name);
```

```
        http_response_code(201);
        return [
            'id'      => $id,
            'name'    => $name,
            'message' => "You added new product: {$name} in database.",
        ];
    }

    /**
     * DELETE /api/admin/products/{id} (Refactored from form_productdetails.php?function=deletep
    roduct)
     */
    public function deleteProduct(int $id): array
    {
        $this->rbac->requireLevel(UserLevel::ADMIN);

        $deletedName = $this->productRepository->delete($id);
        if ($deletedName === null) {
            http_response_code(404);
            return ['error' => 'Product not found.'];
        }

        return ['message' => "You deleted product: {$deletedName} from the database."];
    }
}
```

==========

SECTION 2.11 REST Front Controller (modern/backend/public/index.php)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\backend\public\index.php

==========

```
<?php
declare(strict_types=1);

/**
 * DefMan REST API Front Controller
 * Replaces monolithic 2004 index.php + menubar.php + db.php
 */

use DefMan\Http\Controller\AuthController;
use DefMan\Http\Controller\DefectController;
use DefMan\Http\Controller\ProductController;
use DefMan\Http\Controller\UserController;
use DefMan\Http\Middleware\RbacMiddleware;
use DefMan\Infrastructure\Database\Database;
use DefMan\Repository\AnalysisRepository;
use DefMan\Repository\DefectRepository;
use DefMan\Repository\ProductRepository;
use DefMan\Repository\UserRepository;

spl_autoload_register(static function (string $class): void {
    $prefix = 'DefMan\\';
    if (!str_starts_with($class, $prefix)) {
        return;
    }
    $relative = substr($class, strlen($prefix));
    $file = __DIR__ . '/../src/' . str_replace('\\', '/', $relative) . '.php';
    if (is_file($file)) {
        require_once $file;
    }
});

header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');
```

```
header('Access-Control-Allow-Headers: Content-Type, Authorization');
header('Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit;
}

try {
    $pdo            = Database::getConnection();
    $userRepo       = new UserRepository($pdo);
    $productRepo    = new ProductRepository($pdo);
    $defectRepo     = new DefectRepository($pdo);
    $analysisRepo   = new AnalysisRepository($pdo);
    $rbac           = new RbacMiddleware($userRepo);

    $authCtrl       = new AuthController($userRepo, $rbac);
    $userCtrl       = new UserController($userRepo, $rbac);
    $defectCtrl     = new DefectController($defectRepo, $analysisRepo, $rbac);
    $productCtrl    = new ProductController($productRepo, $rbac);

    $method = $_SERVER['REQUEST_METHOD'];
    $uri    = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH) ?: '/';
    $rawBody = file_get_contents('php://input');
    $body   = $rawBody ? (json_decode($rawBody, true) ?? []) : [];

    $response = match (true) {
        // Auth & Registration
        $method === 'POST' && $uri === '/api/auth/login'         => $authCtrl->login($body),
        $method === 'POST' && $uri === '/api/auth/logout'        => $authCtrl->logout(),
        $method === 'POST' && $uri === '/api/auth/apply'         => $authCtrl->apply($body),
        $method === 'POST' && $uri === '/api/auth/lost-password' => $authCtrl->lostPassword($bod
    y),

        // User Profile (Level 2+)
        $method === 'GET'  && $uri === '/api/users/me'           => $userCtrl->getMe(),
        $method === 'PUT'  && $uri === '/api/users/me'           => $userCtrl->updateMe($body),

        // Admin User Approval & Management (Level 4)
        $method === 'GET'    && $uri === '/api/admin/users'      => $userCtrl->listUsers(),
        $method === 'POST'   && $uri === '/api/admin/users'      => $userCtrl->createUser($body)
    ,
        $method === 'PATCH'  && preg_match('#^/api/admin/users/(\d+)$#', $uri, $m) === 1 => $use
    rCtrl->updateUserStatus((int) $m[1], $body),
        $method === 'DELETE' && preg_match('#^/api/admin/users/(\d+)$#', $uri, $m) === 1 => $use
    rCtrl->deleteUser((int) $m[1]),

        // Products (Public GET, Level 4 Admin POST/DELETE)
        $method === 'GET'    && ($uri === '/api/products' || $uri === '/api/admin/products') =>
    $productCtrl->listProducts(),
        $method === 'POST'   && $uri === '/api/admin/products'   => $productCtrl->createProduct(
    $body),
        $method === 'DELETE' && preg_match('#^/api/admin/products/(\d+)$#', $uri, $m) === 1 => $
    productCtrl->deleteProduct((int) $m[1]),

        // Public & Customer Defect Reporting
        $method === 'POST'   && $uri === '/api/defects/public'   => $defectCtrl->createPublicRep
    ort($body),
        $method === 'POST'   && $uri === '/api/defects'          => $defectCtrl->createCustomerR
    eport($body),
        $method === 'GET'    && $uri === '/api/defects'          => $defectCtrl->listDefects($_G
    ET),
        $method === 'GET'    && preg_match('#^/api/defects/(\d+)$#', $uri, $m) === 1 => $defectC
    trl->getDefect((int) $m[1]),
        $method === 'PUT'    && preg_match('#^/api/defects/(\d+)$#', $uri, $m) === 1 => $defectC
    trl->updateDefectReport((int) $m[1], $body),
```

```
        $method === 'DELETE' && preg_match('#^/api/defects/(\d+)$#', $uri, $m) === 1 => $defectC
    trl->deleteDefect((int) $m[1]),

        // Analyzer Triage, IEEE 1992 Classification & Cost Resolution (Level 3+)
        $method === 'GET'    && $uri === '/api/analyzer/queue'   => $defectCtrl->getAnalyzerQueu
    e($_GET),
        $method === 'PUT'    && preg_match('#^/api/defects/(\d+)/analyze$#', $uri, $m) === 1 =>
    $defectCtrl->analyzeDefect((int) $m[1], $body),
        $method === 'PUT'    && preg_match('#^/api/defects/(\d+)/resolve$#', $uri, $m) === 1 =>
    $defectCtrl->resolveDefect((int) $m[1], $body),

        default => (static function (): array {
            http_response_code(404);
            return ['error' => 'Endpoint not found.'];
        })(),
    };

    echo json_encode($response, JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE);
} catch (\Throwable $e) {
    http_response_code(500);
    echo json_encode([
        'error'   => 'Internal Server Error',
        'details' => $e->getMessage(),
    ]);
}
```

```
================================================================================
SECTION 3.1 TypeScript Enums & Domain Types (modern/frontend/src/types/defman.ts)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\types\defman.ts
================================================================================
```

```
/**
 * IEEE (1992) Standard Classification for Software Anomalies
 * Refactored from: defecttype.txt & faulttypes.html
 */
export enum IeeeFaultType {
  LOGIC                  = 'Logic',
  COMPUTATIONAL          = 'Computational',
  INTERFACE              = 'Interface',
  DATA_HANDLING          = 'Data handling',
  DATA                   = 'Data',
  DOCUMENTATION          = 'Documentation',
  DOCUMENT_QUALITY       = 'Document quality',
  ENHANCEMENT            = 'Enhancement',
  CAUSED_BY_PREVIOUS_FIX = 'Caused by a prev. Fix',
  OTHER                  = 'Other',
}

export const IEEE_FAULT_HEX_COLORS: Record<IeeeFaultType, number> = {
  [IeeeFaultType.LOGIC]:                  0xef4444, // Red-500
  [IeeeFaultType.COMPUTATIONAL]:          0xf97316, // Orange-500
  [IeeeFaultType.INTERFACE]:              0xec4899, // Pink-500
  [IeeeFaultType.DATA_HANDLING]:          0x8b5cf6, // Violet-500
  [IeeeFaultType.DATA]:                   0x6366f1, // Indigo-500
  [IeeeFaultType.DOCUMENTATION]:          0x06b6d4, // Cyan-500
  [IeeeFaultType.DOCUMENT_QUALITY]:       0x14b8a6, // Teal-500
  [IeeeFaultType.ENHANCEMENT]:            0x10b981, // Emerald-500
  [IeeeFaultType.CAUSED_BY_PREVIOUS_FIX]: 0xf59e0b, // Amber-500
  [IeeeFaultType.OTHER]:                  0x64748b, // Slate-500
};

/**
 * Full IEEE (1992) Taxonomy from faulttypes.html
 */
```

```
export const IEEE_1992_TAXONOMY: Record<IeeeFaultType, readonly string[]> = {
  [IeeeFaultType.LOGIC]: [
    'Forgotten cases or steps',
    'Duplicate logic',
    'Extreme conditions neglected',
    'Unnecessary function',
    'Misinterpretation',
    'Missing condition test',
    'Checking wrong variable',
    'Iterating loop incorrectly',
  ],
  [IeeeFaultType.COMPUTATIONAL]: [
    'Equation insufficient or incorrect: Missing computation',
    'Equation insufficient or incorrect: Operand in equation incorrect',
    'Equation insufficient or incorrect: Operator in equation incorrect',
    'Equation insufficient or incorrect: Parentheses used incorrectly',
    'Precision loss: Rounding or truncation fault',
    'Precision loss: Mixed modes',
    'Sign convention fault',
  ],
  [IeeeFaultType.INTERFACE]: [
    'Interrupts handled incorrectly',
    'I/O timing incorrect: Timing fault causes data loss',
    'Subroutine/module mismatch: Wrong subroutine called',
    'Subroutine/module mismatch: Incorrectly-located subroutine call',
    'Subroutine/module mismatch: Non-existent subroutine called',
    'Subroutine/module mismatch: Inconsistent subroutine arguments',
  ],
  [IeeeFaultType.DATA_HANDLING]: [
    'Initialized data incorrectly',
    'Accessed or stored data incorrectly: Flag or index set incorrectly',
    'Accessed or stored data incorrectly: Packed/unpacked data incorrectly',
    'Accessed or stored data incorrectly: Referenced wrong data variable',
    'Accessed or stored data incorrectly: Data referenced out of bounds',
    'Scaling or units of data incorrect',
    'Dimensioned data incorrectly: Variable type incorrect',
    'Dimensioned data incorrectly: Subscripted variable incorrectly',
    'Scope of data incorrect',
  ],
  [IeeeFaultType.DATA]: [
    'Sensor data incorrect or missing',
    'Operator data incorrect or missing',
    'Embedded data in tables incorrect or missing',
    'External data incorrect or missing',
    'Output data incorrect or missing',
    'Input data incorrect or missing',
  ],
  [IeeeFaultType.DOCUMENTATION]: [
    'Ambiguous statement',
    'Incomplete item',
    'Incorrect item',
    'Missing item',
    'Conflicting items',
    'Redundant items',
    'Confusing item',
    'Illogical item',
    'Non-verifiable item',
  ],
  [IeeeFaultType.DOCUMENT_QUALITY]: [
    'Applicable standards not met',
    'Not traceable',
    'Not current',
    'Inconsistencies',
    'Incomplete',
    'No identification',
  ],
```

```
  [IeeeFaultType.ENHANCEMENT]: [
    'Change in program requirements: Add new capability',
    'Change in program requirements: Remove unnecessary capability',
    'Change in program requirements: Update current capability',
    'Improve comments',
    'Improve code efficiency',
    'Implement editorial changes',
    'Improve usability',
    'Software fix of a hardware problem',
    'Other enhancement',
  ],
  [IeeeFaultType.CAUSED_BY_PREVIOUS_FIX]: [
    'Failure caused by a previous fix',
  ],
  [IeeeFaultType.OTHER]: [
    'Other problem',
  ],
};

export enum UserLevel {
  APPLYING = 0,
  PUBLIC   = 1,
  CUSTOMER = 2,
  ANALYZER = 3,
  ADMIN    = 4,
}

export enum DefectSeverity {
  MINOR    = 'minor',
  MAJOR    = 'major',
  CRITICAL = 'critical',
}

export enum DefectPriority {
  MOST_URGENT = 'm_urgent',
  AVERAGE     = 'average',
  LESS_URGENT = 'l_urgent',
}

export enum DefectStatus {
  SUBMITTED = 'submitted',
  ASSIGNED  = 'assigned',
  POSTPONED = 'postponed',
  OPENED    = 'opened',
  FIXED     = 'fixed',
  RESOLVED  = 'resolved',
  CLOSED    = 'closed',
}

export enum PersonType {
  CUSTOMER  = 'Customer',
  DEVELOPER = 'Developer',
  TESTER    = 'Tester',
}

export enum Uniqueness {
  NEW_DEFECT        = 'New defect',
  ALREADY_SUBMITTED = 'Already submitted',
  CRITICAL          = 'Critical',
}

export enum OriginPhase {
  REQUIREMENTS   = 'Requirements',
  DESIGN         = 'Design',
  IMPLEMENTATION = 'Implementation',
  TESTING        = 'Testing',
```

```
  DEPLOYMENT     = 'Deployment',
  MAINTENANCE    = 'Maintenance',
}

export interface AuthUser {
  id: number;
  login: string;
  email: string;
  userlevel: UserLevel;
  role?: string;
  name?: string | null;
  company?: string | null;
  phone?: string | null;
  activate?: number;
  logged?: number;
}

export interface Product {
  id: number;
  name: string;
  report_count?: number;
}

export interface DefectAnalysis {
  id: number;
  devdesc: string | null;
  classification: IeeeFaultType | string;
  dateoriginated: string | null;
  phase: string | null;
  phases?: string[];
  actoriginated: string | null;
  deliverable: string | null;
  whattodo: string | null;
  submitted: string | null;
  assigned: string | null;
  uniqueness: Uniqueness | string;
  priority: 'most urgent' | 'average' | 'less urgent';
  severity: DefectSeverity;
  workaround: string | null;
  proposedfix: string | null;
  status: DefectStatus;
  fixdesc: string | null;
  fixstarted: string | null;
  fixcomplete: string | null;
  fiximpact: string | null;
  fixeffort: number;
  fixcost: number;
  resodesc: string | null;
  resocomplete: string | null;
  resoclosed: string | null;
  costofdefect: number;
  actfound: string | null;
}

export interface DefectReport {
  ID: number;
  Product: string;
  product_id: number | null;
  Environment: string | null;
  CustomerID: number | null;
  Who: string;
  Contactemail: string;
  Whenreported: string;
  Title: string;
  Description: string;
  Discoveredby: string | null;
```

```
  Persontype: PersonType;
  Occured: string;
  Location: string | null;
  Severity: DefectSeverity;
  Priority: DefectPriority;
  PriorityLabel?: string;
  Submitted: string;
  days_since_submitted?: number;
  Addinfo: string | null;
  is_deleted_user?: number;
  classification?: string | null;
  analysis_status?: DefectStatus;
  fixeffort?: number | null;
  fixcost?: number | null;
  costofdefect?: number | null;
  analysis?: DefectAnalysis | null;
}

export interface AnalyzerQueueItem extends DefectAnalysis {
  product: string;
  title: string;
  report_description: string;
  environment: string | null;
  occured: string;
  contact_email: string;
  who: string;
  classification_color: string;
  days_open: number;
  submitted_to_assigned_days: number | null;
  submitted_to_fixed_days: number | null;
  submitted_to_resolved_days: number | null;
  assigned_to_fixed_days: number | null;
  assigned_to_resolved_days: number | null;
}
```

==========

SECTION 3.2 Typed REST API Client (modern/frontend/src/api/client.ts)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\api\client.ts

==========

```
import type {
  AnalyzerQueueItem,
  AuthUser,
  DefectReport,
  Product,
} from '../types/defman';

const API_BASE = import.meta.env?.VITE_API_BASE_URL ?? 'http://localhost:8080';

async function request<T>(path: string, init: RequestInit = {}): Promise<T> {
  const token = localStorage.getItem('defman_token');
  const headers: Record<string, string> = {
    'Content-Type': 'application/json',
    ...(init.headers as Record<string, string> | undefined),
  };
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const res = await fetch(`${API_BASE}${path}`, {
    ...init,
    headers,
  });
```

```
  const data = await res.json();
  if (!res.ok) {
    const message =
      data.error ||
      (Array.isArray(data.errors) ? data.errors.join(' - ') : 'Request failed');
    throw new Error(message);
  }
  return data as T;
}

export const defmanApi = {
  // Auth
  login: (username: string, password: string) =>
    request<{ token: string; user: AuthUser }>('/api/auth/login', {
      method: 'POST',
      body: JSON.stringify({ username, password }),
    }),
  logout: () =>
    request<{ message: string }>('/api/auth/logout', { method: 'POST' }),
  applyForAccount: (payload: {
    username: string;
    password: string;
    verify_password: string;
    email: string;
    name?: string;
    company?: string;
    phone?: string;
  }) =>
    request<{ id: number; message: string }>('/api/auth/apply', {
      method: 'POST',
      body: JSON.stringify(payload),
    }),
  lostPassword: (email: string) =>
    request<{ message: string; reset_token?: string }>(
      '/api/auth/lost-password',
      {
        method: 'POST',
        body: JSON.stringify({ email }),
      }
    ),

  // Products
  getProducts: () =>
    request<{ products: Product[] }>('/api/products'),
  createProduct: (name: string) =>
    request<{ id: number; name: string; message: string }>(
      '/api/admin/products',
      {
        method: 'POST',
        body: JSON.stringify({ name }),
      }
    ),
  deleteProduct: (id: number) =>
    request<{ message: string }>(`/api/admin/products/${id}`, {
      method: 'DELETE',
    }),

  // Defects (Public & Customer)
  submitPublicDefect: (payload: Record<string, unknown>) =>
    request<{ id: number; message: string }>('/api/defects/public', {
      method: 'POST',
      body: JSON.stringify(payload),
    }),
  submitCustomerDefect: (payload: Record<string, unknown>) =>
    request<{ id: number; message: string }>('/api/defects', {
      method: 'POST',
```

```
    body: JSON.stringify(payload),
  }),
getDefects: (params: Record<string, string | number | undefined> = {}) => {
  const qs = new URLSearchParams();
  Object.entries(params).forEach(([k, v]) => {
    if (v !== undefined && v !== '') qs.set(k, String(v));
  });
  return request<{ count: number; defects: DefectReport[] }>(
    `/api/defects?${qs.toString()}`
  );
},
getDefectById: (id: number) =>
  request<DefectReport>(`/api/defects/${id}`),
updateDefectReport: (id: number, payload: Record<string, unknown>) =>
  request<{ message: string }>(`/api/defects/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload),
  }),
deleteDefect: (id: number) =>
  request<{ message: string }>(`/api/defects/${id}`, {
    method: 'DELETE',
  }),

// Analyzer Queue, Triage & Resolution (Level 3)
getAnalyzerQueue: (sort = 'all') =>
  request<{ count: number; queue: AnalyzerQueueItem[] }>(
    `/api/analyzer/queue?sort=${encodeURIComponent(sort)}`
  ),
analyzeDefect: (id: number, payload: Record<string, unknown>) =>
  request<{ message: string }>(`/api/defects/${id}/analyze`, {
    method: 'PUT',
    body: JSON.stringify(payload),
  }),
resolveDefect: (id: number, payload: Record<string, unknown>) =>
  request<{ id: number; status: string; message: string }>(
    `/api/defects/${id}/resolve`,
    {
      method: 'PUT',
      body: JSON.stringify(payload),
    }
  ),

// Admin Users & Profile
getAdminUsers: () =>
  request<{
    users: AuthUser[];
    applicants: AuthUser[];
    generated_password_suggestion: string;
  }>('/api/admin/users'),
createAdminUser: (payload: {
  username: string;
  password: string;
  email: string;
  userlevel: number;
}) =>
  request<{ id: number; message: string }>('/api/admin/users', {
    method: 'POST',
    body: JSON.stringify(payload),
  }),
patchAdminUser: (
  id: number,
  payload: { action: 'approve' | 'role'; userlevel: number }
) =>
  request<{ message: string }>(`/api/admin/users/${id}`, {
    method: 'PATCH',
    body: JSON.stringify(payload),
```

```
    }),
  deleteAdminUser: (id: number) =>
    request<{ message: string }>(`/api/admin/users/${id}`, {
      method: 'DELETE',
    }),
  getProfile: () => request<AuthUser>('/api/users/me'),
  updateProfile: (payload: Record<string, unknown>) =>
    request<{ message: string }>('/api/users/me', {
      method: 'PUT',
      body: JSON.stringify(payload),
    }),
};
```

```
================================================================================
SECTION 3.3 <PublicReportForm /> (modern/frontend/src/components/PublicReportForm.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\PublicReportForm.t
    sx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectPriority,
  DefectSeverity,
  PersonType,
  type Product,
} from '../types/defman';

/**
 * <PublicReportForm />
 * Refactored from: form_raportpub.php (+ helpreportpub.html inline modal)
 * Endpoint: POST /api/defects/public
 */
export const PublicReportForm: React.FC<{ onSuccess?: () => void }> = ({
  onSuccess,
}) => {
  const [products, setProducts] = useState<Product[]>([]);
  const [showHelp, setShowHelp] = useState(false);
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [form, setForm] = useState({
    who: '',
    contact_email: '',
    product: '',
    title: '',
    description: '',
    discovered_by: '',
    persontype: PersonType.CUSTOMER,
    location: '',
    occured: new Date().toISOString().slice(0, 10),
    environment: '',
    severity: DefectSeverity.MINOR,
    priority: DefectPriority.MOST_URGENT,
    addinfo: '',
  });

  useEffect(() => {
    defmanApi.getProducts().then((res) => {
      setProducts(res.products);
      if (res.products.length > 0) {
        setForm((prev) => ({ ...prev, product: res.products[0].name }));
      }
    });
```

```
  }, []);

  const clearForm = () => {
    setForm((prev) => ({
      ...prev,
      who: '',
      contact_email: '',
      title: '',
      description: '',
      discovered_by: '',
      location: '',
      occured: new Date().toISOString().slice(0, 10),
      environment: '',
      addinfo: '',
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const res = await defmanApi.submitPublicDefect(form);
      setStatusMessage(res.message);
      clearForm();
      onSuccess?.();
    } catch (err) {
      setErrorMessage(err instanceof Error ? err.message : 'Submission failed');
    }
  };

  return (
    <div className="bg-slate-900 border border-slate-700 rounded-xl p-6 max-w-2xl mx-auto text-s
    late-100 shadow-lg">
      <div className="flex items-center justify-between border-b border-slate-700 pb-3 mb-4">
        <div>
          <h2 className="text-lg font-bold text-amber-400">
            New Public Defect Report (Level 1)
          </h2>
          <p className="text-xs text-slate-400">
            Fields marked with (<span className="text-red-400">*</span>) are required.
          </p>
        </div>
        <button
          type="button"
          onClick={() => setShowHelp((s) => !s)}
          className="text-xs px-3 py-1 rounded bg-slate-800 hover:bg-slate-700 text-amber-300 bo
    rder border-slate-600"
        >
          {showHelp ? 'Hide Help' : 'Help (helpreportpub.html)'}
        </button>
      </div>

      {showHelp && (
        <div className="mb-4 p-4 rounded-lg bg-slate-950 border border-amber-500/40 text-xs spac
    e-y-1">
          <div className="font-semibold text-amber-300 mb-1">
            How to fill out the public defect report form (Example):
          </div>
          <div><strong>Who reported:</strong> Jason</div>
          <div><strong>Contact Email:</strong> jason@email.com</div>
          <div><strong>Title:</strong> PopUp Menu Crash</div>
          <div><strong>Description:</strong> When adding entries into the popup menu in a certai
    n way, the application will close without any error messages.</div>
          <div><strong>Location:</strong> settings form | <strong>Occurred:</strong> 2004-04-14
    | <strong>Environment:</strong> AMD Athlon XP 2000, WinXP</div>
```

```
        </div>
      )}

      {errorMessage && (
        <div className="mb-4 p-3 rounded bg-red-950/80 border border-red-600 text-amber-300 text
    -sm">
          {errorMessage}
        </div>
      )}
      {statusMessage && (
        <div className="mb-4 p-3 rounded bg-emerald-950/80 border border-emerald-600 text-emeral
    d-300 text-sm">
          {statusMessage}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-3 text-sm">
        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Who reported the problem <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              maxLength={6}
              required
              value={form.who}
              onChange={(e) => setForm({ ...form, who: e.target.value })}
              placeholder="Max 6 chars (e.g. Jason)"
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Contact Email <span className="text-red-400">*</span>
            </span>
            <input
              type="email"
              required
              value={form.contact_email}
              onChange={(e) => setForm({ ...form, contact_email: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Product and release <span className="text-red-400">*</span>
            </span>
            <select
              value={form.product}
              onChange={(e) => setForm({ ...form, product: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              {products.map((p) => (
                <option key={p.id} value={p.name}>
                  {p.name}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Date Problem occurred <span className="text-red-400">*</span>
```

```
            </span>
            <input
              type="date"
              required
              value={form.occured}
              onChange={(e) => setForm({ ...form, occured: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
        </div>

        <label className="block">
          <span className="text-slate-300">
            Title <span className="text-red-400">*</span>
          </span>
          <input
            type="text"
            required
            value={form.title}
            onChange={(e) => setForm({ ...form, title: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <label className="block">
          <span className="text-slate-300">
            Description of the problem <span className="text-red-400">*</span>
          </span>
          <textarea
            rows={4}
            required
            value={form.description}
            onChange={(e) => setForm({ ...form, description: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <div className="grid grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-300">Who discovered defect</span>
            <input
              type="text"
              value={form.discovered_by}
              onChange={(e) => setForm({ ...form, discovered_by: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Person type <span className="text-red-400">*</span>
            </span>
            <select
              value={form.persontype}
              onChange={(e) =>
                setForm({ ...form, persontype: e.target.value as PersonType })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              {Object.values(PersonType).map((pt) => (
                <option key={pt} value={pt}>
                  {pt}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
```

```
            <span className="text-slate-300">Location found</span>
            <input
              type="text"
              value={form.location}
              onChange={(e) => setForm({ ...form, location: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Severity <span className="text-red-400">*</span>
            </span>
            <select
              value={form.severity}
              onChange={(e) =>
                setForm({ ...form, severity: e.target.value as DefectSeverity })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={DefectSeverity.MINOR}>minor</option>
              <option value={DefectSeverity.MAJOR}>major</option>
              <option value={DefectSeverity.CRITICAL}>critical</option>
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Priority <span className="text-red-400">*</span>
            </span>
            <select
              value={form.priority}
              onChange={(e) =>
                setForm({ ...form, priority: e.target.value as DefectPriority })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={DefectPriority.MOST_URGENT}>most urgent</option>
              <option value={DefectPriority.AVERAGE}>average</option>
              <option value={DefectPriority.LESS_URGENT}>less urgent</option>
            </select>
          </label>
        </div>

        <label className="block">
          <span className="text-slate-300">Technical environment</span>
          <textarea
            rows={2}
            value={form.environment}
            onChange={(e) => setForm({ ...form, environment: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <label className="block">
          <span className="text-slate-300">Additional information</span>
          <textarea
            rows={2}
            value={form.addinfo}
            onChange={(e) => setForm({ ...form, addinfo: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <div className="flex items-center gap-3 pt-2">
```

```
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
    ibold"
          >
            Submit Report
          </button>
          <button
            type="button"
            onClick={clearForm}
            className="px-4 py-2 rounded bg-slate-800 hover:bg-slate-700 text-slate-200 border b
    order-slate-600"
          >
            Clear
          </button>
        </div>
      </form>
    </div>
  );
};
```

```
================================================================================
SECTION 3.4 <CustomerReportForm /> (modern/frontend/src/components/CustomerReportForm.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\CustomerReportForm
    .tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  type AuthUser,
  DefectPriority,
  DefectSeverity,
  PersonType,
  type Product,
} from '../types/defman';

/**
 * <CustomerReportForm />
 * Refactored from: form_raport.php (+ helpreport.html)
 * Endpoint: POST /api/defects
 */
export const CustomerReportForm: React.FC<{
  currentUser: AuthUser;
  onSuccess?: () => void;
}> = ({ currentUser, onSuccess }) => {
  const [products, setProducts] = useState<Product[]>([]);
  const [showHelp, setShowHelp] = useState(false);
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [form, setForm] = useState({
    product: '',
    title: '',
    description: '',
    discovered_by: currentUser.name || currentUser.login,
    persontype: PersonType.CUSTOMER,
    location: '',
    occured: new Date().toISOString().slice(0, 10),
    environment: '',
    severity: DefectSeverity.MINOR,
    priority: DefectPriority.MOST_URGENT,
    addinfo: '',
  });
```

```
  useEffect(() => {
    defmanApi.getProducts().then((res) => {
      setProducts(res.products);
      if (res.products.length > 0) {
        setForm((prev) => ({ ...prev, product: res.products[0].name }));
      }
    });
  }, []);

  const clearForm = () => {
    setForm((prev) => ({
      ...prev,
      title: '',
      description: '',
      discovered_by: '',
      location: '',
      occured: new Date().toISOString().slice(0, 10),
      environment: '',
      addinfo: '',
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const res = await defmanApi.submitCustomerDefect(form);
      setStatusMessage(res.message);
      clearForm();
      onSuccess?.();
    } catch (err) {
      setErrorMessage(err instanceof Error ? err.message : 'Submission failed');
    }
  };

  return (
    <div className="bg-slate-900 border border-slate-700 rounded-xl p-6 max-w-2xl mx-auto text-s
    late-100 shadow-lg">
      <div className="flex items-center justify-between border-b border-slate-700 pb-3 mb-4">
        <div>
          <h2 className="text-lg font-bold text-amber-400">
            New Defect Report (Customer ID #{currentUser.id})
          </h2>
          <p className="text-xs text-slate-400">
            Reporter: <span className="text-slate-200">{currentUser.login}</span> ({currentUser.
    email})
          </p>
        </div>
        <button
          type="button"
          onClick={() => setShowHelp((s) => !s)}
          className="text-xs px-3 py-1 rounded bg-slate-800 hover:bg-slate-700 text-amber-300 bo
    rder border-slate-600"
        >
          {showHelp ? 'Hide Help' : 'Help (helpreport.html)'}
        </button>
      </div>

      {showHelp && (
        <div className="mb-4 p-4 rounded-lg bg-slate-950 border border-amber-500/40 text-xs spac
    e-y-1">
          <div className="font-semibold text-amber-300 mb-1">
            How to fill out the defect report form (Example):
          </div>
```

```
        <div><strong>Title:</strong> PopUp Menu Crash</div>
        <div><strong>Description:</strong> When adding entries into the popup menu in a certai
    n way, the application will close without any error messages.</div>
        <div><strong>Who discovered:</strong> Sabrina | <strong>Location:</strong> settings fo
    rm | <strong>Occurred:</strong> 2004-04-14</div>
        <div><strong>Technical environment:</strong> AMD Athlon XP 2000, WinXP</div>
      </div>
    )}

    {errorMessage && (
      <div className="mb-4 p-3 rounded bg-red-950/80 border border-red-600 text-amber-300 text
    -sm">
        {errorMessage}
      </div>
    )}
    {statusMessage && (
      <div className="mb-4 p-3 rounded bg-emerald-950/80 border border-emerald-600 text-emeral
    d-300 text-sm">
        {statusMessage}
      </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-3 text-sm">
      <div className="grid grid-cols-2 gap-3">
        <label className="block">
          <span className="text-slate-300">
            Product and release <span className="text-red-400">*</span>
          </span>
          <select
            value={form.product}
            onChange={(e) => setForm({ ...form, product: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          >
            {products.map((p) => (
              <option key={p.id} value={p.name}>
                {p.name}
              </option>
            ))}
          </select>
        </label>
        <label className="block">
          <span className="text-slate-300">
            Date Problem occurred <span className="text-red-400">*</span>
          </span>
          <input
            type="date"
            required
            value={form.occured}
            onChange={(e) => setForm({ ...form, occured: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
      </div>

      <label className="block">
        <span className="text-slate-300">
          Title <span className="text-red-400">*</span>
        </span>
        <input
          type="text"
          required
          value={form.title}
          onChange={(e) => setForm({ ...form, title: e.target.value })}
          className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
        />
      </label>
```

```
        <label className="block">
          <span className="text-slate-300">
            Description of the problem <span className="text-red-400">*</span>
          </span>
          <textarea
            rows={4}
            required
            value={form.description}
            onChange={(e) => setForm({ ...form, description: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <div className="grid grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-300">Who discovered defect</span>
            <input
              type="text"
              value={form.discovered_by}
              onChange={(e) => setForm({ ...form, discovered_by: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Person type <span className="text-red-400">*</span>
            </span>
            <select
              value={form.persontype}
              onChange={(e) =>
                setForm({ ...form, persontype: e.target.value as PersonType })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              {Object.values(PersonType).map((pt) => (
                <option key={pt} value={pt}>
                  {pt}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">Location found</span>
            <input
              type="text"
              value={form.location}
              onChange={(e) => setForm({ ...form, location: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Severity <span className="text-red-400">*</span>
            </span>
            <select
              value={form.severity}
              onChange={(e) =>
                setForm({ ...form, severity: e.target.value as DefectSeverity })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={DefectSeverity.MINOR}>minor</option>
```

```
              <option value={DefectSeverity.MAJOR}>major</option>
              <option value={DefectSeverity.CRITICAL}>critical</option>
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Priority <span className="text-red-400">*</span>
            </span>
            <select
              value={form.priority}
              onChange={(e) =>
                setForm({ ...form, priority: e.target.value as DefectPriority })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={DefectPriority.MOST_URGENT}>most urgent</option>
              <option value={DefectPriority.AVERAGE}>average</option>
              <option value={DefectPriority.LESS_URGENT}>less urgent</option>
            </select>
          </label>
        </div>

        <label className="block">
          <span className="text-slate-300">Technical environment</span>
          <textarea
            rows={2}
            value={form.environment}
            onChange={(e) => setForm({ ...form, environment: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <label className="block">
          <span className="text-slate-300">Additional information</span>
          <textarea
            rows={2}
            value={form.addinfo}
            onChange={(e) => setForm({ ...form, addinfo: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>

        <div className="flex items-center gap-3 pt-2">
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
    ibold"
          >
            Submit Report
          </button>
          <button
            type="button"
            onClick={clearForm}
            className="px-4 py-2 rounded bg-slate-800 hover:bg-slate-700 text-slate-200 border b
    order-slate-600"
          >
            Clear
          </button>
        </div>
      </form>
    </div>
  );
};
```

```
================================================================================
SECTION 3.5 <DefectReportGrid /> (modern/frontend/src/components/DefectReportGrid.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectReportGrid.t
    sx
================================================================================

import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectPriority,
  type DefectReport,
  DefectSeverity,
  DefectStatus,
  type Product,
} from '../types/defman';

/**
 * <DefectReportGrid />
 * Refactored from: form_raportlist.php & form_list.php
 * Endpoint: GET /api/defects with multi-column sorting and filtering by priority, status, sever
    ity, and product
 */
export const DefectReportGrid: React.FC<{
  onSelectDefect: (id: number) => void;
  onAnalyzeDefect?: (id: number) => void;
}> = ({ onSelectDefect, onAnalyzeDefect }) => {
  const [defects, setDefects] = useState<DefectReport[]>([]);
  const [products, setProducts] = useState<Product[]>([]);
  const [fullDetailsMode, setFullDetailsMode] = useState(false);

  const [filters, setFilters] = useState({
    severity: '',
    priority: '',
    status: '',
    product: '',
    sort: 'date',
  });

  const loadData = useCallback(async () => {
    const [defRes, prodRes] = await Promise.all([
      defmanApi.getDefects(filters),
      defmanApi.getProducts(),
    ]);
    setDefects(defRes.defects);
    setProducts(prodRes.products);
  }, [filters]);

  useEffect(() => {
    loadData();
  }, [loadData]);

  const handleDelete = async (id: number) => {
    if (!window.confirm(`Are you SURE that you want to DELETE\n\nDefect ID: ${id}?`)) {
      return;
    }
    await defmanApi.deleteDefect(id);
    loadData();
  };

  const formatDaysLabel = (days?: number) => {
    if (days === undefined || days === null || days <= 0) return 'today';
    if (days === 1) return '1 day';
    return `${days} days`;
  };

  return (
```

```
    <div className="bg-slate-900 border border-slate-700 rounded-xl p-5 text-slate-100 shadow-lg
     space-y-4">
      <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-70
    0 pb-3">
        <div>
          <h2 className="text-lg font-bold text-amber-400">
            All Defect Reports ({defects.length})
          </h2>
          <p className="text-xs text-slate-400">
            Multi-column sorting & filtering by severity, priority, status, and product
          </p>
        </div>
        <button
          type="button"
          onClick={() => setFullDetailsMode((m) => !m)}
          className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700 text-amber-300
    border border-slate-600"
        >
          {fullDetailsMode ? 'Summary Grid (form_raportlist.php)' : 'Full Details View (form_lis
    t.php)'}
        </button>
      </div>

      {/* Filter & Sort Controls */}
      <div className="grid grid-cols-2 md:grid-cols-5 gap-3 text-xs bg-slate-950/60 p-3 rounded-
    lg border border-slate-800">
        <label className="block">
          <span className="text-slate-400">Severity</span>
          <select
            value={filters.severity}
            onChange={(e) => setFilters({ ...filters, severity: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2 py-1.5"
          >
            <option value="">All severities</option>
            <option value={DefectSeverity.CRITICAL}>critical</option>
            <option value={DefectSeverity.MAJOR}>major</option>
            <option value={DefectSeverity.MINOR}>minor</option>
          </select>
        </label>

        <label className="block">
          <span className="text-slate-400">Priority</span>
          <select
            value={filters.priority}
            onChange={(e) => setFilters({ ...filters, priority: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2 py-1.5"
          >
            <option value="">All priorities</option>
            <option value={DefectPriority.MOST_URGENT}>most urgent</option>
            <option value={DefectPriority.AVERAGE}>average</option>
            <option value={DefectPriority.LESS_URGENT}>less urgent</option>
          </select>
        </label>

        <label className="block">
          <span className="text-slate-400">Status</span>
          <select
            value={filters.status}
            onChange={(e) => setFilters({ ...filters, status: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2 py-1.5"
          >
            <option value="">All statuses</option>
            {Object.values(DefectStatus).map((st) => (
              <option key={st} value={st}>
                {st}
              </option>
```

```
          ))}
        </select>
      </label>

      <label className="block">
        <span className="text-slate-400">Show Only Product</span>
        <select
          value={filters.product}
          onChange={(e) => setFilters({ ...filters, product: e.target.value })}
          className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2 py-1.5"
        >
          <option value="">All products</option>
          {products.map((p) => (
            <option key={p.id} value={p.name}>
              {p.name} ({p.report_count ?? 0})
            </option>
          ))}
        </select>
      </label>

      <label className="block">
        <span className="text-slate-400">Sort By</span>
        <select
          value={filters.sort}
          onChange={(e) => setFilters({ ...filters, sort: e.target.value })}
          className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2 py-1.5"
        >
          <option value="date">Submitted Date</option>
          <option value="severity">Severity</option>
          <option value="priority">Priority</option>
          <option value="product">Product</option>
          <option value="contact">Contact</option>
        </select>
      </label>
    </div>

    {/* Table / Full Details */}
    {!fullDetailsMode ? (
      <div className="overflow-x-auto">
        <table className="w-full text-left text-xs border-collapse">
          <thead>
            <tr className="bg-slate-800 text-slate-300 border-b border-slate-700">
              <th className="p-2.5">ID</th>
              <th className="p-2.5">Product</th>
              <th className="p-2.5">Contact</th>
              <th className="p-2.5">Title</th>
              <th className="p-2.5">Description</th>
              <th className="p-2.5">Occurred</th>
              <th className="p-2.5">Environment</th>
              <th className="p-2.5">Severity</th>
              <th className="p-2.5">Priority</th>
              <th className="p-2.5">Submitted</th>
              <th className="p-2.5 text-right">Actions</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-800">
            {defects.map((d) => (
              <tr key={d.ID} className="hover:bg-slate-800/60">
                <td className="p-2.5 font-mono text-amber-400">#{d.ID}</td>
                <td className="p-2.5 font-medium">{d.Product}</td>
                <td className="p-2.5">
                  {d.Contactemail}
                  {d.is_deleted_user === 1 && (
                    <span className="ml-1 text-[10px] text-red-400 font-semibold">
                      [DELETED USER]
                    </span>
```

```
                  )}
                </td>
                <td className="p-2.5 font-semibold text-slate-100">{d.Title}</td>
                <td className="p-2.5 max-w-xs truncate text-slate-300">
                  {d.Description}
                </td>
                <td className="p-2.5 whitespace-nowrap">{d.Occured}</td>
                <td className="p-2.5 max-w-[160px] truncate text-slate-400">
                  {d.Environment}
                </td>
                <td className="p-2.5">
                  <span
                    className={`px-2 py-0.5 rounded font-semibold ${
                      d.Severity === DefectSeverity.CRITICAL
                        ? 'bg-red-900/60 text-red-300 border border-red-700'
                        : d.Severity === DefectSeverity.MAJOR
                        ? 'bg-orange-900/60 text-orange-300 border border-orange-700'
                        : 'bg-slate-800 text-slate-300'
                    }`}
                  >
                    {d.Severity}
                  </span>
                </td>
                <td className="p-2.5">{d.PriorityLabel || d.Priority}</td>
                <td className="p-2.5 whitespace-nowrap">
                  {d.Submitted} ({formatDaysLabel(d.days_since_submitted)})
                </td>
                <td className="p-2.5 text-right whitespace-nowrap space-x-1.5">
                  <button
                    type="button"
                    onClick={() => onSelectDefect(d.ID)}
                    className="px-2 py-1 rounded bg-slate-700 hover:bg-slate-600 text-slate-10
0"
                  >
                    Report
                  </button>
                  {onAnalyzeDefect && (
                    <button
                      type="button"
                      onClick={() => onAnalyzeDefect(d.ID)}
                      className="px-2 py-1 rounded bg-amber-500 hover:bg-amber-400 text-slate-
950 font-semibold"
                    >
                      Analyze
                    </button>
                  )}
                  <button
                    type="button"
                    onClick={() => handleDelete(d.ID)}
                    className="px-2 py-1 rounded bg-red-900/70 hover:bg-red-800 text-red-200"
                  >
                    Delete
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    ) : (
      /* Full Details Mode (form_list.php) */
      <div className="space-y-4">
        {defects.map((d) => (
          <div
            key={d.ID}
            className="rounded-lg bg-slate-950 border border-slate-800 p-4 grid grid-cols-1 md
```

```
    :grid-cols-2 gap-2 text-xs"
            >
              <div><strong>ID:</strong> #{d.ID}</div>
              <div><strong>Product and release:</strong> {d.Product}</div>
              <div><strong>Who reported:</strong> {d.Who}</div>
              <div><strong>Contact email:</strong> {d.Contactemail}</div>
              <div><strong>When reported:</strong> {d.Whenreported}</div>
              <div><strong>Date occurred:</strong> {d.Occured}</div>
              <div><strong>Who discovered:</strong> {d.Discoveredby || '--'}</div>
              <div><strong>Type of person:</strong> {d.Persontype}</div>
              <div><strong>Location found:</strong> {d.Location || '--'}</div>
              <div><strong>Technical environment:</strong> {d.Environment || '--'}</div>
              <div><strong>Severity:</strong> {d.Severity}</div>
              <div><strong>Priority:</strong> {d.PriorityLabel || d.Priority}</div>
              <div className="md:col-span-2"><strong>Title:</strong> {d.Title}</div>
              <div className="md:col-span-2"><strong>Description:</strong> {d.Description}</div>
              <div className="md:col-span-2"><strong>Additional Info:</strong> {d.Addinfo || '--'
    }</div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
};
```

```
================================================================================
SECTION 3.6 <DefectDetailModal /> (modern/frontend/src/components/DefectDetailModal.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectDetailModal.
    tsx
================================================================================
```

```
import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectPriority,
  type DefectReport,
  DefectSeverity,
  PersonType,
  type Product,
} from '../types/defman';

/**
 * <DefectDetailModal />
 * Refactored from: form_defrap.php
 * Endpoint: GET /api/defects/{id} & PUT /api/defects/{id}
 */
export const DefectDetailModal: React.FC<{
  defectId: number;
  canEdit: boolean;
  onClose: () => void;
  onOpenAnalyze?: (id: number) => void;
  onUpdated?: () => void;
}> = ({ defectId, canEdit, onClose, onOpenAnalyze, onUpdated }) => {
  const [defect, setDefect] = useState<DefectReport | null>(null);
  const [products, setProducts] = useState<Product[]>([]);
  const [message, setMessage] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    Promise.all([
      defmanApi.getDefectById(defectId),
      defmanApi.getProducts(),
    ]).then(([defRes, prodRes]) => {
```

```
      setDefect(defRes);
      setProducts(prodRes.products);
    });
  }, [defectId]);

  if (!defect) {
    return null;
  }

  const handleSave = async (e: React.FormEvent) => {
    e.preventDefault();
    setMessage(null);
    setError(null);
    try {
      const res = await defmanApi.updateDefectReport(defect.ID, {
        product: defect.Product,
        environment: defect.Environment,
        title: defect.Title,
        description: defect.Description,
        discovered_by: defect.Discoveredby,
        persontype: defect.Persontype,
        occured: defect.Occured,
        location: defect.Location,
        severity: defect.Severity,
        priority: defect.Priority,
        submitted: defect.Submitted,
        addinfo: defect.Addinfo,
      });
      setMessage(res.message);
      onUpdated?.();
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Update failed');
    }
  };

  return (
    <div className="fixed inset-0 z-50 bg-black/75 flex items-center justify-center p-4">
      <div className="bg-slate-900 border border-slate-700 rounded-xl max-w-2xl w-full p-6 text-
  slate-100 shadow-2xl max-h-[90vh] overflow-y-auto">
        <div className="flex items-center justify-between border-b border-slate-700 pb-3 mb-4">
          <div>
            <h3 className="text-lg font-bold text-amber-400">
              Defect Report -- Defect ID #{defect.ID}
            </h3>
            <p className="text-xs text-slate-400">
              Reported by {defect.Who} ({defect.Contactemail}) at {defect.Whenreported}
            </p>
          </div>
          <div className="flex items-center gap-2">
            {onOpenAnalyze && (
              <button
                type="button"
                onClick={() => onOpenAnalyze(defect.ID)}
                className="text-xs px-3 py-1.5 rounded bg-amber-500 hover:bg-amber-400 text-slat
  e-950 font-semibold"
              >
                Open Analyzer Workbench
              </button>
            )}
            <button
              type="button"
              onClick={onClose}
              className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700 text-slate-
  300"
            >
              Close
```

```
          </button>
        </div>
      </div>

      {error && (
        <div className="mb-3 p-2.5 rounded bg-red-950 border border-red-600 text-amber-300 tex
  t-xs">
          {error}
        </div>
      )}
      {message && (
        <div className="mb-3 p-2.5 rounded bg-emerald-950 border border-emerald-600 text-emera
  ld-300 text-xs">
          {message}
        </div>
      )}

      <form onSubmit={handleSave} className="space-y-3 text-xs">
        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-400">Product and release *</span>
            <select
              disabled={!canEdit}
              value={defect.Product}
              onChange={(e) => setDefect({ ...defect, Product: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
  5"
            >
              {products.map((p) => (
                <option key={p.id} value={p.name}>
                  {p.name}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-400">Title *</span>
            <input
              type="text"
              disabled={!canEdit}
              value={defect.Title}
              onChange={(e) => setDefect({ ...defect, Title: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
  5"
            />
          </label>
        </div>

        <label className="block">
          <span className="text-slate-400">Description of the problem *</span>
          <textarea
            rows={3}
            disabled={!canEdit}
            value={defect.Description}
            onChange={(e) => setDefect({ ...defect, Description: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.5"
          />
        </label>

        <div className="grid grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-400">Who discovered</span>
            <input
              type="text"
              disabled={!canEdit}
              value={defect.Discoveredby || ''}
```

```
              onChange={(e) =>
                setDefect({ ...defect, Discoveredby: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            />
          </label>
          <label className="block">
            <span className="text-slate-400">Person type *</span>
            <select
              disabled={!canEdit}
              value={defect.Persontype}
              onChange={(e) =>
                setDefect({ ...defect, Persontype: e.target.value as PersonType })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            >
              {Object.values(PersonType).map((pt) => (
                <option key={pt} value={pt}>
                  {pt}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-400">Location found</span>
            <input
              type="text"
              disabled={!canEdit}
              value={defect.Location || ''}
              onChange={(e) => setDefect({ ...defect, Location: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            />
          </label>
        </div>

        <div className="grid grid-cols-4 gap-3">
          <label className="block">
            <span className="text-slate-400">Date Occurred *</span>
            <input
              type="date"
              disabled={!canEdit}
              value={defect.Occured}
              onChange={(e) => setDefect({ ...defect, Occured: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            />
          </label>
          <label className="block">
            <span className="text-slate-400">Submitted *</span>
            <input
              type="date"
              disabled={!canEdit}
              value={defect.Submitted}
              onChange={(e) => setDefect({ ...defect, Submitted: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            />
          </label>
          <label className="block">
            <span className="text-slate-400">Severity *</span>
            <select
              disabled={!canEdit}
              value={defect.Severity}
```

```
              onChange={(e) =>
                setDefect({ ...defect, Severity: e.target.value as DefectSeverity })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            >
              <option value={DefectSeverity.MINOR}>minor</option>
              <option value={DefectSeverity.MAJOR}>major</option>
              <option value={DefectSeverity.CRITICAL}>critical</option>
            </select>
          </label>
          <label className="block">
            <span className="text-slate-400">Priority *</span>
            <select
              disabled={!canEdit}
              value={defect.Priority}
              onChange={(e) =>
                setDefect({ ...defect, Priority: e.target.value as DefectPriority })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.
    5"
            >
              <option value={DefectPriority.MOST_URGENT}>most urgent</option>
              <option value={DefectPriority.AVERAGE}>average</option>
              <option value={DefectPriority.LESS_URGENT}>less urgent</option>
            </select>
          </label>
        </div>

        <label className="block">
          <span className="text-slate-400">Technical environment</span>
          <textarea
            rows={2}
            disabled={!canEdit}
            value={defect.Environment || ''}
            onChange={(e) =>
              setDefect({ ...defect, Environment: e.target.value })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.5"
          />
        </label>

        <label className="block">
          <span className="text-slate-400">Additional information</span>
          <textarea
            rows={2}
            disabled={!canEdit}
            value={defect.Addinfo || ''}
            onChange={(e) => setDefect({ ...defect, Addinfo: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-700 px-2.5 py-1.5"
          />
        </label>

        {canEdit && (
          <div className="pt-2 flex justify-end">
            <button
              type="submit"
              className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font
    -semibold"
            >
              Update Report
            </button>
          </div>
        )}
      </form>
    </div>
```

```
    </div>
  );
};
```

```
================================================================================
```

SECTION 3.7 <AnalyzerQueue /> (modern/frontend/src/components/AnalyzerQueue.tsx)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\AnalyzerQueue.tsx

```
================================================================================

import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import type { AnalyzerQueueItem } from '../types/defman';

/**
 * <AnalyzerQueue />
 * Refactored from: form_alist.php & form_analyzed.php
 * Endpoint: GET /api/analyzer/queue
 */
export const AnalyzerQueue: React.FC<{
  onOpenWorkbench: (defectId: number) => void;
}> = ({ onOpenWorkbench }) => {
  const [queue, setQueue] = useState<AnalyzerQueueItem[]>([]);
  const [sort, setSort] = useState<string>('all');
  const [detailedView, setDetailedView] = useState<boolean>(false);

  const loadQueue = useCallback(async () => {
    const res = await defmanApi.getAnalyzerQueue(sort);
    setQueue(res.queue);
  }, [sort]);

  useEffect(() => {
    loadQueue();
  }, [loadQueue]);

  const formatDeltaDays = (days: number | null) => {
    if (days === null || days === undefined) return 'n/a';
    if (days === 0) return 'the same day';
    if (days === 1) return '1 day';
    return `${days} days`;
  };

  return (
    <div className="bg-slate-900 border border-slate-700 rounded-xl p-5 text-slate-100 shadow-lg
     space-y-4">
      <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-70
    0 pb-3">
        <div>
          <h2 className="text-lg font-bold text-amber-400">
            Analyzer Triage & Cost Queue ({queue.length} Analyzes)
          </h2>
          <p className="text-xs text-slate-400">
            IEEE (1992) fault classification, lead-time deltas, effort hours, and EUR defect cos
    t
          </p>
        </div>
        <div classNameName="flex items-center gap-2">
          <select
            value={sort}
            onChange={(e) => setSort(e.target.value)}
            classNameName="text-xs rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          >
            <option value="all">Sort: All (by Priority)</option>
            <option value="date">Sort: Submitted Date</option>
            <option value="critical">Filter Severity: critical</option>
```

```
          <option value="major">Filter Severity: major</option>
          <option value="minor">Filter Severity: minor</option>
          <option value="most urgent">Filter Priority: most urgent</option>
          <option value="average">Filter Priority: average</option>
          <option value="less urgent">Filter Priority: less urgent</option>
          <option value="submitted">Status: submitted</option>
          <option value="assigned">Status: assigned</option>
          <option value="opened">Status: opened</option>
          <option value="fixed">Status: fixed</option>
          <option value="resolved">Status: resolved</option>
          <option value="closed">Status: closed</option>
        </select>
        <button
          type="button"
          onClick={() => setDetailedView((v) => !v)}
          className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700 text-amber-30
0 border border-slate-600"
        >
          {detailedView ? 'Summary Table (form_analyzed.php)' : 'Full Lead-Time Metrics (form_
alist.php)'}
        </button>
      </div>
    </div>

    {!detailedView ? (
      <div className="overflow-x-auto">
        <table className="w-full text-left text-xs border-collapse">
          <thead>
            <tr className="bg-slate-800 text-slate-300 border-b border-slate-700">
              <th className="p-2.5">ID</th>
              <th className="p-2.5">Product</th>
              <th className="p-2.5">Developer Description</th>
              <th className="p-2.5">IEEE 1992 Classification</th>
              <th className="p-2.5">Status</th>
              <th className="p-2.5">Severity</th>
              <th className="p-2.5">Priority</th>
              <th className="p-2.5">Effort / Cost</th>
              <th className="p-2.5">Sub -> Assg</th>
              <th className="p-2.5">Assg -> Reso</th>
              <th className="p-2.5 text-right">Edit</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-800">
            {queue.map((item) => (
              <tr key={item.id} className="hover:bg-slate-800/60">
                <td className="p-2.5 font-mono text-amber-400">#{item.id}</td>
                <td className="p-2.5 font-medium">{item.product}</td>
                <td className="p-2.5 max-w-xs truncate">
                  {item.devdesc || item.title}
                </td>
                <td className="p-2.5">
                  <span
                    className="px-2 py-0.5 rounded text-slate-950 font-semibold"
                    style={{ backgroundColor: item.classification_color }}
                  >
                    {item.classification}
                  </span>
                </td>
                <td className="p-2.5 uppercase font-mono">{item.status}</td>
                <td className="p-2.5">{item.severity}</td>
                <td className="p-2.5">{item.priority}</td>
                <td className="p-2.5 whitespace-nowrap font-mono">
                  {Number(item.fixeffort).toFixed(1)}h / EUR {Number(item.costofdefect || item.fi
xcost).toFixed(0)}
                </td>
                <td className="p-2.5 whitespace-nowrap">
```

```
                  {formatDeltaDays(item.submitted_to_assigned_days)}
                </td>
                <td className="p-2.5 whitespace-nowrap">
                  {formatDeltaDays(item.assigned_to_resolved_days)}
                </td>
                <td className="p-2.5 text-right">
                  <button
                    type="button"
                    onClick={() => onOpenWorkbench(item.id)}
                    className="px-2.5 py-1 rounded bg-amber-500 hover:bg-amber-400 text-slate-
    950 font-semibold"
                  >
                    Workbench
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    ) : (
      <div className="space-y-4">
        {queue.map((item) => (
          <div
            key={item.id}
            className="rounded-lg bg-slate-950 border border-slate-800 p-4 grid grid-cols-2 md
    :grid-cols-4 gap-2.5 text-xs"
          >
            <div><strong>ID:</strong> #{item.id} ({item.product})</div>
            <div><strong>IEEE Fault Type:</strong> {item.classification}</div>
            <div><strong>Origin Phase:</strong> {item.phase || '--'}</div>
            <div><strong>Uniqueness:</strong> {item.uniqueness}</div>
            <div><strong>Originated Activity:</strong> {item.actoriginated || '--'}</div>
            <div><strong>Deliverable:</strong> {item.deliverable || '--'}</div>
            <div><strong>Detection Activity:</strong> {item.actfound || '--'}</div>
            <div><strong>Problem Status:</strong> {item.status}</div>
            <div><strong>Fix Effort:</strong> {item.fixeffort} hours</div>
            <div><strong>Cost of Fix:</strong> EUR {item.fixcost}</div>
            <div><strong>Total Defect Cost:</strong> EUR {item.costofdefect}</div>
            <div><strong>Submitted -> Assigned:</strong> {formatDeltaDays(item.submitted_to_ass
    igned_days)}</div>
            <div><strong>Submitted -> Fixed:</strong> {formatDeltaDays(item.submitted_to_fixed_
    days)}</div>
            <div><strong>Submitted -> Resolved:</strong> {formatDeltaDays(item.submitted_to_res
    olved_days)}</div>
            <div><strong>Assigned -> Fixed:</strong> {formatDeltaDays(item.assigned_to_fixed_da
    ys)}</div>
            <div><strong>Assigned -> Resolved:</strong> {formatDeltaDays(item.assigned_to_resol
    ved_days)}</div>
            <div className="col-span-2 md:col-span-4 flex justify-end pt-1">
              <button
                type="button"
                onClick={() => onOpenWorkbench(item.id)}
                className="px-3 py-1 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 fo
    nt-semibold"
              >
                Open Analyzer & Resolution Workbench
              </button>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>
);
};
```

```
================================================================================
SECTION 3.8 <DefectAnalyzerWorkbench /> (modern/frontend/src/components/DefectAnalyzerWorkbench.
    tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectAnalyzerWork
    bench.tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import {
  DefectPriority,
  DefectSeverity,
  DefectStatus,
  IEEE_1992_TAXONOMY,
  IeeeFaultType,
  OriginPhase,
  Uniqueness,
} from '../types/defman';
import { DefectResolutionForm } from './DefectResolutionForm';

/**
 * <DefectAnalyzerWorkbench />
 * Refactored from: form_analyze.php, form_analyzed.php, faulttypes.html, helpanalyze.html
 * Endpoint: PUT /api/defects/{id}/analyze
 */
export const DefectAnalyzerWorkbench: React.FC<{
  defectId: number;
  onClose?: () => void;
  onSaved?: () => void;
}> = ({ defectId, onClose, onSaved }) => {
  const [activeTab, setActiveTab] = useState<'analyze' | 'resolve'>('analyze');
  const [showTaxonomyModal, setShowTaxonomyModal] = useState(false);
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [form, setForm] = useState({
    devdesc: '',
    classification: IeeeFaultType.LOGIC as string,
    classification_other: '',
    dateoriginated: new Date().toISOString().slice(0, 10),
    phases: [OriginPhase.IMPLEMENTATION] as string[],
    actoriginated: '',
    deliverable: '',
    actfound: '',
    submitted: new Date().toISOString().slice(0, 10),
    assigned: new Date().toISOString().slice(0, 10),
    uniqueness: Uniqueness.NEW_DEFECT as string,
    severity: DefectSeverity.MAJOR,
    priority: 'most urgent',
    workaround: '',
    proposedfix: '',
    status: DefectStatus.OPENED,
  });

  useEffect(() => {
    defmanApi.getDefectById(defectId).then((res) => {
      if (res.analysis) {
        const a = res.analysis;
        const isStandardEnum = Object.values(IeeeFaultType).includes(
          a.classification as IeeeFaultType
        );
        setForm({
          devdesc: a.devdesc || '',
```

```
        classification: isStandardEnum
          ? (a.classification as string)
          : IeeeFaultType.OTHER,
        classification_other: isStandardEnum ? '' : (a.classification || ''),
        dateoriginated: a.dateoriginated || res.Occured,
        phases: a.phase ? a.phase.split(',').filter(Boolean) : [],
        actoriginated: a.actoriginated || '',
        deliverable: a.deliverable || '',
        actfound: a.actfound || '',
        submitted: a.submitted || res.Submitted,
        assigned: a.assigned || new Date().toISOString().slice(0, 10),
        uniqueness: a.uniqueness || Uniqueness.NEW_DEFECT,
        severity: a.severity || res.Severity,
        priority: a.priority || 'most urgent',
        workaround: a.workaround || '',
        proposedfix: a.proposedfix || '',
        status: a.status || DefectStatus.OPENED,
      });
    }
  });
}, [defectId]);

const togglePhase = (phase: string) => {
  setForm((prev) => ({
    ...prev,
    phases: prev.phases.includes(phase)
      ? prev.phases.filter((p) => p !== phase)
      : [...prev.phases, phase],
  }));
};

const handleAnalyzeSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setErrorMessage(null);
  setStatusMessage(null);
  try {
    const res = await defmanApi.analyzeDefect(defectId, form);
    setStatusMessage(res.message);
    onSaved?.();
  } catch (err) {
    setErrorMessage(err instanceof Error ? err.message : 'Failed to save analysis');
  }
};

return (
  <div className="bg-slate-900 border border-slate-700 rounded-xl p-6 text-slate-100 shadow-xl
   space-y-4">
    {/* Header & Navigation Bar matching 2004 top links: Defect analysis - Defect fix - Defect
   resolution */}
    <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-70
  0 pb-3">
      <div>
        <h2 className="text-lg font-bold text-amber-400">
          Defect Analyzer Workbench -- Defect ID #{defectId}
        </h2>
        <div className="flex items-center gap-2 mt-1 text-xs">
          <button
            type="button"
            onClick={() => setActiveTab('analyze')}
            className={`px-3 py-1 rounded font-semibold ${
              activeTab === 'analyze'
                ? 'bg-amber-500 text-slate-950'
                : 'bg-slate-800 text-slate-300'
            }`}
          >
            1. IEEE 1992 Fault Analysis (form_analyze.php)
```

```
          </button>
          <button
            type="button"
            onClick={() => setActiveTab('resolve')}
            className={'px-3 py-1 rounded font-semibold ${
              activeTab === 'resolve'
                ? 'bg-amber-500 text-slate-950'
                : 'bg-slate-800 text-slate-300'
            }'}
          >
            2. Fix Effort & Cost Resolution (form_deffix.php / form_defreso.php)
          </button>
        </div>
      </div>
      <div className="flex items-center gap-2">
        <button
          type="button"
          onClick={() => setShowTaxonomyModal((v) => !v)}
          className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700 text-amber-30
0 border border-slate-600"
        >
          {showTaxonomyModal ? 'Hide IEEE 1992 Faults' : 'IEEE 1992 Fault Reference (faulttype
s.html)'}
        </button>
        {onClose && (
          <button
            type="button"
            onClick={onClose}
            className="text-xs px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700 text-slate-
300"
          >
            Close
          </button>
        )}
      </div>
    </div>

    {showTaxonomyModal && (
      <div className="p-4 rounded-lg bg-slate-950 border border-amber-500/40 text-xs grid grid
-cols-1 md:grid-cols-2 gap-3 max-h-60 overflow-y-auto">
        {Object.entries(IEEE_1992_TAXONOMY).map(([category, subs]) => (
          <div key={category} className="space-y-1">
            <div className="font-bold text-amber-400">{category}</div>
            <ul className="list-disc list-inside text-slate-400 space-y-0.5">
              {subs.map((s) => (
                <li key={s}>{s}</li>
              ))}
            </ul>
          </div>
        ))}
      </div>
    )}

    {activeTab === 'analyze' ? (
      <form onSubmit={handleAnalyzeSubmit} className="space-y-4 text-xs">
        {errorMessage && (
          <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300">
            {errorMessage}
          </div>
        )}
        {statusMessage && (
          <div className="p-3 rounded bg-emerald-950 border border-emerald-600 text-emerald-30
0">
            {statusMessage}
          </div>
        )}
```

```
        <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
          <label className="block md:col-span-2">
            <span className="text-slate-300">
              Defect description by developer <span className="text-red-400">*</span>
            </span>
            <input
              type="text"
              required
              value={form.devdesc}
              onChange={(e) => setForm({ ...form, devdesc: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>

          <label className="block">
            <span className="text-slate-300">
              Problem status <span className="text-red-400">*</span>
            </span>
            <select
              value={form.status}
              onChange={(e) =>
                setForm({ ...form, status: e.target.value as DefectStatus })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              {Object.values(DefectStatus).map((st) => (
                <option key={st} value={st}>
                  {st}
                </option>
              ))}
            </select>
          </label>
        </div>

        {/* IEEE 1992 Classification + legacy toggleKentta Other field */}
        <div className="grid grid-cols-1 md:grid-cols-2 gap-3 bg-slate-950/60 p-3 rounded-lg b
order border-slate-800">
          <label className="block">
            <span className="text-slate-300">
              Classification of defect (IEEE, 1992) <span className="text-red-400">*</span>
            </span>
            <select
              disabled={form.classification_other.trim().length > 0}
              value={
                form.classification_other.trim().length > 0
                  ? IeeeFaultType.OTHER
                  : form.classification
              }
              onChange={(e) =>
                setForm({ ...form, classification: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5
disabled:opacity-50"
            >
              {Object.values(IeeeFaultType).map((ft) => (
                <option key={ft} value={ft}>
                  {ft}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Other classification (auto-locks dropdown when typed)
            </span>
```

```
            <input
              type="text"
              value={form.classification_other}
              onChange={(e) =>
                setForm({
                  ...form,
                  classification_other: e.target.value,
                  classification:
                    e.target.value.length > 0
                      ? IeeeFaultType.OTHER
                      : form.classification,
                })
              }
              placeholder="Custom fault classification..."
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
        </div>

        {/* Phase Originated Checkboxes */}
        <div>
          <span className="text-slate-300 block mb-1.5">
            Phase originated <span className="text-red-400">*</span>
          </span>
          <div className="grid grid-cols-2 md:grid-cols-6 gap-2 bg-slate-950/60 p-3 rounded-lg
    border border-slate-800">
            {Object.values(OriginPhase).map((phase) => (
              <label
                key={phase}
                className="flex items-center gap-2 cursor-pointer"
              >
                <input
                  type="checkbox"
                  checked={form.phases.includes(phase)}
                  onChange={() => togglePhase(phase)}
                  className="rounded border-slate-600"
                />
                <span>{phase}</span>
              </label>
            ))}
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
          <label className="block">
            <span className="text-slate-300">
              Date defect originated <span className="text-red-400">*</span>
            </span>
            <input
              type="date"
              required
              value={form.dateoriginated}
              onChange={(e) =>
                setForm({ ...form, dateoriginated: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Date submitted <span className="text-red-400">*</span>
            </span>
            <input
              type="date"
              required
              value={form.submitted}
```

```
            onChange={(e) => setForm({ ...form, submitted: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">
            Date assigned <span className="text-red-400">*</span>
          </span>
          <input
            type="date"
            required
            value={form.assigned}
            onChange={(e) => setForm({ ...form, assigned: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
        <label className="block">
          <span className="text-slate-300">
            Deliverable originated <span className="text-red-400">*</span>
          </span>
          <input
            type="text"
            required
            value={form.deliverable}
            onChange={(e) =>
              setForm({ ...form, deliverable: e.target.value })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">
            Activity which originated defect <span className="text-red-400">*</span>
          </span>
          <input
            type="text"
            required
            value={form.actoriginated}
            onChange={(e) =>
              setForm({ ...form, actoriginated: e.target.value })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">
            Activity that should have found defect <span className="text-red-400">*</span>
          </span>
          <input
            type="text"
            required
            value={form.actfound}
            onChange={(e) => setForm({ ...form, actfound: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
        <label className="block">
          <span className="text-slate-300">
            Uniqueness <span className="text-red-400">*</span>
          </span>
```

```
            <select
              value={form.uniqueness}
              onChange={(e) =>
                setForm({ ...form, uniqueness: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              {Object.values(Uniqueness).map((u) => (
                <option key={u} value={u}>
                  {u}
                </option>
              ))}
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Severity by developer <span className="text-red-400">*</span>
            </span>
            <select
              value={form.severity}
              onChange={(e) =>
                setForm({
                  ...form,
                  severity: e.target.value as DefectSeverity,
                })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={DefectSeverity.MINOR}>minor</option>
              <option value={DefectSeverity.MAJOR}>major</option>
              <option value={DefectSeverity.CRITICAL}>critical</option>
            </select>
          </label>
          <label className="block">
            <span className="text-slate-300">
              Priority by developer <span className="text-red-400">*</span>
            </span>
            <select
              value={form.priority}
              onChange={(e) => setForm({ ...form, priority: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value="most urgent">most urgent</option>
              <option value="average">average</option>
              <option value="less urgent">less urgent</option>
            </select>
          </label>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">Work around</span>
            <textarea
              rows={2}
              value={form.workaround}
              onChange={(e) =>
                setForm({ ...form, workaround: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">
              Proposed fix <span className="text-red-400">*</span>
            </span>
            <textarea
```

```
                rows={2}
                required
                value={form.proposedfix}
                onChange={(e) =>
                  setForm({ ...form, proposedfix: e.target.value })
                }
                className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
              />
            </label>
          </div>

          <div className="flex items-center justify-end gap-3 pt-2">
            <button
              type="submit"
              className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-s
    emibold"
            >
              Save Analysis
            </button>
          </div>
        </form>
      ) : (
        <DefectResolutionForm defectId={defectId} onSaved={onSaved} />
      )}
    </div>
  );
};
```

```
================================================================================
SECTION 3.9 <DefectResolutionForm /> (modern/frontend/src/components/DefectResolutionForm.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectResolutionFo
    rm.tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import { DefectStatus } from '../types/defman';

/**
 * <DefectResolutionForm />
 * Refactored from: form_deffix.php & form_defreso.php
 * Endpoint: PUT /api/defects/{id}/resolve
 */
export const DefectResolutionForm: React.FC<{
  defectId: number;
  onSaved?: () => void;
}> = ({ defectId, onSaved }) => {
  const [hourlyRateEur, setHourlyRateEur] = useState<number>(30); // 2004 helpanalyze.html defau
    lt: 30 EUR/h
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [form, setForm] = useState({
    status: DefectStatus.RESOLVED,
    fixdesc: '',
    fixstarted: new Date().toISOString().slice(0, 10),
    fixcomplete: new Date().toISOString().slice(0, 10),
    fiximpact: '',
    fixeffort: 2,
    fixcost: 60,
    resodesc: '',
    resocomplete: new Date().toISOString().slice(0, 10),
    resoclosed: new Date().toISOString().slice(0, 10),
    costofdefect: 240,
```

```
  });

  useEffect(() => {
    defmanApi.getDefectById(defectId).then((res) => {
      if (res.analysis) {
        const a = res.analysis;
        setForm({
          status: a.status || DefectStatus.RESOLVED,
          fixdesc: a.fixdesc || '',
          fixstarted: a.fixstarted || new Date().toISOString().slice(0, 10),
          fixcomplete: a.fixcomplete || new Date().toISOString().slice(0, 10),
          fiximpact: a.fiximpact || '',
          fixeffort: Number(a.fixeffort ?? 2),
          fixcost: Number(a.fixcost ?? 60),
          resodesc: a.resodesc || '',
          resocomplete: a.resocomplete || new Date().toISOString().slice(0, 10),
          resoclosed: a.resoclosed || new Date().toISOString().slice(0, 10),
          costofdefect: Number(a.costofdefect ?? 240),
        });
      }
    });
  }, [defectId]);

  const handleEffortChange = (effortHours: number) => {
    setForm((prev) => ({
      ...prev,
      fixeffort: effortHours,
      fixcost: Number((effortHours * hourlyRateEur).toFixed(2)),
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const res = await defmanApi.resolveDefect(defectId, form);
      setStatusMessage(res.message);
      onSaved?.();
    } catch (err) {
      setErrorMessage(err instanceof Error ? err.message : 'Resolution save failed');
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4 text-xs">
      {errorMessage && (
        <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300">
          {errorMessage}
        </div>
      )}
      {statusMessage && (
        <div className="p-3 rounded bg-emerald-950 border border-emerald-600 text-emerald-300">
          {statusMessage}
        </div>
      )}

      {/* Section 1: Defect Fix (form_deffix.php) */}
      <div className="p-4 rounded-lg bg-slate-950/70 border border-slate-800 space-y-3">
        <div className="font-bold text-amber-400">
          Part A -- Defect Fix & Effort Tracking (form_deffix.php)
        </div>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">Description of fix *</span>
            <textarea
```

```
            rows={2}
            required
            value={form.fixdesc}
            onChange={(e) => setForm({ ...form, fixdesc: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Deliverables impacted by fix</span>
          <textarea
            rows={2}
            value={form.fiximpact}
            onChange={(e) => setForm({ ...form, fiximpact: e.target.value })}
            placeholder="e.g. MMF 1.5 settings.cpp popup.cpp"
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
      </div>

      <div className="grid grid-cols-2 md:grid-cols-5 gap-3">
        <label className="block">
          <span className="text-slate-300">Date work started *</span>
          <input
            type="date"
            required
            value={form.fixstarted}
            onChange={(e) => setForm({ ...form, fixstarted: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Date work completed *</span>
          <input
            type="date"
            required
            value={form.fixcomplete}
            onChange={(e) => setForm({ ...form, fixcomplete: e.target.value })}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Effort expended (hours) *</span>
          <input
            type="number"
            step="0.5"
            min="0"
            required
            value={form.fixeffort}
            onChange={(e) => handleEffortChange(Number(e.target.value))}
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Rate (EUR /h helper)</span>
          <input
            type="number"
            step="5"
            value={hourlyRateEur}
            onChange={(e) => {
              const rate = Number(e.target.value);
              setHourlyRateEur(rate);
              setForm((prev) => ({
                ...prev,
                fixcost: Number((prev.fixeffort * rate).toFixed(2)),
              }));
            }}
```

```
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Cost of fix (EUR  EUR) *</span>
          <input
            type="number"
            step="0.01"
            min="0"
            required
            value={form.fixcost}
            onChange={(e) =>
              setForm({ ...form, fixcost: Number(e.target.value) })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
      </div>
    </div>

    {/* Section 2: Defect Resolution (form_defreso.php) */}
    <div className="p-4 rounded-lg bg-slate-950/70 border border-slate-800 space-y-3">
      <div className="font-bold text-amber-400">
        Part B -- Defect Resolution & Total Cost (form_defreso.php)
      </div>
      <label className="block">
        <span className="text-slate-300">Description of resolution *</span>
        <textarea
          rows={2}
          required
          value={form.resodesc}
          onChange={(e) => setForm({ ...form, resodesc: e.target.value })}
          className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
        />
      </label>

      <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
        <label className="block">
          <span className="text-slate-300">Resolution Status *</span>
          <select
            value={form.status}
            onChange={(e) =>
              setForm({ ...form, status: e.target.value as DefectStatus })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          >
            {Object.values(DefectStatus).map((st) => (
              <option key={st} value={st}>
                {st}
              </option>
            ))}
          </select>
        </label>
        <label className="block">
          <span className="text-slate-300">Date of resolution *</span>
          <input
            type="date"
            required
            value={form.resocomplete}
            onChange={(e) =>
              setForm({ ...form, resocomplete: e.target.value })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
          />
        </label>
        <label className="block">
```

```
            <span className="text-slate-300">Date closed *</span>
            <input
              type="date"
              required
              value={form.resoclosed}
              onChange={(e) => setForm({ ...form, resoclosed: e.target.value })}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">Total Cost of Defect (EUR  EUR) *</span>
            <input
              type="number"
              step="0.01"
              min="0"
              required
              value={form.costofdefect}
              onChange={(e) =>
                setForm({ ...form, costofdefect: Number(e.target.value) })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-2.5 py-1.5"
            />
          </label>
        </div>
      </div>

      <div className="flex justify-end">
        <button
          type="submit"
          className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-semib
    old"
        >
          Save Fix & Resolution Metrics
        </button>
      </div>
    </form>
  );
};



================================================================================
SECTION 3.10 <AdminUserApproval /> (modern/frontend/src/components/AdminUserApproval.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\AdminUserApproval.
    tsx
================================================================================

import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import { type AuthUser, UserLevel } from '../types/defman';

/**
 * <AdminUserApproval />
 * Refactored from: form_usermanagement.php, form_apply.php, form_newuser.php, randompassword.ph
    p
 * Endpoints: GET/POST/PATCH/DELETE /api/admin/users
 */
export const AdminUserApproval: React.FC = () => {
  const [users, setUsers] = useState<AuthUser[]>([]);
  const [applicants, setApplicants] = useState<AuthUser[]>([]);
  const [statusMessage, setStatusMessage] = useState<string | null>(null);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  const [newUser, setNewUser] = useState({
    username: '',
    password: '',
```

```
    email: '',
    userlevel: UserLevel.CUSTOMER,
  });

  const loadUsers = useCallback(async () => {
    const res = await defmanApi.getAdminUsers();
    setUsers(res.users);
    setApplicants(res.applicants);
    setNewUser((prev) => ({
      ...prev,
      password: prev.password || res.generated_password_suggestion,
    }));
  }, []);

  useEffect(() => {
    loadUsers();
  }, [loadUsers]);

  const handleApprove = async (user: AuthUser, level = UserLevel.CUSTOMER) => {
    if (!window.confirm('Are you SURE that you want to ACCEPT\nUser: ${user.login.toUpperCase()}
    ?')) {
      return;
    }
    const res = await defmanApi.patchAdminUser(user.id, {
      action: 'approve',
      userlevel: level,
    });
    setStatusMessage(res.message);
    loadUsers();
  };

  const handleDeleteOrDecline = async (user: AuthUser, isDecline = false) => {
    const verb = isDecline ? 'DECLINE' : 'DELETE';
    if (!window.confirm('Are you SURE that you want to ${verb}\nUser: ${user.login.toUpperCase()
    }?')) {
      return;
    }
    const res = await defmanApi.deleteAdminUser(user.id);
    setStatusMessage(res.message);
    loadUsers();
  };

  const handleRoleChange = async (userId: number, level: UserLevel) => {
    const res = await defmanApi.patchAdminUser(userId, {
      action: 'role',
      userlevel: level,
    });
    setStatusMessage(res.message);
    loadUsers();
  };

  const handleCreateUser = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMessage(null);
    setStatusMessage(null);
    try {
      const res = await defmanApi.createAdminUser(newUser);
      setStatusMessage(res.message);
      setNewUser({
        username: '',
        password: '',
        email: '',
        userlevel: UserLevel.CUSTOMER,
      });
      loadUsers();
    } catch (err) {
```

```
      setErrorMessage(err instanceof Error ? err.message : 'Failed to add user');
    }
  };

  const roleLabel = (level: UserLevel) => {
    switch (level) {
      case UserLevel.PUBLIC:
        return 'Public';
      case UserLevel.CUSTOMER:
        return 'Customer';
      case UserLevel.ANALYZER:
        return 'Analyzer';
      case UserLevel.ADMIN:
        return 'Admin';
      default:
        return 'Applying';
    }
  };

  return (
    <div className="space-y-6 text-slate-100">
      {errorMessage && (
        <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300 text-xs">
          {errorMessage}
        </div>
      )}
      {statusMessage && (
        <div className="p-3 rounded bg-emerald-950 border border-emerald-600 text-emerald-300 te
    xt-xs">
          {statusMessage}
        </div>
      )}

      {/* Active Users Table (form_usermanagement.php top table) */}
      <div className="bg-slate-900 border border-slate-700 rounded-xl p-5 shadow-lg space-y-3">
        <h2 className="text-lg font-bold text-amber-400">
          User Management ({users.length} Active Accounts)
        </h2>
        <div className="overflow-x-auto">
          <table className="w-full text-left text-xs border-collapse">
            <thead>
              <tr className="bg-slate-800 text-slate-300 border-b border-slate-700">
                <th className="p-2.5">ID</th>
                <th className="p-2.5">Username</th>
                <th className="p-2.5">Full Name</th>
                <th className="p-2.5">Email</th>
                <th className="p-2.5">Company</th>
                <th className="p-2.5">Phone</th>
                <th className="p-2.5">Userlevel</th>
                <th className="p-2.5">Status</th>
                <th className="p-2.5 text-right">Delete User</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-800">
              {users.map((u) => (
                <tr key={u.id} className="hover:bg-slate-800/50">
                  <td className="p-2.5 font-mono">#{u.id}</td>
                  <td className="p-2.5 font-semibold">{u.login}</td>
                  <td className="p-2.5 text-amber-300">{u.name || '--'}</td>
                  <td className="p-2.5">{u.email}</td>
                  <td className="p-2.5">{u.company || '--'}</td>
                  <td className="p-2.5">{u.phone || '--'}</td>
                  <td className="p-2.5">
                    <select
                      value={u.userlevel}
                      onChange={(e) =>
```

```
                    handleRoleChange(u.id, Number(e.target.value) as UserLevel)
                  }
                  className="rounded bg-slate-800 border border-slate-600 px-2 py-1"
                >
                  <option value={UserLevel.PUBLIC}>1 - Public</option>
                  <option value={UserLevel.CUSTOMER}>2 - Customer</option>
                  <option value={UserLevel.ANALYZER}>3 - Analyzer</option>
                  <option value={UserLevel.ADMIN}>4 - Admin</option>
                </select>
              </td>
              <td className="p-2.5">
                {u.logged === 1 ? (
                  <span className="text-emerald-400 font-semibold">Online</span>
                ) : (
                  <span className="text-red-400 font-semibold">Offline</span>
                )}
              </td>
              <td className="p-2.5 text-right">
                {u.userlevel !== UserLevel.ADMIN && (
                  <button
                    type="button"
                    onClick={() => handleDeleteOrDecline(u, false)}
                    className="px-2.5 py-1 rounded bg-red-900/70 hover:bg-red-800 text-red-2
00"
                  >
                    Delete
                  </button>
                )}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>

  {/* Applying Users Queue (form_usermanagement.php bottom table + form_apply.php) */}
  <div className="bg-slate-900 border border-slate-700 rounded-xl p-5 shadow-lg space-y-3">
    <h3 className="text-base font-bold text-amber-400">
      Applying Users Approval Queue ({applicants.length})
    </h3>
    {applicants.length === 0 ? (
      <p className="text-xs text-slate-400">No pending user applications.</p>
    ) : (
      <div className="overflow-x-auto">
        <table className="w-full text-left text-xs border-collapse">
          <thead>
            <tr className="bg-slate-800 text-slate-300 border-b border-slate-700">
              <th className="p-2.5">ID</th>
              <th className="p-2.5">Username</th>
              <th className="p-2.5">Full Name</th>
              <th className="p-2.5">Email</th>
              <th className="p-2.5">Company</th>
              <th className="p-2.5">Phone</th>
              <th className="p-2.5">Status</th>
              <th className="p-2.5 text-right">Actions</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-800">
            {applicants.map((app) => (
              <tr key={app.id}>
                <td className="p-2.5 font-mono">#{app.id}</td>
                <td className="p-2.5 font-semibold">{app.login}</td>
                <td className="p-2.5 text-amber-300">{app.name || '--'}</td>
                <td className="p-2.5">{app.email}</td>
                <td className="p-2.5">{app.company || '--'}</td>
```

```
                  <td className="p-2.5">{app.phone || '--'}</td>
                  <td className="p-2.5">{roleLabel(app.userlevel)}</td>
                  <td className="p-2.5 text-right space-x-2">
                    <button
                      type="button"
                      onClick={() => handleDeleteOrDecline(app, true)}
                      className="px-2.5 py-1 rounded bg-red-900/70 hover:bg-red-800 text-red-2
00"
                    >
                      Decline
                    </button>
                    <button
                      type="button"
                      onClick={() => handleApprove(app, UserLevel.CUSTOMER)}
                      className="px-2.5 py-1 rounded bg-emerald-600 hover:bg-emerald-500 text-
white font-semibold"
                    >
                      Accept (Customer)
                    </button>
                    <button
                      type="button"
                      onClick={() => handleApprove(app, UserLevel.ANALYZER)}
                      className="px-2.5 py-1 rounded bg-amber-500 hover:bg-amber-400 text-slat
e-950 font-semibold"
                    >
                      Accept (Analyzer)
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>

    {/* Add New User Form (form_newuser.php) */}
    <form
      onSubmit={handleCreateUser}
      className="bg-slate-900 border border-slate-700 rounded-xl p-5 shadow-lg space-y-3 max-w
-xl text-xs"
    >
      <h3 className="text-base font-bold text-amber-400">
        Add New User (form_newuser.php)
      </h3>
      <div className="grid grid-cols-2 gap-3">
        <label className="block">
          <span className="text-slate-300">Username (Alphanumeric) *</span>
          <input
            type="text"
            required
            value={newUser.username}
            onChange={(e) =>
              setNewUser({ ...newUser, username: e.target.value })
            }
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <label className="block">
          <span className="text-slate-300">Generated / Custom Password *</span>
          <input
            type="text"
            required
            value={newUser.password}
            onChange={(e) =>
              setNewUser({ ...newUser, password: e.target.value })
```

```
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5 fo
    nt-mono"
            />
          </label>
        </div>
        <div className="grid grid-cols-2 gap-3">
          <label className="block">
            <span className="text-slate-300">Email *</span>
            <input
              type="email"
              required
              value={newUser.email}
              onChange={(e) =>
                setNewUser({ ...newUser, email: e.target.value })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">Userlevel *</span>
            <select
              value={newUser.userlevel}
              onChange={(e) =>
                setNewUser({
                  ...newUser,
                  userlevel: Number(e.target.value) as UserLevel,
                })
              }
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            >
              <option value={UserLevel.PUBLIC}>Public (Level 1)</option>
              <option value={UserLevel.CUSTOMER}>Customer (Level 2)</option>
              <option value={UserLevel.ANALYZER}>Analyzer (Level 3)</option>
              <option value={UserLevel.ADMIN}>Admin (Level 4)</option>
            </select>
          </label>
        </div>
        <button
          type="submit"
          className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-semib
    old"
        >
          Add User
        </button>
      </form>
    </div>
  );
};
```

```
================================================================================
SECTION 3.11 <ProductManager /> (modern/frontend/src/components/ProductManager.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\ProductManager.tsx
================================================================================

import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from '../api/client';
import type { Product } from '../types/defman';

/**
 * <ProductManager />
 * Refactored from: form_productdetails.php
 * Endpoints: GET/POST/DELETE /api/admin/products
 */
```

```
export const ProductManager: React.FC = () => {
  const [products, setProducts] = useState<Product[]>([]);
  const [name, setName] = useState('');
  const [message, setMessage] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  const loadProducts = useCallback(async () => {
    const res = await defmanApi.getProducts();
    setProducts(res.products);
  }, []);

  useEffect(() => {
    loadProducts();
  }, [loadProducts]);

  const handleAdd = async (e: React.FormEvent) => {
    e.preventDefault();
    setMessage(null);
    setError(null);
    try {
      const res = await defmanApi.createProduct(name);
      setMessage(res.message);
      setName('');
      loadProducts();
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to create product');
    }
  };

  const handleDelete = async (p: Product) => {
    if (!window.confirm(`Are you SURE that you want to DELETE\nProduct: ${p.name} ?`)) {
      return;
    }
    setMessage(null);
    setError(null);
    try {
      const res = await defmanApi.deleteProduct(p.id);
      setMessage(res.message);
      loadProducts();
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to delete product');
    }
  };

  return (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6 text-slate-100">
      <div className="bg-slate-900 border border-slate-700 rounded-xl p-5 shadow-lg space-y-3">
        <h2 className="text-lg font-bold text-amber-400">
          All Products ({products.length})
        </h2>
        {error && (
          <div className="p-2.5 rounded bg-red-950 border border-red-600 text-amber-300 text-xs"
    >
            {error}
          </div>
        )}
        {message && (
          <div className="p-2.5 rounded bg-emerald-950 border border-emerald-600 text-emerald-30
    0 text-xs">
            {message}
          </div>
        )}
        <table className="w-full text-left text-xs border-collapse">
          <thead>
            <tr className="bg-slate-800 text-slate-300 border-b border-slate-700">
              <th className="p-2.5">ID</th>
```

```
              <th className="p-2.5">Product Name</th>
              <th className="p-2.5">Reports</th>
              <th className="p-2.5 text-right">Delete</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-800">
            {products.map((p) => (
              <tr key={p.id}>
                <td className="p-2.5 font-mono">#{p.id}</td>
                <td className="p-2.5 font-medium">{p.name}</td>
                <td className="p-2.5 font-mono text-slate-400">
                  {p.report_count ?? 0}
                </td>
                <td className="p-2.5 text-right">
                  <button
                    type="button"
                    onClick={() => handleDelete(p)}
                    className="px-2.5 py-1 rounded bg-red-900/70 hover:bg-red-800 text-red-200"
                  >
                    Delete
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>

      <form
        onSubmit={handleAdd}
        className="bg-slate-900 border border-slate-700 rounded-xl p-5 shadow-lg space-y-3 h-fit
     text-xs"
      >
        <h3 className="text-lg font-bold text-amber-400">Add New Product</h3>
        <label className="block">
          <span className="text-slate-300">Product name (max 100 chars) *</span>
          <input
            type="text"
            maxLength={100}
            required
            value={name}
            onChange={(e) => setName(e.target.value)}
            placeholder="e.g. MMF 2.0"
            className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
          />
        </label>
        <div className="flex gap-2">
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
    ibold"
          >
            Add product
          </button>
          <button
            type="button"
            onClick={() => setName('')}
            className="px-4 py-2 rounded bg-slate-800 hover:bg-slate-700 text-slate-300"
          >
            Clear
          </button>
        </div>
      </form>
    </div>
  );
};
```

```
================================================================================
SECTION 3.12 <UserProfileSettings /> (modern/frontend/src/components/UserProfileSettings.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\UserProfileSetting
    s.tsx
================================================================================

import React, { useEffect, useState } from 'react';
import { defmanApi } from '../api/client';

/**
 * <UserProfileSettings />
 * Refactored from: form_userdetails.php (Password settings, Email settings, Personal settings)
 * Endpoint: GET / PUT /api/users/me
 */
export const UserProfileSettings: React.FC = () => {
  const [email, setEmail] = useState('');
  const [name, setName] = useState('');
  const [company, setCompany] = useState('');
  const [phone, setPhone] = useState('');
  const [changePassword, setChangePassword] = useState('');
  const [verifyPassword, setVerifyPassword] = useState('');

  const [message, setMessage] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    defmanApi.getProfile().then((res) => {
      setEmail(res.email || '');
      setName(res.name || '');
      setCompany(res.company || '');
      setPhone(res.phone || '');
    });
  }, []);

  const handleUpdate = async (
    e: React.FormEvent,
    section: 'password' | 'email' | 'personal'
  ) => {
    e.preventDefault();
    setMessage(null);
    setError(null);
    try {
      const payload =
        section === 'password'
          ? {
              section,
              change_password: changePassword,
              verify_password: verifyPassword,
            }
          : section === 'email'
          ? { section, email }
          : { section, name, company, phone };

      const res = await defmanApi.updateProfile(payload);
      setMessage(res.message);
      if (section === 'password') {
        setChangePassword('');
        setVerifyPassword('');
      }
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Update failed');
    }
  };
```

```
  return (
    <div className="space-y-4 text-slate-100 text-xs">
      {error && (
        <div className="p-3 rounded bg-red-950 border border-red-600 text-amber-300">
          {error}
        </div>
      )}
      {message && (
        <div className="p-3 rounded bg-emerald-950 border border-emerald-600 text-emerald-300">
          {message}
        </div>
      )}

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        {/* Password Settings */}
        <form
          onSubmit={(e) => handleUpdate(e, 'password')}
          className="bg-slate-900 border border-slate-700 rounded-xl p-5 space-y-3"
        >
          <h3 className="text-base font-bold text-amber-400">Password Settings</h3>
          <label className="block">
            <span className="text-slate-300">Change password</span>
            <input
              type="password"
              required
              value={changePassword}
              onChange={(e) => setChangePassword(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">Verify new password</span>
            <input
              type="password"
              required
              value={verifyPassword}
              onChange={(e) => setVerifyPassword(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
  ibold"
          >
            Update Password
          </button>
        </form>

        {/* Email Settings */}
        <form
          onSubmit={(e) => handleUpdate(e, 'email')}
          className="bg-slate-900 border border-slate-700 rounded-xl p-5 space-y-3"
        >
          <h3 className="text-base font-bold text-amber-400">Email Settings</h3>
          <label className="block">
            <span className="text-slate-300">Change Email</span>
            <input
              type="email"
              required
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
```

```
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
    ibold"
          >
            Update Email
          </button>
        </form>

        {/* Personal Settings */}
        <form
          onSubmit={(e) => handleUpdate(e, 'personal')}
          className="bg-slate-900 border border-slate-700 rounded-xl p-5 space-y-3"
        >
          <h3 className="text-base font-bold text-amber-400">Personal Settings</h3>
          <label className="block">
            <span className="text-slate-300">Full name</span>
            <input
              type="text"
              value={name}
              onChange={(e) => setName(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">Name of Company</span>
            <input
              type="text"
              maxLength={50}
              value={company}
              onChange={(e) => setCompany(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <label className="block">
            <span className="text-slate-300">Phonenumber</span>
            <input
              type="text"
              maxLength={50}
              value={phone}
              onChange={(e) => setPhone(e.target.value)}
              className="mt-1 w-full rounded bg-slate-800 border border-slate-600 px-3 py-1.5"
            />
          </label>
          <button
            type="submit"
            className="px-4 py-2 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font-sem
    ibold"
          >
            Update Personal Info
          </button>
        </form>
      </div>
    </div>
  );
};
```

```
================================================================================
SECTION 3.13 PixiJS v8 WebGPU <DefectTriageCanvas /> (modern/frontend/src/components/DefectTriag
    eCanvas.tsx)
File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\components\DefectTriageCanvas
    .tsx
================================================================================
```

```
import React, { useEffect, useRef, useState } from 'react';
import {
  Application,
  Container,
  type FederatedPointerEvent,
  Graphics,
  Text,
} from 'pixi.js';
import { defmanApi } from '../api/client';
import {
  type AnalyzerQueueItem,
  DefectSeverity,
  DefectStatus,
  IEEE_FAULT_HEX_COLORS,
  IeeeFaultType,
} from '../types/defman';

interface TriageLaneConfig {
  id: 'open' | 'analyzing' | 'fixed' | 'resolved';
  label: string;
  targetStatus: DefectStatus;
  xStart: number;
  xEnd: number;
}

const CANVAS_WIDTH = 1200;
const CANVAS_HEIGHT = 680;
const LANE_WIDTH = CANVAS_WIDTH / 4;

const TRIAGE_LANES: readonly TriageLaneConfig[] = [
  {
    id: 'open',
    label: 'OPEN (Submitted / Postponed)',
    targetStatus: DefectStatus.SUBMITTED,
    xStart: 0,
    xEnd: LANE_WIDTH,
  },
  {
    id: 'analyzing',
    label: 'ANALYZING (Assigned / Opened)',
    targetStatus: DefectStatus.OPENED,
    xStart: LANE_WIDTH,
    xEnd: LANE_WIDTH * 2,
  },
  {
    id: 'fixed',
    label: 'FIXED (Dev Complete)',
    targetStatus: DefectStatus.FIXED,
    xStart: LANE_WIDTH * 2,
    xEnd: LANE_WIDTH * 3,
  },
  {
    id: 'resolved',
    label: 'RESOLVED (QA Verified / Closed)',
    targetStatus: DefectStatus.RESOLVED,
    xStart: LANE_WIDTH * 3,
    xEnd: CANVAS_WIDTH,
  },
];

function mapStatusToLaneIndex(status: DefectStatus | string): number {
  switch (status) {
    case DefectStatus.SUBMITTED:
    case DefectStatus.POSTPONED:
      return 0;
    case DefectStatus.ASSIGNED:
```

```
    case DefectStatus.OPENED:
      return 1;
    case DefectStatus.FIXED:
      return 2;
    case DefectStatus.RESOLVED:
    case DefectStatus.CLOSED:
      return 3;
    default:
      return 0;
  }
}

function getFaultHexColor(classification: string): number {
  const match = Object.values(IeeeFaultType).find(
    (ft) => ft.toLowerCase() === (classification || '').toLowerCase()
  );
  return match
    ? IEEE_FAULT_HEX_COLORS[match]
    : IEEE_FAULT_HEX_COLORS[IeeeFaultType.OTHER];
}

function getSeverityGlowColor(severity: DefectSeverity | string, priority: string): {
  color: number;
  alpha: number;
  width: number;
} {
  if (severity === DefectSeverity.CRITICAL) {
    return { color: 0xff2e4c, alpha: 0.9, width: 5 };
  }
  if (severity === DefectSeverity.MAJOR || priority === 'most urgent') {
    return { color: 0xf59e0b, alpha: 0.75, width: 3.5 };
  }
  return { color: 0x38bdf8, alpha: 0.45, width: 2 };
}

function computeNodeRadius(effortHours: number, monetaryCostEur: number): number {
  const effortComponent = Math.sqrt(Math.max(1, effortHours)) * 4.5;
  const costComponent = Math.log10(Math.max(10, monetaryCostEur + 10)) * 7;
  return Math.min(54, Math.max(20, Math.round(12 + effortComponent + costComponent)));
}

/**
 * <DefectTriageCanvas />
 * Powered by PixiJS v8 (WebGPU primary with WebGL fallback)
 * Renders defects from form_alist.php / form_raportlist.php with:
 *   - Node fill color by IEEE (1992) Fault Type (defecttype.txt)
 *   - Outer ring glow by Severity & Priority
 *   - Node radius scaled proportionally to Fix Effort (Hours) & Monetary Cost (EUR)
 *   - Pan/zoom viewport, hover telemetry tooltip, click-to-open <DefectAnalyzerWorkbench />,
 *     and drag-and-drop across status lanes triggering PUT /api/defects/{id}/resolve
 */
export const DefectTriageCanvas: React.FC<{
  items: AnalyzerQueueItem[];
  onSelectDefectForWorkbench: (defectId: number) => void;
  onStatusChanged: () => void;
}> = ({ items, onSelectDefectForWorkbench, onStatusChanged }) => {
  const hostRef = useRef<HTMLDivElement | null>(null);
  const [hoveredItem, setHoveredItem] = useState<{
    item: AnalyzerQueueItem;
    screenX: number;
    screenY: number;
  } | null>(null);
  const [rendererBackend, setRendererBackend] = useState<string>('Initializing WebGPU...');
  const [zoomLevel, setZoomLevel] = useState<number>(1);

  useEffect(() => {
```

```
    const hostElement = hostRef.current;
    if (!hostElement) return;

    let destroyed = false;
    const app = new Application();

    const initPixiCanvas = async () => {
      await app.init({
        preference: 'webgpu',
        antialias: true,
        width: CANVAS_WIDTH,
        height: CANVAS_HEIGHT,
        backgroundColor: 0x090d16,
        resolution: window.devicePixelRatio || 1,
        autoDensity: true,
      });

      if (destroyed) {
        app.destroy(true);
        return;
      }

      hostElement.innerHTML = '';
      hostElement.appendChild(app.canvas);

      const rendererType =
        app.renderer.name?.toLowerCase().includes('webgpu') ? 'WebGPU' : 'WebGL2';
      setRendererBackend(`PixiJS v8 (${rendererType})`);

      // Root world container supporting pan & zoom
      const world = new Container();
      app.stage.addChild(world);

      // 1. Draw Status Lane Columns & Headers
      const lanesGraphics = new Graphics();
      world.addChild(lanesGraphics);

      TRIAGE_LANES.forEach((lane, idx) => {
        const bgColor = idx % 2 === 0 ? 0x0f172a : 0x111c33;
        lanesGraphics
          .rect(lane.xStart, 0, LANE_WIDTH, CANVAS_HEIGHT)
          .fill({ color: bgColor, alpha: 0.85 })
          .stroke({ color: 0x1e293b, width: 1.5 });

        const headerText = new Text({
          text: lane.label,
          style: {
            fontFamily: 'Inter, monospace',
            fontSize: 12,
            fontWeight: 'bold',
            fill: 0xfbbf24,
          },
        });
        headerText.position.set(lane.xStart + 16, 16);
        world.addChild(headerText);
      });

      // 2. Group defects by lane and render interactive nodes
      const laneCounts = [0, 0, 0, 0];

      items.forEach((defect) => {
        const laneIndex = mapStatusToLaneIndex(defect.status);
        const slotIndex = laneCounts[laneIndex]++;
        const col = slotIndex % 2;
        const row = Math.floor(slotIndex / 2);
```

```
        const initialX =
          TRIAGE_LANES[laneIndex].xStart + 90 + col * 125;
        const initialY = 95 + row * 115;

        const radius = computeNodeRadius(
          Number(defect.fixeffort || 0),
          Number(defect.costofdefect || defect.fixcost || 0)
        );
        const fillHex = getFaultHexColor(defect.classification);
        const glow = getSeverityGlowColor(defect.severity, defect.priority);

        const nodeContainer = new Container();
        nodeContainer.position.set(initialX, initialY);
        nodeContainer.eventMode = 'static';
        nodeContainer.cursor = 'grab';

        // Outer severity/priority glow ring + IEEE 1992 Fault Type circle
        const circleGfx = new Graphics();
        circleGfx
          .circle(0, 0, radius + glow.width + 3)
          .fill({ color: glow.color, alpha: 0.18 })
          .stroke({ color: glow.color, width: glow.width, alpha: glow.alpha });

        circleGfx
          .circle(0, 0, radius)
          .fill({ color: fillHex, alpha: 0.92 })
          .stroke({ color: 0xffffff, width: 1.5, alpha: 0.8 });

        nodeContainer.addChild(circleGfx);

        // Node label (#ID + Effort/Cost)
        const idLabel = new Text({
          text: '#${defect.id}\nEUR ${Number(defect.costofdefect || defect.fixcost || 0).toFixed(0)
    }',
          style: {
            fontFamily: 'monospace',
            fontSize: 10,
            fontWeight: 'bold',
            fill: 0xffffff,
            align: 'center',
          },
        });
        idLabel.anchor.set(0.5);
        nodeContainer.addChild(idLabel);

        // Drag & Click Interaction State
        let isDragging = false;
        let pointerMovedDistance = 0;
        let dragStartPos = { x: 0, y: 0 };

        nodeContainer.on('pointerover', (e: FederatedPointerEvent) => {
          setHoveredItem({
            item: defect,
            screenX: e.global.x,
            screenY: e.global.y,
          });
        });

        nodeContainer.on('pointerout', () => {
          if (!isDragging) {
            setHoveredItem(null);
          }
        });

        nodeContainer.on('pointerdown', (e: FederatedPointerEvent) => {
          e.stopPropagation();
```

```
        isDragging = true;
        pointerMovedDistance = 0;
        nodeContainer.cursor = 'grabbing';
        nodeContainer.alpha = 0.85;
        dragStartPos = world.toLocal(e.global);
      });

      nodeContainer.on('globalpointermove', (e: FederatedPointerEvent) => {
        if (!isDragging) return;
        const currentLocal = world.toLocal(e.global);
        const dx = currentLocal.x - dragStartPos.x;
        const dy = currentLocal.y - dragStartPos.y;
        pointerMovedDistance += Math.hypot(dx, dy);
        nodeContainer.position.x += dx;
        nodeContainer.position.y += dy;
        dragStartPos = currentLocal;
      });

      const finishInteraction = async () => {
        if (!isDragging) return;
        isDragging = false;
        nodeContainer.cursor = 'grab';
        nodeContainer.alpha = 1;

        // If minimal movement, treat as a click -> open <DefectAnalyzerWorkbench />
        if (pointerMovedDistance < 6) {
          onSelectDefectForWorkbench(defect.id);
          return;
        }

        // Determine target status lane from drop X coordinate
        const dropX = Math.max(0, Math.min(CANVAS_WIDTH - 1, nodeContainer.position.x));
        const targetLaneIndex = Math.min(
          TRIAGE_LANES.length - 1,
          Math.floor(dropX / LANE_WIDTH)
        );
        const targetLane = TRIAGE_LANES[targetLaneIndex];

        // Snap X into lane center band
        nodeContainer.position.x =
          targetLane.xStart + LANE_WIDTH / 2;

        if (targetLane.targetStatus !== defect.status) {
          await defmanApi.resolveDefect(defect.id, {
            status: targetLane.targetStatus,
          });
          onStatusChanged();
        }
      };

      nodeContainer.on('pointerup', finishInteraction);
      nodeContainer.on('pointerupoutside', finishInteraction);

      world.addChild(nodeContainer);
    });

    // 3. Stage Panning & Wheel Zooming
    let stagePanning = false;
    let panLast = { x: 0, y: 0 };

    app.stage.eventMode = 'static';
    app.stage.hitArea = app.screen;

    app.stage.on('pointerdown', (e: FederatedPointerEvent) => {
      stagePanning = true;
      panLast = { x: e.global.x, y: e.global.y };
```

```
    });

    app.stage.on('globalpointermove', (e: FederatedPointerEvent) => {
      if (!stagePanning) return;
      world.position.x += e.global.x - panLast.x;
      world.position.y += e.global.y - panLast.y;
      panLast = { x: e.global.x, y: e.global.y };
    });

    app.stage.on('pointerup', () => {
      stagePanning = false;
    });

    const onWheel = (e: WheelEvent) => {
      e.preventDefault();
      const factor = e.deltaY < 0 ? 1.1 : 0.9;
      const nextScale = Math.min(2.2, Math.max(0.55, world.scale.x * factor));
      world.scale.set(nextScale);
      setZoomLevel(Number(nextScale.toFixed(2)));
    };

    app.canvas.addEventListener('wheel', onWheel, { passive: false });
  };

  initPixiCanvas();

  return () => {
    destroyed = true;
    try {
      app.destroy(true, { children: true });
    } catch {
      // Safe cleanup if unmount occurs mid-init
    }
  };
}, [items, onSelectDefectForWorkbench, onStatusChanged]);

return (
  <div className="bg-slate-900 border border-slate-700 rounded-xl p-4 text-slate-100 shadow-xl
   space-y-3 relative">
    <div className="flex flex-wrap items-center justify-between gap-3">
      <div>
        <h2 className="text-lg font-bold text-amber-400">
          Visual Defect Triage & IEEE (1992) Cost Canvas
        </h2>
        <p className="text-xs text-slate-400">
          Drag nodes across lanes (Open -> Analyzing -> Fixed -> Resolved) to trigger{' '}
          <code className="text-amber-300">PUT /api/defects/&#123;id&#125;/resolve</code>, or
  click any node to open{' '}
          <code className="text-amber-300">&lt;DefectAnalyzerWorkbench /&gt;</code>.
        </p>
      </div>
      <div className="flex items-center gap-3 text-xs">
        <span className="px-2.5 py-1 rounded bg-slate-800 border border-slate-700 font-mono te
  xt-emerald-400">
          {rendererBackend}
        </span>
        <span className="px-2.5 py-1 rounded bg-slate-800 border border-slate-700 font-mono">
          Zoom: {Math.round(zoomLevel * 100)}%
        </span>
      </div>
    </div>

    {/* IEEE 1992 Color Legend */}
    <div className="flex flex-wrap items-center gap-2 text-[11px] bg-slate-950/70 p-2.5 rounde
  d-lg border border-slate-800">
      <span className="font-semibold text-slate-400 mr-1">
```

```
          IEEE (1992) Fault Color:
        </span>
        {Object.entries(IEEE_FAULT_HEX_COLORS).map(([fault, hex]) => (
          <span
            key={fault}
            className="flex items-center gap-1.5 px-2 py-0.5 rounded bg-slate-900 border border-
    slate-800"
          >
            <span
              className="w-2.5 h-2.5 rounded-full inline-block"
              style={{
                backgroundColor: `#${hex.toString(16).padStart(6, '0')}`,
              }}
            />
            <span>{fault}</span>
          </span>
        ))}
      </div>

      {/* PixiJS v8 Viewport Host */}
      <div
        ref={hostRef}
        className="w-full overflow-hidden rounded-lg border border-slate-800 bg-slate-950"
      />

      {/* Hover Telemetry Tooltip */}
      {hoveredItem && (
        <div
          className="pointer-events-none absolute z-30 bg-slate-950/95 border border-amber-500/6
    0 rounded-lg p-3 text-xs shadow-2xl max-w-xs space-y-1"
          style={{
            left: Math.min(CANVAS_WIDTH - 280, hoveredItem.screenX + 24),
            top: Math.max(80, hoveredItem.screenY + 40),
          }}
        >
          <div className="font-bold text-amber-400">
            #{hoveredItem.item.id} -- {hoveredItem.item.title}
          </div>
          <div className="text-slate-300">
            <strong>Product:</strong> {hoveredItem.item.product}
          </div>
          <div>
            <strong>IEEE 1992 Fault:</strong> {hoveredItem.item.classification}
          </div>
          <div>
            <strong>Severity / Priority:</strong> {hoveredItem.item.severity} /{' '}
            {hoveredItem.item.priority}
          </div>
          <div>
            <strong>Fix Effort:</strong> {hoveredItem.item.fixeffort}h |{' '}
            <strong>Fix Cost:</strong> EUR {hoveredItem.item.fixcost}
          </div>
          <div>
            <strong>Total Cost of Defect:</strong> EUR {hoveredItem.item.costofdefect}
          </div>
          <div className="text-[10px] text-amber-300 pt-1">
            Click node to open Workbench * Drag across lanes to update status
          </div>
        </div>
      )}
    </div>
  );
};
```

==========

SECTION 3.14 React 19 SPA Root <App /> (modern/frontend/src/App.tsx)

File: C:\Users\lkarh\Downloads\defman-2026\src\modern\frontend\src\App.tsx

==========

```
import React, { useCallback, useEffect, useState } from 'react';
import { defmanApi } from './api/client';
import { AdminUserApproval } from './components/AdminUserApproval';
import { AnalyzerQueue } from './components/AnalyzerQueue';
import { CustomerReportForm } from './components/CustomerReportForm';
import { DefectAnalyzerWorkbench } from './components/DefectAnalyzerWorkbench';
import { DefectDetailModal } from './components/DefectDetailModal';
import { DefectReportGrid } from './components/DefectReportGrid';
import { DefectTriageCanvas } from './components/DefectTriageCanvas';
import { ProductManager } from './components/ProductManager';
import { PublicReportForm } from './components/PublicReportForm';
import { UserProfileSettings } from './components/UserProfileSettings';
import {
  type AnalyzerQueueItem,
  type AuthUser,
  UserLevel,
} from './types/defman';

type ActiveView =
  | 'canvas'
  | 'reports'
  | 'analyzer-queue'
  | 'submit-report'
  | 'public-report'
  | 'products'
  | 'users'
  | 'profile';

/**
 * Modern React 19 SPA Root
 * Replaces monolithic 2004 index.php + menubar.php + form_blank.php + opr.css
 */
export const App: React.FC = () => {
  const [currentUser, setCurrentUser] = useState<AuthUser | null>(null);
  const [activeView, setActiveView] = useState<ActiveView>('public-report');
  const [selectedDefectModalId, setSelectedDefectModalId] = useState<number | null>(null);
  const [workbenchDefectId, setWorkbenchDefectId] = useState<number | null>(null);
  const [queueItems, setQueueItems] = useState<AnalyzerQueueItem[]>([]);

  // Auth box state
  const [loginUsername, setLoginUsername] = useState('');
  const [loginPassword, setLoginPassword] = useState('');
  const [authError, setAuthError] = useState<string | null>(null);

  const loadAnalyzerCanvasData = useCallback(async () => {
    if (currentUser && currentUser.userlevel >= UserLevel.ANALYZER) {
      const res = await defmanApi.getAnalyzerQueue('all');
      setQueueItems(res.queue);
    }
  }, [currentUser]);

  useEffect(() => {
    loadAnalyzerCanvasData();
  }, [loadAnalyzerCanvasData]);

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    setAuthError(null);
    try {
      const res = await defmanApi.login(loginUsername, loginPassword);
      localStorage.setItem('defman_token', res.token);
```

```
      setCurrentUser(res.user);
      setActiveView(
        res.user.userlevel >= UserLevel.ANALYZER ? 'canvas' : 'submit-report'
      );
    } catch (err) {
      setAuthError(err instanceof Error ? err.message : 'Authentication failed');
    }
  };

  const handleLogout = async () => {
    await defmanApi.logout();
    localStorage.removeItem('defman_token');
    setCurrentUser(null);
    setActiveView('public-report');
  };

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 font-sans">
      {/* Top Menubar (Refactored from menubar.php) */}
      <header className="bg-slate-900 border-b border-slate-800 px-6 py-3">
        <div className="max-w-7xl mx-auto flex flex-wrap items-center justify-between gap-4">
          <div className="flex items-center gap-3">
            <span className="text-xl font-black tracking-tight text-amber-400">
              DefMan 2026
            </span>
            <span className="text-xs px-2 py-0.5 rounded bg-slate-800 text-slate-400 border bord
    er-slate-700">
              IEEE (1992) Fault & Cost Analytics
            </span>
          </div>

          {currentUser ? (
            <div className="flex flex-wrap items-center gap-2 text-xs">
              <span className="text-slate-300 mr-2">
                Logged in: <strong className="text-amber-300">{currentUser.login}</strong> (Leve
    l {currentUser.userlevel})
              </span>

              {currentUser.userlevel >= UserLevel.CUSTOMER && (
                <button
                  type="button"
                  onClick={() => setActiveView('submit-report')}
                  className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
                >
                  Defect Report
                </button>
              )}

              {currentUser.userlevel >= UserLevel.CUSTOMER && (
                <button
                  type="button"
                  onClick={() => setActiveView('reports')}
                  className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
                >
                  List All Reports
                </button>
              )}

              {currentUser.userlevel >= UserLevel.ANALYZER && (
                <>
                  <button
                    type="button"
                    onClick={() => setActiveView('canvas')}
                    className="px-3 py-1.5 rounded bg-amber-500/20 text-amber-300 border border-
    amber-500/40 hover:bg-amber-500/30 font-semibold"
                  >
```

```
                WebGPU Triage Canvas
              </button>
              <button
                type="button"
                onClick={() => setActiveView('analyzer-queue')}
                className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
              >
                List All Analyzes
              </button>
            </>
          )}

          {currentUser.userlevel === UserLevel.ADMIN && (
            <>
              <button
                type="button"
                onClick={() => setActiveView('products')}
                className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
              >
                Product Management
              </button>
              <button
                type="button"
                onClick={() => setActiveView('users')}
                className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
              >
                User Management
              </button>
            </>
          )}

          {currentUser.userlevel > UserLevel.PUBLIC && (
            <button
              type="button"
              onClick={() => setActiveView('profile')}
              className="px-3 py-1.5 rounded bg-slate-800 hover:bg-slate-700"
            >
              Own Settings
            </button>
          )}

          <button
            type="button"
            onClick={handleLogout}
            className="px-3 py-1.5 rounded bg-red-900/60 hover:bg-red-800 text-red-200"
          >
            Logout
          </button>
        </div>
      ) : (
        <form onSubmit={handleLogin} className="flex items-center gap-2 text-xs">
          {authError && <span className="text-red-400">{authError}</span>}
          <input
            type="text"
            placeholder="Username (e.g. zuck, julia, sabrin)"
            value={loginUsername}
            onChange={(e) => setLoginUsername(e.target.value)}
            className="rounded bg-slate-800 border border-slate-700 px-2.5 py-1"
          />
          <input
            type="password"
            placeholder="Password"
            value={loginPassword}
            onChange={(e) => setLoginPassword(e.target.value)}
            className="rounded bg-slate-800 border border-slate-700 px-2.5 py-1"
          />
```

```
            <button
              type="submit"
              className="px-3 py-1 rounded bg-amber-500 hover:bg-amber-400 text-slate-950 font
    -semibold"
            >
              Login &gt;
            </button>
          </form>
        )}
      </div>
    </header>

    {/* Main Content Area */}
    <main className="max-w-7xl mx-auto p-6 space-y-6">
      {workbenchDefectId !== null && (
        <DefectAnalyzerWorkbench
          defectId={workbenchDefectId}
          onClose={() => setWorkbenchDefectId(null)}
          onSaved={() => {
            loadAnalyzerCanvasData();
          }}
        />
      )}

      {activeView === 'public-report' && <PublicReportForm />}

      {activeView === 'submit-report' && currentUser && (
        <CustomerReportForm currentUser={currentUser} />
      )}

      {activeView === 'reports' && (
        <DefectReportGrid
          onSelectDefect={(id) => setSelectedDefectModalId(id)}
          onAnalyzeDefect={
            currentUser && currentUser.userlevel >= UserLevel.ANALYZER
              ? (id) => setWorkbenchDefectId(id)
              : undefined
          }
        />
      )}

      {activeView === 'analyzer-queue' && (
        <AnalyzerQueue
          onOpenWorkbench={(id) => setWorkbenchDefectId(id)}
        />
      )}

      {activeView === 'canvas' && (
        <DefectTriageCanvas
          items={queueItems}
          onSelectDefectForWorkbench={(id) => setWorkbenchDefectId(id)}
          onStatusChanged={loadAnalyzerCanvasData}
        />
      )}

      {activeView === 'products' && <ProductManager />}
      {activeView === 'users' && <AdminUserApproval />}
      {activeView === 'profile' && <UserProfileSettings />}
    </main>

    {/* Defect Report Detail Modal (form_defrap.php) */}
    {selectedDefectModalId !== null && (
      <DefectDetailModal
        defectId={selectedDefectModalId}
        canEdit={Boolean(
          currentUser && currentUser.userlevel >= UserLevel.ANALYZER
```

```
          )}
          onClose={() => setSelectedDefectModalId(null)}
          onOpenAnalyze={(id) => {
            setSelectedDefectModalId(null);
            setWorkbenchDefectId(id);
          }}
        />
      )}
    </div>
  );
};

export default App;
```